Onyx — Kernel internals

Onyx · multi-process OS on Raspberry Pi 4

This document describes the inner workings of the **Onyx kernel** (the code under `kernel/`). It does **not** describe Circle, which is used as the HAL/driver stack; we only point out the seams with Circle where necessary.

All file paths are relative to the repository root. The values (addresses, constants) come from the code; the layout constants live in `kernel/include/kern/layout.h`.

Contents

1. Scope: ours vs. Circle
2. Boot sequence
3. Address-space map
4. Memory management: `CAddressSpace`
5. Scheduling
6. Exceptions and vectors
7. ELF loader and process model
8. The kapi ABI table
9. Stream / stdio subsystem
10. Graphics subsystem (GUI)
11. Network subsystem (WLAN, TCP/IP, NTP)
12. Sound and the second core
13. Post-mortem debug console
14. App cores (cores 2 and 3)
15. The GPU (V3D)
16. The RAM: volume
17. GPIO: the 40-pin header (v92)
18. The volumes: USB sticks, eject, format

---

1. Scope: ours vs. Circle

Written by Onyx	Reused from Circle
Per-process address spaces (`mm/addrspace.cpp`)	64 KB page allocator (`palloc`/`pfree`)
Scheduler (`sched/scheduler.cpp`, `sched/task.cpp`)	`taskswitch.S`, mutex/semaphore/event
`VBAR_EL1` vectors, trap frame (`arch/aarch64/`)	`InterruptHandler` (GIC + EOI), `FIQStub`
ELF loader (`proc/elf.cpp`)	EMMC + FatFs, USB HID
ABI table + kapi impl. (`sys/`)	`C2DGraphics` (framebuffer), `CTimer`

Written by Onyx	Reused from Circle
GUI: rendering, compositor + window manager, shared surfaces (`gui/`)	`CCharGenerator` (bitmap font)

The kernel is linked against Circle's static libraries. **Two Circle files are replaced** (`lib/sched/scheduler.cpp` and `task.cpp`): because our `.o` files appear in `OBJS` *before* the `--start-group` archives, the linker resolves `CScheduler`/`CTask` from our versions and never pulls in those from `libsched.a`.

A **shadow header** (`kernel/compat/circle/sched/scheduler.h`) is placed **before** `circle/include` on the include path, so that `<circle/sched/scheduler.h>` resolves to ours (which adds `OnTimerTick()`, the notion of a time slice, etc., while keeping Circle's public + friend API).

---

2. Boot sequence

The entry point is `main()` (`kernel/main.cpp`), called by Circle's `sysinit` **after** it has already: switched the CPU to EL1, enabled the MMU (64 KB granule), initialized the GIC, run the static C++ constructors.

All the logic lives in the **`CKernel`** class (`kernel/kernel.cpp`):

`CKernel::Initialize()` — in order

1. **Text console**: `m_Screen` (HDMI) then `m_Serial` (115200). The boot log goes to the HDMI screen so it is visible **without a serial cable**. A `CLogSwitch` routes the logger's output (see §13).
2. **Interrupts + timer**: `m_Interrupt.Initialize()`, `m_Timer.Initialize()`.
3. **Exception vectors**: `install_vectors()` installs our `VBAR_EL1` (`KVectorTable`) over Circle's.
4. **Scheduler**:
   - `m_Timer.RegisterPeriodicHandler(PeriodicTick)` — wires the tick to `OnTimerTick()`.
   - `AddrSpaceInit()` — captures the kernel's TTBR0 (base of the shared L2 table).
   - `RegisterTaskSwitchHandler(AddressSpaceTaskSwitch)` — on each task switch, activates the new task's address space.
   - `RegisterTaskTerminationHandler(AddressSpaceTaskTerminate)` — frees the address space when a task ends.
5. **ABI table**: `KApiTableInit()` fills the `kapi` pointer table (see §8), then `El0Init()` builds from it the EL0 table and code page every process maps (§6, *Protected mode*); core 0's EL0 controls (`El0CoreInit`). Then the **cores 1–3** (`COnyxCores`): core 1 the sound (§12), cores 2–3 the app cores (§14), core 3 the network instead with `netcore=1` (§11); `dispdma=` / `gpudirect=` read here.
6. **Graphics + SD**: `m_2DGraphics.Initialize()` (32 bpp framebuffer); `m_EMMC` + `f_mount()` (FatFs) of `SD:` (partition 1, the boot FAT32 one), then of **`SD1:` … `SD3:`** (partitions 2–4) when they hold a FAT or **exFAT** file system (see *Volumes* in §8). The theme from `SD:/etc/theme.txt`; the mouse cursor is built in (`BuiltinCursor`, the 12×19 arrow that was `SD:skins/mousecur.bin` — the card has no `skins/` folder any more: the window chrome is drawn user-side).
7. **RAM:**: `RamFsInit()` — the RAM volume, its size from `system.ini`'s `ramfs=` (§16).
8. **USB**: `m_USB.Initialize()` (mouse + HID keyboard, hot-plug).

`CKernel::Run()` — the service loop

1. Logs machine info.

2. Launches the **init program** (`StartAutostart`): the ELF named by the `cmdline.txt` option `init=` (e.g. `init=SD:/bin/init`), defaulting to `SD:bin/init` when the option is absent. The kernel does not parse any launch list itself — init (PID-1 style) reads `SD:/etc/autostart` and starts everything from there (by default `voronoy`, which paints the wallpaper then exits, `setup`, the daemons and the remote access: see the file). Pointing `init=` at a different ELF swaps the whole userland launcher (e.g. a recovery shell) without rebuilding the kernel.
3. Launches the **kernel service tasks**:
    - `CCompositorTask` — presents the screen at up to ~60 Hz, **only what changed** (dirty rectangles): every visual change says where — `ScreenDirtyRect` for a part (`kapi_present`/`SYS_present` and chrome: the caller's window; window add / remove / raise / move / resize / alpha: its old and new places; the cursor: its old and new 16×16 spots, with the drag badge) or `ScreenDirty` for all of it (wallpaper, full screen end, drag & drop start / end). Rectangles that touch are merged (at most 16; more, or over 2/3 of the screen, = the whole screen) and `g_nScreenGen` is bumped. The task takes the damage (`ScreenTakeDamage`) and for each rectangle composites with the screen image **clipped** to it (`GImage::SetClip`: every blit and fill stays inside) and sends just that rectangle to the display (`C2DGraphics::UpdateDisplay (x, y, w, h)`, our Circle patches 6 / 12: the frame buffer's 2D DMA reads the rectangle in place, source and destination strides; `CCompositorTask::Present` starts it with `UpdateDisplayAsync` and **yields** to the other tasks until its interrupt says it is done, instead of spinning); the whole screen only when told, plus a safety refresh every 2 s. A window animating (a game, an emulator) costs its own area instead of the whole screen, a mouse move two cursor-sized spots. An idle desktop thus costs almost no CPU (the heartbeat's fps shows the real composites). It is created **before** `init` (no boot-log pause any more) and the main task hands it the CPU at once with `CScheduler::YieldTo()` (scan starts at that task instead of the round-robin next), so it presents before the first app's window. Starting it late — after a 6 s pause, with a single GUI app already running — used to hang the boot.
    - `CReaperTask` — reaps terminated tasks every ~50 ms (frees `CTask` + stack + address space outside the scheduler core). It also blinks the green ACT LED as a headless sign of life: 1 s period while the network is down, 0.2 s once `NetIsUp()`; a frozen LED means the scheduler stopped.
    - `CInputTask` (if USB is present) — pumps keyboard/mouse events to the window manager; every 100 ms it also looks for new gamepads (`upad1..4`) and USB MIDI devices (`umidi1..4`, v68: §8 *USB MIDI*), also after a hot plug. It takes **every** USB keyboard (`ukbd1..4`, each with the layout snapshot), not only `ukbd1`: a wireless mouse's receiver (Logitech's nano receiver, e.g. the M187's) has a boot-keyboard interface of its own, which may enumerate first and take `ukbd1` — the real keyboard, `ukbd2`, was then ignored. Each keyboard's raw report is kept and the window manager gets them merged (modifiers ORed, held keys joined), so one keyboard's empty report does not release the keys held on another. The mouse is still `mouse1` only. **Print Screen** (USB usage 0x46) is no longer the kernel's (kapi v90): the held keys' report goes to **Elegant** (`KAPI_WS_IN_HELD_USB`), which sees the key's press (`print_screen`, `user/Servers/common/serve.cpp`) and tells the **Screenshot** app through its service `screenshot` (message type 1, `"now"` or `"window <id>"` — with Alt, the window that has the keyboard, which Elegant knows), else starts it (`kapi_exec_as ("SD:/apps/screenshot.app/main", "--now" | "--window <id>", "screenshot")`). No key event reaches the app in front (the keymap gives Print Screen no character).

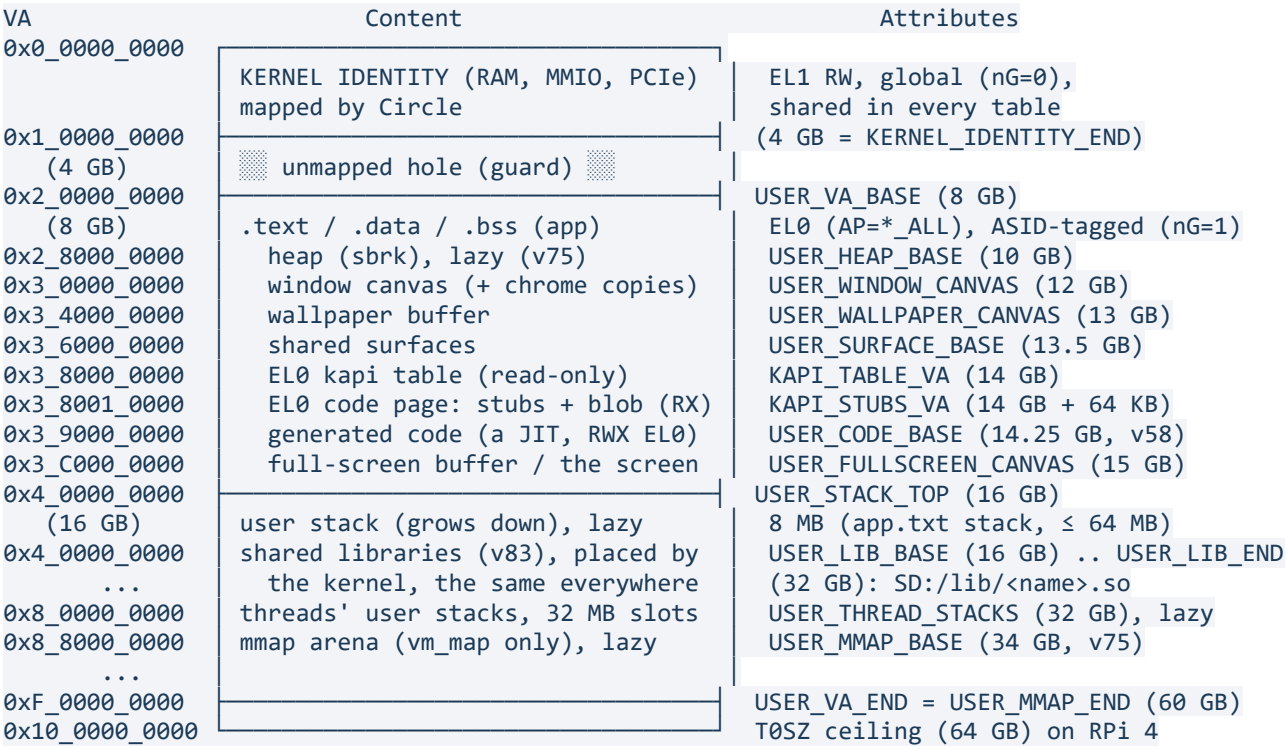
- **`CGuiWatchdogTask`** (skipped with `watchdog=0` in `cmdline.txt`) — once a second, checks the GUI and writes to the kernel log (`kmsg`): `compositor STALLED` when `CWindowManager::FrameCount()` has not moved for 2 s (with every task's `name:state`), `app '<title>' NOT PUMPING events` when a window has queued events and its owner has not called `PopEvent` for 2 s (plus the matching "recovered" lines), and a **heartbeat** every `heartbeat=` seconds from `cmdline.txt` (default 5, 0 = off): uptime, fps, mouse/key deltas, tasks by state, windows with queued/dropped events. Counters live in `CWindowManager` (frames, `OnMouse`/`OnKey` calls, `Snapshot()`) and `CWindow` (`QueuedEvents`, `DroppedEvents`, `LastPumpTicks`).

4. The main task runs an idle loop (`MsSleep`).

---

3. Address-space map

One `TTBR0` per process. The kernel (identity) must stay at the **low VAs** (because identity requires VA == PA), so **kernel and user coexist in `TTBR0`**: the kernel L2 entries are **shared**, the user L2/L3 are **per process**.



- **64 KB granule, EL1&0 stage-1 only**, `TTBR1` disabled.
- 2 levels: one **L2 entry = 512 MB** (points to an L3 table), one **L3 page = 64 KB**.
- **8-bit ASID** (256 contexts), taken from `TTBR0_EL1[63:48]`.
- `USER_LOAD_BASE = USER_VA_BASE`: apps are linked at 8 GB (see user.ld).
- **Lazy** (v75, §4 *Demand paging*): the heap, the stacks and the mmap arena are regions whose pages are filled on first touch; the image, the canvases, surfaces, the kapi pages, the code arena, the full-screen buffers and the sound ring are mapped at once (`FIXED` / `IMAGE`).

---

4. Memory management: `CAddressSpace`

Source: `kernel/mm/addrspace.cpp`, `kernel/include/kern/addrspace.h`.

A `CAddressSpace` object = a process. It owns: the L2 table, the ASID, a PID, the window pointer, the stdin/stdout streams, the process handle, the exit code, and the argv/cwd string.

Where the app pages come from, by board RAM. An app's own frames (ELF segments, heap, stack, `vm_map`, `code_alloc`) are taken by `MapNewPage` with `palloc_high()`: the **high zone** (1–3 GB, plus the RAM reclaimed above 3 GB) on a 2 / 4 / 8 GB Pi 4. A **1 GB** Pi 4 has no high zone: `palloc_high()` falls back to the low pager (`PAGE_RESERVE`, 256 MB), which the apps then share with every process's page tables. The kernel heap (window canvases, the wallpaper, the full-screen buffer, the GPU's memory, ~680 MB) is the same on every board. So Onyx runs on 1 GB — the desktop and the small apps take a few MB each — but the apps all together have ~250 MB instead of ~2 GB or more: the big ones (Jet Browser, the N64 / GameCube emulators) can run short (`sbrk` / `new` return 0). the Task Manager's Memory tab then shows an app pool of 0 (`ram_detail`: the high zone only).

Construction

1. **Allocates a fresh L2 table** (`palloc`, one 64 KB page).
2. **Copies the kernel L2 descriptors** from the template captured at boot (`memcpy` of the whole L2 page). The kernel entries point to the **same shared L3 tables**; the L2 slots covering the user area stay invalid (zero).
3. **Allocates an ASID** (1..255; 0 is reserved for the kernel/global).
4. **Maps the EL0 `kapi` table** read-only at `KAPI_TABLE_VA` (14 GB) with `KPAGE_ATTR_APP_RODATA`, and the **EL0 code page** (the system-call stubs and the user-side routines, §6 *Protected mode*) at `KAPI_STUBS_VA` with `KPAGE_ATTR_APP_CODE`. Both physical pages are kernel globals shared by every process (not "owned" by it → never freed at destruction).

`MapPage(VA, PA, attrs, bOwned)`

Maps a 64 KB page (VA and PA aligned to 64 KB):

- `GetOrCreateL3(L2_INDEX(VA))`: if the L2 entry is invalid, allocates a fresh L3 table (zeroed, `DSB ISHST`) and writes the L2 table descriptor.
- Fills the L3 page descriptor: `AttrIndx`, `AP`, `SH`, `AF=1`, `nG`, output = PA, `PXN`, `UXN`. The software bit `PAGE_SW_OWNED` (the *Ignored* field, bit 55) marks the frames to free at destruction. Since v75 both descriptors are built aside and **stored as one 64-bit word**: an app core running a job of this space may walk the table at that very moment.

Page attribute matrix (`layout.h` presets)

Use	AttrIndx	AP	nG	PXN	UXN
`KPAGE_ATTR_APP_CODE` (app code, the EL0 code page)	NORMAL	RO_ALL	1	1	**0** (exec. EL0)
`KPAGE_ATTR_APP_DATA` (data/heap/stacks/canv	NORMAL	RW_ALL	1	1	1

Use	AttrIndx	AP	nG	PXN	UXN
as/surfaces)					
`KPAGE_ATTR_APP_RODATA` (the EL0 kapi table; v77: a program's read-only segment without `PF_X`)	NORMAL	RO_ALL	1	1	1
`KPAGE_ATTR_APP_RWX` (generated code, v58 `code_alloc`)	NORMAL	RW_ALL	1	1	**0** (exec. EL0)
`KPAGE_ATTR_APP_SCREEN` (`fullscreen_direct`, the displayed framebuffer)	COHERENT (uncached)	RW_ALL	1	1	1

App pages are **accessible at EL0** (`AP=*_ALL`; the kernel works in them in place: the A72 has no PAN), executable **at EL0 only** for code (`UXN=0`, `PXN=1` everywhere: the kernel never runs app code), and `nG=1` (ASID-tagged) → isolation between applications. The identity region keeps Circle's `AP=RW_EL1`: out of the apps' reach (§6, *Protected mode*).

Activation and context switch

```
// CAddressSpace::Activate()
u64 ttbr0 = MAKE_TTBR0(phys(L2), asid);   // = phys | (asid << 48)
asm volatile ("msr ttbr0_el1, %0; isb" :: "r"(ttbr0) : "memory");
```

No TLB flush on switch (TLB tagged by ASID). The `AddressSpaceTaskSwitch` hook calls `Activate()` for app tasks, or `ActivateKernelAddressSpace()` for kernel tasks (which have no address space).

Demand paging (v75)

Source: `kernel/sys/vm.cpp`, `kern/vm.h`; the plan: `docs/POSIX-PLAN.md` §3.1.

- **Regions.** Each space has a `TVmSpace`: a sorted array of `TVma { start, end, prot, kind }` (binary-searched, at most 4096). Lazy kinds: `ANON` (`vm_map`, only in the mmap arena `[34 GB, 60 GB)`), `HEAP` (`[USER_HEAP_BASE, break rounded up)`), `STACK` (the main stack below 16 GB, 8 MB by default; each thread's at the top of its 32 MB slot). Eager kinds, noted for `vm_query` only: `IMAGE` (the ELF loader), `FIXED` (`MapContig`: canvases, surfaces, the sound ring, the kapi pages, the full-

screen buffers; the code arena). Nothing is mapped for a lazy region until it is touched. (v76) `SHM`: a shared memory object mapped by `shm_map` (in the arena, lazy, its frames the object's and not owned by the space: §8 *v76: IPC*).

- **The fill** (`VmFaultIn`): the region must be lazy and allow the access; a zeroed frame from `palloc_high`, a `DSB ISHST`, the PTE (`AP` from the region: `RW_ALL`, `RO_ALL`, or `RO_EL1` for `PROT_NONE` — the frame kept, the app shut out), `DSB`. A software walk: it never yields and works for any space. **No TLBI** (an invalid entry is never cached). Who fills: an **EL0 translation fault** (`sys/el0.cpp`: the access retried by the `eret`; a permission fault the region now allows — a stale TLB entry — gets a local `TLBI VAE1`); a **kernel probe** (`UserReadable` / `UserWritable`, §6); a **failed fault-safe copy** (retried in C after the fill, never in the exception); the **app-core pager** (§14); `vm_map (POPULATE)` and `MADV_WILLNEED` (a yield every 64 pages).
- **Out of memory.** A fill is refused when the app pool (the high zone + the low pager, whose free lists `ram_detail` reads) is under **16 MB** (`VM_RESERVE`). At EL0 the process is killed: kmsg `vm: <name> (pid N) killed: out of memory (page fault at …, … KB resident)`, the notice " ran out of memory", term reason `KAPI_PROC_OOM`, status −9. In a kapi the call fails with its error value; on an app core the job ends `CORE_FAULT`. **Heuristic overcommit**: one `sbrk` growth or one writable `vm_map` (without `NORESERVE`) larger than the free pool − 16 MB fails (`sbrk` → −1, `vm_map` → −`ENOMEM`); nothing else is counted. (Pi round 1: Circle's `CPageAllocator::Allocate` advanced its bump pointer before it found its region full and never stepped it back, so each failed attempt — `palloc_high` tries the full segment 0 before every page it takes elsewhere — made `GetFreeSpace` and `meminfo` one page short for ever: an OOM kill seemed to lose ~1.1 GB and the OOM check fired with half the memory free. Fixed in the Circle fork, docs/05 §24.)
- **Frames leaving** (`vm_unmap`, `MADV_DONTNEED`/`FREE`, an `sbrk` shrink, an ended thread's stack) and **protections lowered** (`vm_protect`): the PTE written, `DSB ISHST`, `TLBI VAE1IS (va >> 12 | ASID << 48)` per page (`TLBI ASIDE1IS` above 64 pages), `DSB ISH`, `ISB` — inner shareable, so cores 2–3 are told — and only then the frames freed (`pfree`, after waking any `wait_word` sleeper on them). Protection changes need no break-before-make (the output address never changes).
- **Pins and the deferred zap.** A probe pins its range for the calling task until its system call returns (`Syscall` → `VmUnpinTask`; one union per task, 40 slots). A page under **another** task's pin is not dropped (nor write-protected) at once: its PTE gets a software bit (`ZAP` / `SYNC`, bits 56–57), the range is queued, and the last unpin over it drops it (or gives it its region's protection). The region change itself is immediate. So a kapi working in place (a file read yielding between 64 KB chunks, a blocked pipe read) never sees its buffer vanish, nobody waits, and the kernel never touches a freed frame. A range mapped again before that adopts the pages left, zeroed.
- **The EL1 safety net**: `SyncHandlerEL1`, after the uaccess fixup, fills a lazy page that kernel code touched outside the helpers (core 0, the current app, a translation fault at a user VA), logs `vm: kernel touched an unpopulated user page at pc …` once, and retries — instead of a panic.
- **Costs.** A touched page costs 64 KB (a thread's first stack page, a lone `malloc` page); one L3 table (64 KB, low pager) per 512 MB slot touched; reservations cost nothing. Zeroing a page takes about 10 µs. `vm_stats` reports a process's resident bytes (page tables included), its lazy and writable VA, its fills and its page tables.

Destruction (and an important pitfall)

When destroying the address space:

1. Signals EOF on stdout, releases the streams, marks the process handle as terminated.

2. **Invalidates the TLB for this ASID** *before* freeing anything: `tlbi aside1is, Xt ; dsb ish ; isb`. (Reusing a page-table frame while the walk is still cached would break translations.)
3. Removes and destroys the window.
4. (v75) `VmTeardown`: the regions and the pending zaps freed (after the app cores and the GPU let go of the space).
5. Walks the **user** L2/L3 slots and frees the frames marked `PAGE_SW_OWNED` (pages whose zap was deferred included).
6. (v77) Drops its reference on its program's image (§7): the image's shared frames are mapped **not owned**, so step 5 leaves them; the last process of a program that is not preloaded frees them here, after the tables that named them. (When an app core did not stop, the space is leaked as before, and so is this reference.)

> ⚠ **Pitfall: never free an L3 table shared with the kernel.** Because each address space **copies** the kernel L2 descriptors, some L2 entries point to the **same** L3 tables as the kernel. Before freeing an L3 from the user area, the destructor **compares** the table address against the reference kernel L2's and **skips** any shared table. Freeing a shared kernel table would corrupt the translations of the *entire system*. (See also the design note "per-AS L2 copies the kernel L2".)

---

5. Scheduling

Source: `kernel/sched/scheduler.cpp`, `kernel/sched/task.cpp`, `kernel/compat/circle/sched/scheduler.h`.

Preemptive scheduling (track A)

The kernel **preempts application code**. A 100 Hz timer tick drives `OnTimerTick`, which counts down the running task's time slice (`SCHED_SLICE_TICKS` = 20 ms) and sets a reschedule flag when it expires. Every app runs at **EL0** (§6, *Protected mode*): an IRQ taken from EL0 enters through `El0IrqEntry`, which builds the app's full frame (all registers, FP/SIMD) **on the task's own kernel stack** and runs Circle's interrupt handler there; `El0IrqExit` then, when the slice is over, calls `OnPreempt` + `Yield` right on that stack — the task is parked **exactly like a voluntary yielder** (one resume path), and is resumed by `El0Return`'s `eret` to the interrupted instruction. That is the **only** preemption point. (Before kapi v74 apps ran at EL1t on their task's stack, and a `PreemptTrampoline` redirected the IRQ's return to switch there; it is gone.)

An IRQ taken at EL1 — kernel code, including an app's kapi call — never switches tasks: the **kernel is non-preemptive** (the classic Unix model). A long, non-yielding *kernel* loop would still block other tasks — drop a `if (IsReschedPending()) Yield();` cooperative point into any such loop if one appears. Tasks also still switch **voluntarily** (`Yield`, `MsSleep`, `present`, `wait`, …), unchanged.

Preemption points in file I/O. The long kernel operations an app can ask for are cut into pieces with a `Yield` between them: `kapi_read` / `kapi_save_file` move at most 64 KB per `f_read` / `f_write` (`ChunkedRead` / `ChunkedWrite` in `sys/kapi.cpp`), the ELF loader 128 KB. Without them one read of a 28 MB file (a Doom WAD, a GBA ROM, a newlib app's `fopen`, which reads the whole file) stopped every other task — the compositor, the cursor, the sound feeders — for seconds. Between two pieces the FatFs volume lock is free, so other tasks' file calls get through; the caller's buffer stays valid (its address space is active again when it resumes).

File-system speed. The FatFs disk layer keeps a 1 MB **sector cache** of the single-sector reads (the FAT, the folders), write-through (`docs/05` §8; `sdcache=0` turns it off): opening files and walking folders stopped asking the card for every sector. And `f_read` / `f_write` move a file's contiguous clusters in **one** SD command (`docs/05` §10), instead of one per cluster (with 512-byte clusters, one per 512 bytes: 2 MB/s). A read of 1 MB or more logs its time split (`fs: read ... SD: N commands, waiting, data port, other tasks`). The card runs in High Speed (50 MHz, `docs/05` §9; `sdhs=0` = 25 MHz). The boot log says what is in use (`SD card mounted (SD:): ...`); `/bin/fsbench` measures.

Yielding SD waits. The SD driver also yields while the card keeps it waiting: after a 2 ms spin (a normal command ends within that: yielding for it cost a whole time slice per command, 2 MB/s), `CEMMCDevice::TimeoutWait` calls `OnyxDriverWait ()` (a weak hook of our Circle fork, `docs/05` §7), which `Yield`s if IRQs are on. The FatFs volume lock is held during that wait, so it is a **sleeping** lock (`OnyxFsLockTake` / `OnyxFsLockGive`, `sys/fslock.cpp`, re-entrant, its waiters `Yield`): with Circle's spin lock a second task entering FatFs would spin forever against a holder it never lets run. The holder is in a **no-kill section** (`CScheduler::EnterNoKill` / `LeaveNoKill`): `TerminateTask` on it only marks it, and it ends in `LeaveNoKill`, once the lock is free — a dead owner would lock the card forever. (Before: 100–200 ms freezes each time the menu bar read the `app.txt` files, or an app opened a file in a big directory.) Many **short** waits in a row also yield (2026-10-01): a long multi-block write (one SD command for a file's contiguous clusters, the card taking a few hundred µs per block, each wait well under 2 ms) kept core 0 for 200 ms (`stall: jet:cache ran 213 ms without yielding`, PC in `TimeoutWait`). Each turn of a shorter wait calls `OnyxDriverPoll ()` (a second weak hook, `docs/05` §7), which `Yield`s once the task has run **10 ms** (`DRIVER_SLICE_US`) since its last `Yield` (`CScheduler::GetLastYield`), under the same conditions (a scheduler on this core, IRQs on, not core 1's crash dump). Between two blocks of a transfer is a safe point: the volume lock is held, and the SD host holds a PIO transfer until it is served. Reads (40 MB/s) yield about every 400 KB, writes every 50–100 KB. Nothing outside FatFs drives the SD card while tasks run: the boot's `CEMMCDevice::Initialize` comes before the mount, and core 1's crash dump writes raw sectors only with core 0 stopped (it never yields).

Stall watchdog. To find the kernel code that still keeps the CPU too long, the scheduler notes the time of every `Yield()`. When the running task (not idle) has not passed through `Yield()` for more than `SCHED_STALL_US` (100 ms), the IRQ exit path (`KernelIRQExit` → `CScheduler::StallSample`) samples the interrupted PC and LR every `SCHED_STALL_SAMPLE_US` (50 ms, up to 8 samples). The task's next `Yield()` closes the report (task name, duration) into a small ring, and the reaper task writes it to the kernel log:

```
stall: doom ran 850 ms without yielding; pc/lr: 1a2b40/1a2a10 1a2b48/1a2a10 ...
```

Kernel addresses resolve with `kernel/kernel8-rpi4.map` (or `aarch64-none-elf-addr2line -f -e kernel/kernel8-rpi4.elf <pc>`). No sample at all ("IRQs masked") means the task ran with interrupts off the whole time. (Its first catch: `12dbe4/138044` = `LeaveCritical` from `CTerminalDevice::Write` — the boot-console log writes, see §11 `MuteNormal`.)

CPU hogs vs yielders (dynamic priority). Circle's network and Wi-Fi code waits in `Yield()` loops (`qlock`/`sleep`/`tsleep` in `addon/wlan/p9proc.cpp`, the `CNetTask` `Process` loop, socket send/recv — also inside an app's `kapi_*` call, e.g. `vncd`). In plain round-robin, each of those yields hands the CPU to a CPU-bound app for a full slice, so the loops crawl. Driver timeouts then expire and the Wi-Fi link drops (the `spin` test app took the network down, and VNC/telnet with it). So the scheduler lowers the priority of the tasks the timer has to preempt:

- The preemption (`El0IrqExit`, `sys/el0.cpp`) calls `CScheduler::OnPreempt()`, which bumps the task's **preemption streak** (`m_nPreemptStreak[slot]`). Any voluntary `Yield` (a kapi wait, `msleep`, `present`, a kernel task's loop) resets it.
- A task with a streak ≥ `SCHED_HOG_STREAK` (2) is a **CPU hog**. An app that computes more than a slice now and then but yields in between (`vncd` encoding a frame, then sending it) is not one.
- A hog's preemption opens a **burst** of `SCHED_BURST_US` (60 ms). During the burst `GetNextTask` **skips the hogs** (and idle): kernel tasks and apps that yield are served first. The burst ends early once none of them is ready.
- A hog also runs with a **one-tick slice** (≤ 10 ms instead of `SCHED_SLICE_TICKS`), so the kernel's yield loops never wait more than ~10 ms behind it.

A hog thus keeps roughly 15 % of the CPU while others need it, and all of it otherwise. Both are tunable at boot for A/B testing: `cmdline.txt` `slice=` (ticks) and `hogsched=0` (plain round-robin) → `CScheduler::Configure`.

`CScheduler` (shadow) and `CTask`

- Circle's public + friend API is preserved (`Yield`, `Sleep/MsSleep`, `GetCurrentTask`, `AddTask`, `BlockTask`, `WakeTasks`…) → Circle drivers that block (USB, etc.) work without modification.
- **The task list has no limit.** Circle's table of `MAX_TASKS` (40) pointers — and its `LogPanic` ("System limit of tasks exceeded") when full — is gone: the tasks are a **circular linked list** of `TSchedNode`s (`scheduler.h`), each with the scheduler's own state about its task (the no-kill depth, a pending kill, the preemption streak, killed from outside). `CTask` itself cannot grow (its layout is Circle's, shared with the prebuilt libraries), hence the separate node; `AddTask` allocates it, the reaper frees it. The round robin scans the circle from the node after `m_pScan` (the last one chosen; `YieldTo` points it just before its task); `m_pCurNode` is the running task's node, so the current task's state is reached without a search. `EnumerateTasks`'s callback must not yield.
- **A process's tasks are one group** — the tasks whose `TASK_USER_DATA_USER` is the same address space (the app's threads, §7). `TerminateTask` on an app's task terminates the whole group (`TerminateGroup`: all but the caller); `CountGroup` counts its live ones. A task **killed** from outside may still be on a wait list of its process (a mutex, an event, a socket): the reaper frees it only once no task of its group is alive, and runs the termination handlers of a batch **before** deleting any of its tasks — the last handler frees the process (its sockets closed, its objects' wait lists emptied) while the tasks still exist.
- `WakeTasks` **tolerates a task that is no longer blocked**: a `BlockTask` with a timeout that expired is made Ready by `GetNextTask` but stays on the event's list until it runs again and unlinks itself; an event set in that window (the V3D's frame-done interrupt right after one of the drawing task's 2 ms waits ended) found it Ready, and Circle's assertion halted all four cores — the random freeze of `n64emu` with the GPU (found with the crash record, `sched/scheduler.cpp(742)`). Such a task is now only unlinked.
- **"Real time" tasks (v68, `SetPriority`, `kapi_thread_priority`).** A node's `nPrio` (`SCHED_PRIO_HIGH`) makes `GetNextTask` try `ScanPrio` first — only while such tasks exist (`m_nPrioTasks`): the first ready real-time task after `m_pScan` wins, before the burst and the round robin. The current task is left out of that scan, so a real-time thread that yields while still ready (a spin on `kapi_lock` held by an ordinary thread) lets the others run. A tick (`OnTimerTick`, after `WordWaitTick`) that finds one ready while an ordinary task runs sets the resched flag (`m_bPrioPreempt`): the IRQ exit preempts the app at once (in its own code only, as always), and `OnPreempt` does not count that towards the app's hog streak. The priority holds only while the task yields by itself: preempted at the end of its slice

(a busy loop), it is marked `bPrioSpent` — an ordinary task — until its next voluntary `Yield` (sleep, wait). So a real-time thread cannot starve the system; an audio pump that renders and sleeps keeps it.

- `Yield()` performs the context switch **with the IRQ disabled** for atomicity, then **restores the full `DAIF`** of the resumed task (each task keeps its own IRQ enable state).
- The **only** change in `task.cpp`: `TaskEntry()` re-enables the IRQ (`msr daifclr, #2`) so that a **fresh task** does not start with the IRQ masked inherited from the switcher.
- The idle task uses `wfi` (on core 0; on the network core it only pauses: no timer tick there).
- **One scheduler per core that runs tasks.** `CScheduler::Get ()` returns the scheduler of the calling core (`s_pThis[core]`, the core read from `MPIDR_EL1`): core 0's, and with `netcore=1` core 3's (§11). A `CTask` joins the scheduler of the core that creates it, so Circle's code — tasks, `CSynchronizationEvent`, `MsSleep`, `Yield` — works unchanged on either core. The wait lists of the events share one spin lock (an event may be set from another core). Cores 1-2 have none (`IsActive ()` is FALSE there; `Get ()` falls back to core 0's, as before). Circle's own libraries, built with Circle's `scheduler.h`, read `s_pThis` as a single pointer in their inline `IsActive ()`: element 0, core 0's.

Per-task data

`CTask`'s `TASK_USER_DATA_USER` slot holds the task's `CAddressSpace*` pointer (0 for kernel tasks) — the same one for all the tasks of a process (its threads). The PID, the state (R/S/B/N), the name and the ASID are accessible via the address space and the enumeration API (see §8). The address space counts its tasks (`AddTask` / `DropTask`; the first is its **main task**, `GetMainTask`); the reaping (`reaper`) calls the termination handler — which drops the task, and frees the address space with the last one — then destroys the `CTask`.

---

6. Exceptions and vectors

Source: `kernel/arch/aarch64/vectors.S`, `kernel/arch/aarch64/exception.cpp`, `kernel/include/kern/trapframe.h`.

`KVectorTable` (our `VBAR_EL1`)

16 entries (4 groups × 4 types). A crucial point from hardware bring-up: the **EL1t group (Current EL, SP0)** is the one actually taken by the main thread and the tasks (which run in EL1t). It must route to the real handlers (`SyncEL1Entry` / `IrqEntry` / `FIQStub`) and **not** to `BadModeEntry` — otherwise the first tick crashes.

Group	Sync	IRQ	FIQ	SError
Current EL SP0 (EL1t)	SyncEL1	Irq	FIQStub	BadMode
Current EL SPx (EL1h)	SyncEL1	Irq	FIQStub	BadMode
Lower EL AArch64 (EL0)	El0Sync	El0Irq	FIQStub	BadMode
Lower EL	BadMode ×4			

Group	Sync	IRQ	FIQ	SError
AArch32				

IRQ path

`IrqEntry`: `SAVE_TRAP` → **unmasks the FIQ** (`DAIFClr,#1`, required by Circle's `EnterCritical`) → calls Circle's `InterruptHandler` (GIC dispatch + periodic tick + EOI) → `KernelIRQExit` (the crash-log and stall samples; an app core's stop; **no task switch**: an IRQ at EL1 interrupts kernel code — the apps' IRQs come through `El0IrqEntry`, §5) → re-masks the FIQ → `RESTORE_TRAP` + `ERET`.

On cores 2 and 3 (app cores, §14) the same `KVectorTable` is installed: their only IRQ is the stop IPI, and `KernelIRQExit` hands it to `AppCoreOnIRQExit`; a synchronous fault there goes to `AppCoreOnFault` instead of the kernel panic.

> Bring-up note: leaving the FIQ masked during `EnterCritical` caused a Circle assertion to fail (`synchronize64.cpp`). Hence the `DAIFClr,#1`/`DAIFSet,#1` around the call, as Circle's `IRQStub` does.

Trap frame

`TTrapFrame` (800 bytes): `x[0..30]`, `sp_el0`, `elr_el1`, `spsr_el1`, then the full FP/SIMD state — `v[0..31]` (the 128-bit `q` registers), `fpsr`, `fpcr`. Saved by the assembler macro `SAVE_TRAP` onto the stack on exception entry, restored by `RESTORE_TRAP_ERET`.

The FP/SIMD block is saved on **every** trap so floating-point user code keeps its registers across a context switch. The cooperative `TaskSwitch` (Circle) only preserves the callee-saved `d8–d15` mandated by the AArch64 PCS — enough for a voluntary `Yield()`, but **not** enough when the timer preempts a thread mid-computation. Saving the whole register file here (and in `El0IrqEntry`'s frame, see § "Preemptive scheduling") keeps hardware float correct under both voluntary switches and preemption from the IRQ-exit path. FP/SIMD is enabled at EL0/EL1 at boot (`CPACR_EL1`, `circle/lib/startup64.S`), so the `stp`/`ldp` `q`-register forms never trap.

Faults

Every app runs at EL0 (§ *Protected mode* below): a fault in an app arrives through the EL0 vectors and kills that process only — unless it is a translation fault in a lazy region (v75, §4 *Demand paging*): the page is filled and the access retried. The kill's kmsg says why when the regions tell: `(stack overflow)` within 1 MB below a stack, `(PROT_NONE access)`. `SyncHandlerEL1` sees kernel faults only: **`UAccessFixup`** first (a fault inside a fault-safe copy routine, below: the kapi returns its error, or fills the page and retries), then the v75 safety net (a lazy page kernel code touched: filled), otherwise it is a kernel bug → the post-mortem console (`DumpAndHalt`) and the crash record.

Fault-safe access to app memory (`kern/uaccess.h`, step 2)

Every kapi checks the pointers it gets from an app **at its entry point** (`kapi_*`; the helpers below them take kernel memory): `sys/uaccess.cpp`, `arch/aarch64/uaccess.S`.

- **Who may point where.** A kernel caller (no scheduler yet, or a task without an address space) is trusted. An app's pointer must lie in the user VA range `[USER_VA_BASE, USER_VA_END)`, full stop (its `CTask` stack is its kernel stack, never accepted).

- **`UserRange` / `UserRangeAvail`**: `[p, p+n)` inside what the caller may use, computed as "bytes left from p", never `p+n` (no wrap). **`UserReadable` / `UserWritable`**: the range, then each 64 KB page translated with the **app's** permissions, `AT S1E0R/W` (v75; `S1E1R/W` before) — used where the kernel works directly in the app's buffer (file reads, pipes, sockets, GPU pixels). Since v75 a page of a lazy region not there yet is **filled** first, and the range is **pinned** until the system call returns: another thread's `vm_unmap` / `DONTNEED` / `mprotect` over it is deferred (§4 *Demand paging*). A probe never yields.
- **Copies**: `UserCopyIn`, `UserCopyOut`, `UserGet`, `UserPut`, `UserStrOut` and `CUserStr` (a kernel copy of an app string, 256 bytes inline, the heap beyond; paths over 511 characters are refused): leaf assembly routines, word-sized when both pointers are 8-aligned (byte-wise otherwise: the display is Device memory). Plain `LDR`/`STR` after the range check (the A72 has neither PAN nor UAO). (v75) A copy that faulted on an unfilled lazy page (the fixup records `FAR` / `ESR`) has it filled in C, then runs again — one more page each time, so it ends.
- **The fixup table**: a hand-made `.rodata` list of `{start, end, recovery}`, one per routine; `SyncHandlerEL1` looks the faulting ELR up first and resumes at the recovery label (the routine returns −1). It works for kapis reached by `svc` from EL0 too (they run at EL1t).
- Every pointer-taking kapi fails with its usual error value on a bad pointer; the queues (`klog_read`, `mailbox_recv`, `pop_event`, `pop_post`, `readdir`, `vfs_next`, `tcp_accept`, `thread_join`) check before they consume anything; the GPU structs are copied and their inner pointers (`kapi_gpu_frame.pixels`, `kapi_gpu_program.vs/cs/fs`) checked on the copy. Code pointers (handlers, `post`, `thread_create`'s `fn`) are not data and are not checked. **A new kapi that takes a pointer must do the same.**

The NULL page: Circle maps every identity page, page 0 included, `AP = RW_EL1, UXN = 1`: an app's NULL load, store or call faults at EL0 (the app is killed), and a NULL pointer handed to a kapi is refused by the range check.

Per-process handles (`kern/handle.h`, step 1)

`open`, `opendir`, `pipe`, `file_in`, `file_out`, `spawn`, `stdin_stream`, `stdout_stream` return a **per-process handle**, not a kernel pointer: `(generation << 16) | (index + 1)` (never 0, at most 0xFFFFFF, through the same `void *`). The `CHandleTable` is a member of `CAddressSpace` (threads share it; kernel tasks share one kernel table); entries `{object, type FILE/DIR/STREAM/PROCESS, kind FATFS/RAMFS/VFS, generation 1..255, closing, pins}`; 16 entries growing to 4096. A lookup needs index, generation and type to match an open entry; anything else (a forged or stale handle, a raw pointer) gets the kapi's failure value (`read` −1, `fsize` 0, `stream_write` −1, `wait` −1, `proc_done` 1, the closes do nothing). A kapi that may yield **pins** the entry (`CHandleUse`): a close by another thread meanwhile only marks it, the last unpin closes. **The handles still open are closed when the process ends** (`~CAddressSpace` → `CloseAll`; a stream's last release, which may write to the card, is deferred to the reaper task: `HandlesRunDeferred`). Streams and `CProcess` records are **reference-counted** (a pipe: the creator's handle and each child; a `CProcess`: the spawner's handle and the child), so either side can end first. **Sockets** are owner-checked (the request's pid, also on the network core); a descendant of the owner adopts one on first use (`ftpd` hands its sessions their socket). (v75) `OFILE` entries (`file_open`); (v76) `LSOCK` (a local socket: its handle's value is its socket number) and `SHM` (a shared memory object, the kind is the access) — and every passable object (open file, stream, local socket, shm) can be **carried into another process's table** by `sock_sendmsg` or `spawn_ex2` (§8 *v76: IPC*). `vfs_req_data` / `vfs_reply` only take requests addressed to the calling provider.

Protected mode: apps at EL0 (steps 3–5)

Source: `kernel/arch/aarch64/el0.S`, `el0blob.S`, `kernel/sys/el0.cpp`, `kern/el0.h`. Design and history: EL0-PROTECTED-MODE.md.

Every process runs at EL0 (kapi v74; v73 had it opt-in): the apps, the `/bin` tools, Koton's plugins, the BASIC runner — anything loaded from an ELF. The kernel and its own tasks run at EL1 and are not preempted. (`cmdline.txt` `appmode=` / `protected=` / `appfault=` / `nullguard=` and `app.txt` `mode =` of v73 are gone; old cards that still set them get no effect.)

- **Mappings** (the `KPAGE_ATTR_APP_*` presets of `layout.h`): data `RW_ALL`, read-only data `RO_ALL`, code (and `code_alloc`, and v78 `PROT_EXEC` regions) `UXN = 0, PXN = 1`, everything else `PXN = UXN = 1`; the screen (`fullscreen_direct`) is Device memory: the user-side `memcpy`/`memset` align their stores. The identity region 0–4 GB keeps Circle's `AP = RW_EL1, UXN = 1`: EL0 can neither read, write nor execute it. There is no PAN on the A72: the kernel reads and writes EL0 pages directly.
- **Stacks**: the `CTask` stack (256 KB, `EL0_KSTACK_SIZE`) is the task's **kernel** stack; the user stack (app.txt's `stack`, 1–64 MB) is mapped in the address space; a thread's at `USER_THREAD_STACKS` (32 GB) + (index + 1) × 32 MB with a 16 MB+ guard below.
- **Entry from EL0** (`EL0_ENTRY`, sync and IRQ): `TPIDR_EL1` holds the top of the running task's kernel stack (written by `El0Return` just before each `eret`, so it follows the scheduler with no switch hook); the full `TTrapFrame` (x0–x30, `SP_EL0`, ELR, SPSR, q0–q31, FPSR, FPCR) is built at `TPIDR_EL1 - 800`, `SP_EL0` pointed at it and `SPSel` set to 0: the handler runs at **EL1t on the task's kernel stack**, so the kapis that yield (a file read, a wait) switch tasks as any kernel code does. **Exit** (`El0Return`): DAIF masked, `SPSel = 1`, `TPIDR_EL1` set, the frame restored (x0 last), `eret; dsb nsh; isb`. First entry: `El0Enter(entry, user_sp, arg, lr)` (a zeroed frame, SPSR `0x300` = EL0t with IRQ/FIQ on).
- **System calls** (`El0SyncHandler`): `svc #0` with the table slot in `x8`, arguments in x0–x7 (no kapi takes more, none on the stack or in FP registers), result in x0. Refused (x0 = 0): slot 0, out of range, a null kernel slot, and the user-side slots (memcpy, memset, memmove, pump_events, wait_for_exit, pump_wait — the kernel's pump would run user handlers at EL1).
- **IRQ from EL0** (`El0IrqEntry` → `El0IrqExit`): Circle's `InterruptHandler` on the kernel stack; on core 0 the crash-log and stall samples, and at the end of the slice `OnPreempt` + `Yield` right there — **the only preemption point**: an IRQ taken at EL1 (kernel code) never switches tasks; on cores 2–3 `AppCoreOnIRQExit`.
- **A fault from EL0** (any synchronous exception but `svc`) kills the process: a kmsg line `el0: <name> (pid N) killed: …`, then (2026-10-03) `el0: <name> backtrace: <lr> <lr> …` — up to 16 return addresses read along the frame-pointer chain (x29 → {next x29, lr}, `UAccessCopy`, the chain must climb), the callers to give to `addr2line` (a program built with frame pointers: the POSIX ports and WebKit are) —, a desktop notice (`IpcNotify`), `kapi_exit(-11)`. SError still goes to `BadModeEntry`.
- **The per-process kapi table**: at boot `El0Init` fills two pages shared by every protected process: the **EL0 table**, mapped read-only at `KAPI_TABLE_VA` (14 GB), and the **EL0 code page** at `KAPI_STUBS_VA` (14 GB + 64 KB). Slot *n* points at the stub `movz x8,#n; svc #0; ret; nop` at `+n × 16`; the user-side entries point into the blob (`el0blob.S`, position-independent, at `+8 KB`): **`memcpy`/`memmove`/`memset`** in user code, **`pump_events`** (posts via `pop_post`, then events via `pop_event`, a key handler bracketed by `event_mods` so `get_modifiers` is right), **`wait_for_exit`**, **`pump_wait`** (`pump_sleep` then the pump), and the return paths `El0ThreadReturn` (→ `thread_exit`), `El0MainReturn` (→ `exit(0)`), `El0CoreReturn`. Existing binaries run unmodified.
- **Per core** (`El0CoreInit`): `CNTKCTL_EL1.EL0PCTEN/EL0VCTEN` (the counters at EL0); `SCTLR_EL1` UCI (`dc cvau`/`ic ivau` for the JITs), nTWE/nTWI (`wfe`/`wfi`), UCT (`ctr_el0`), DZE (`dc zva`); **`TPIDRRO_EL0` = the**

core number (`kapi__core` in `user/Kits/appkit/appkit.h` reads it from v73 — apps built earlier read `mpidr_el1` and are killed at EL0: rebuild them); `PMUSERENR_EL0.EN` only with `cmdline.txt el0pmu=1`. **`TPIDR_EL0`** (the thread pointer, writable at EL0) is the app's: the kernel never writes it but for a thread's start (v75: `thread_create_ex`'s `tls`, 0 before) and an app-core job's (its caller's value), and Circle's `TaskSwitch` saves and restores it per task — an EL0 preemption goes through it too.

- **Threads**: `thread_create` in a protected process `eret`s to `fn` on its user stack, returning through `El0ThreadReturn`. **App cores**: `core_run` for a protected owner enters `fn` at EL0 on the app's stack; the job's end is an `svc` from `El0CoreReturn` (→ `AppCoreEl0Done`); any other `svc` or fault → `KAPI_CORE_FAULT`.
- **ID register reads** (v74): on the A72 an `mrs` of an EL1 ID register at EL0 is an undefined instruction; `El0SyncHandler` emulates it (`EmulateMrs`: the instruction decoded, the value written into its register, ELR + 4), as Linux does: `MIDR_EL1`, `REVIDR_EL1` (real), `MPIDR_EL1` (the current core), `ID_AA64PFR0_EL1` (masked to EL0/EL1, FP, AdvSIMD, SVE, DIT, CSV2/3), `ID_AA64DFR0_EL1` (DebugVer only), DFR1/AFR0/AFR1 and the AArch32 ID registers 0, the other `ID_AA64*` (ISAR, MMFR, PFR1, ZFR0) the hardware's (a snapshot of core 0's at boot). Any other system register or an `msr` kills the app. Counted in `proc_stats`' `emulated`.
- **System-call statistics** (v74): each process counts its `svc`s (a total, a per-second rate over a window of CNTPCT, a saturating count per table slot: 820 bytes); `proc_stats(pid, out)` returns them (§8). `ps` (SYSC/s), the Task Manager and `/bin/sysstat` show them.
- The kernel's `TKApiTable` is only the system-call dispatch table (never mapped into an app); its `pump_events`, `wait_for_exit`, `pump_wait`, `memset`, `memcpy`, `memmove` slots are 0 (the EL0 table points them at the user-side code).
- **The tool** `tools/el0scan.sh [file|dir…]` disassembles ELFs and archives and reports any instruction an app cannot run at EL0 (privileged system registers, cache/TLB maintenance other than `dc cvau/cvac/civac/zva` and `ic ivau`, `eret`, `hvc`, `smc`…): run it on the card after a build (`./tools/el0scan.sh sdcard/apps sdcard/bin sdcard/koton`).

Not closed yet: the powerful kapis (`reboot`, `kill_pid`, `inject_*`, `screen_grab`, `win_read`…) stay open to every app, and the GPU (an app's V3D shaders can reach physical memory).

---

7. ELF loader and process model

Source: `kernel/proc/elf.cpp`, `kernel/include/kern/elf.h`, `kernel/proc/image.cpp`, `kernel/include/kern/image.h`, task model in `kernel.cpp`.

Program images (v77): loaded once, shared, kept

*The study and the stages: `docs/ELF-LOADER-PLAN.md`. Until v77 a start read the whole file into the kernel heap, then copied its segments into frames of the process: each byte moved twice, an 80 MB program (the WebKit port's) took 80 MB of kernel heap for a moment and 80 MB of frames **per process**, and every start read the card again.*

The headers first (`ElfReadPlan`). The ELF header and the program headers are read on their own and checked: the ELF64 magic, `ELFCLASS64`, `EM_AARCH64 = 183`, type `ET_EXEC` / `ET_DYN`; the program headers inside the file; for each **`PT_LOAD`** segment with memory, its bytes inside the file and its addresses inside the user area (no wrap); at most 16 such segments; **no two segments in one 64 KB page** (before v77

the second one silently replaced the first one's page — our programs are linked with 64 KB pages, so none did). A file size above the memory size is cut to it, as before. A refusal comes with a reason, which the start's error line shows (`cannot load SD:/bin/x: segment past end of image`).

The image object (`TImage`). One per program in memory:

- for each **read-only** segment (no `PF_W`: `R+X` code, `R` data — 99.8 % of a large program): its 64 KB frames, allocated from the app pool, zeroed, and filled **straight from the file** — no whole-file buffer, no second copy, and the file's other sections (symbols, debug information) are never read. Reads are gathered up to 128 KB where the frames follow each other in memory (a read per page where they do not), with a yield every 128 KB, as the old loop did;
- for each **writable** segment: a copy of its file bytes in the kernel heap (156 KB at most in today's programs), from which every process gets its own pages. A process therefore never needs the file again, which is what lets `pkg` replace a program's file while it runs.

A process's mapping (`ImageMap`). The read-only segments' frames are mapped **not owned** (`MapPage(..., bOwned = FALSE)`) with `KPAGE_ATTR_APP_CODE` (`PF_X`) or `KPAGE_ATTR_APP_RODATA`; the writable ones get fresh owned pages (`MapNewPage`, `APP_DATA`, or `APP_CODE` if `PF_X`, as before), the copy's bytes, and a zero bss. The regions are noted as before (`KAPI_VMK_IMAGE`). The address space holds one reference on the image (`CAddressSpace::SetImage`), dropped at the very end of its destructor. The caller then runs `SyncDataAndInstructionCache()` once, as `LoadELF` always did.

Nothing writes a shared frame after the load. The loader writes it through the kernel's identity mapping before any page table names it; in every process its pages are `AP = RO_ALL` — read-only for EL0 **and** for EL1 — so a kapi's copy-out into a program's code or constants fails as it always did (`UserWritable` uses `AT S1E0W`), no vm call can change them (`vm_protect`, `vm_unmap` and `vm_advise` only work in the mmap arena and the lazy regions), the kernel has no debugger stub that patches code, and the teardown frees only the frames flagged `PAGE_SW_OWNED`. One difference from before: a read-only segment **without** `PF_X` was mapped read-write; it is now really read-only (none of today's programs has one: their constants are in the `R+X` segment).

The key is the program's full path, in its canonical form — `ImageCanonPath`, the one function that the run, the preload, the unload, the query and the file layer's hook all use:

Rule	Example
lower case, the volume included — ASCII letters only	`SD:/Apps/MonApp.App/main` → `sd:/apps/monapp.app/main`
the volume always first: the path's own, else the caller's (`/x`: the root of the working directory's volume; anything else: under the working directory, as `ResolvePath`); no working directory (the kernel's own launches): `SD:/`, which is what FatFs makes of a path without a volume	`ls` in `SD:/bin` → `sd:/bin/ls`; `SD:apps/x.app/main` → `sd:/apps/x.app/main`
the spellings of **one** volume collapse: its name in any case, FatFs's numeric form (`0:` is the first name of `FF_VOLUME_STRS`, `1:` the second…) and the kernel's `SD0:`	`0:/bin/ls`, `SD0:/bin/ls`, `sd:/bin/ls` → `sd:/bin/ls`

Rule	Example
two volumes stay two keys: `SD:` (the card's first partition) and `SD1:` … `SD3:` (its others), `USB1:`, `USB1P2:`… (`USB:` is `usb1:`, as `SD0:` is `sd:`)	`SD1:/apps/x.app/main` → `sd1:/apps/x.app/main`
`/` between the names (`\` too, as FatFs takes it), none doubled, none at the end, no `.` / `..`, a name's trailing dots and spaces dropped (FatFs drops them)	`SD:\apps//x.app/./main.` → `sd:/apps/x.app/main`
at most 255 characters, 64 names deep; beyond: no key (the program is loaded, never shared)	

Bytes that are not ASCII letters are kept as they are. FAT also folds accented letters, so two spellings of one accented name (`É…` / `é…`) are two keys: the file may be in memory twice, and a change made through the other spelling is not seen by the hook below — the wrong *file* is never run for a path, but an old *version* could be, until `unload`. The same holds for a file reached by its 8.3 short name. Onyx's programs have plain ASCII names.

A start (`CUserProcessTask::Run`, `ProgramImage`). `ImageOpen (path)`: an image with that key → a reference, **the card is not touched** — no open, no directory lookup (the launchers' existence check asks the image list first; the user stack's size, `app.txt`'s `stack =`, is kept in the image after the first start). None → the file is opened and streamed into a new image. An image **being loaded** by another task → the start waits for it (it looks again every 5 ms; the loader yields), then shares it; if that load fails, the waiters get its error and nothing is left. A task that is loading cannot be killed (it has no address space yet, so no pid).

Lifetime. An image lives while a process maps it or while it is **pinned** (a preload). Dropping the last reference of an image that is not pinned frees its frames (their `wait_word` waiters woken first). `image_unload` — and the file layer's hook — take the **name** and the pin away at once: no new process maps it, the processes running it keep it, and its frames go with the last of them. Images do not survive a restart.

Where the file changes, the image loses its name (`ImageFileChanged`). Since a run of a path that has an image never looks at the file, the file kapis tell the image code:

Operation	Hook
`remove`, `path_unlink`	the path (a folder: every program under it)
`rename`, `path_rename`	both names; a renamed folder: every program under either name
`file_open` for writing, with `KAPI_O_CREAT` or `KAPI_O_TRUNC`; `file_out` (truncate or append); `save_file`	the path, at the open **and** at the close of the written file (an image made from the half-written file in between is dropped too)
a write to `<folder>/app.txt`	the image of `<folder>/main` stays; it forgets its stack size (read again at its next start)

Not covered: the accented and short-name spellings above; a card changed on a PC (images do not survive a restart, so there is nothing to cover); the kernel's own writes (`lastcrash.txt`, the clock file: never programs); `RAM:` and the providers' volumes (`FTP:`): programs are only run from FatFs volumes. The hook costs one comparison when no image exists, a path canonicalisation and a walk of the image list otherwise; it does no I/O.

Preload (`image_preload`, `/bin/preload`). A kernel task (`CPreloadTask`) loads the program's image and pins it; the call returns at once. A start during that load waits for it. A preload is refused when the image would take the app pool's 16 MB reserve (`VmCommitOK`); a process's own start takes what there is, as before. `pkg` asks `image_list (path)` before it replaces a file: a kept program is unloaded, replaced, and preloaded again (`user/Libs/pkg/pkglib.h` `move`).

Accounting. A process's owned pages (`ps`' `PAGES`, `vm_stats.resident`, the crash record) no longer include its program's read-only segments: they are counted once, in `meminfo`'s app figure (`g_nUserPages + ImagePagesTotal ()`), and listed by `preload`.

The log. One line per start, in the kernel log (`kmsg`, docs/04 §8): `image SD:/bin/wctest: loaded in 5123 ms, mapped in 3 ms (81856 KB shared, 192 KB private)` — `loaded` (read from the card now), `shared` (in memory: no read) or `shared after a wait`; a preload logs `image sd:/bin/wctest: loaded in 5123 ms, kept (81856 KB shared)`.

Shared libraries (v83): one copy of the code, placed by the kernel

(The design and its reasons: `docs/SHARED-LIBS-PLAN.md`. Writing and using a library: docs/03 Shared libraries.)

A shared library is **an image** — the same object as a program's (`TImage`, `proc/image.cpp`): read once, its read-only frames mapped *not owned* into every process that uses it, its writable bytes kept in the object and copied into private pages per process, reference counted, preloadable, unnamed when its file changes. What is different:

- **The file.** `SD:/lib/<name>.so`: an `ET_DYN` linked **at 0** by `user/Runtime/lib.ld` — position-independent code (`-fPIC`), **two `PT_LOAD`** (read + execute: the headers, `.rela.dyn`, the code, the read-only data; read + write: `.data.rel.ro`, `.init_array`, `.dynamic`, `.got`, `.data`, `.bss`) on separate 64 KB pages, one `PT_DYNAMIC` inside the writable one. **The ELF entry is the library's export table** (`ld -e onyx_lib_table`): the kernel needs no symbol table. `ElfReadPlan (..., ELF_KIND_LIB)` checks that shape and nothing else is accepted; a program is refused as a library, and a library as a program (`ELF_KIND_ANY`, a preload's: whichever the file is).
- **Its place.** The kernel places a library **once for the whole system**, when it loads its file (`LibPlace`): the lowest free range of the **library arena**, `USER_LIB_BASE` (16 GB) .. `USER_LIB_END` (32 GB) of every user space, a page left between two libraries. The live images are the allocator's state: the range is taken before the load's first yield and comes back when the image is freed. So a library has the same address in every process — but **not a link-time address**: another build, another boot, another order of loading gives another place. A process still running an old build of a library whose file was replaced keeps that build's range; the new build is placed elsewhere.
- **Its relocations, once.** After the segments are read (`LibRelocate`): the dynamic section's `DT_RELA` / `DT_RELASZ` / `DT_RELAENT` are walked, and for each entry `base + addend` is written into the image's **copy of the writable segment** (`TImgSeg::pInit`). Only `R_AARCH64_RELATIVE` (1027) is applied (`R_AARCH64_NONE` is skipped); any other type fails the load with its number in the log, as do `DT_TEXTREL` / `DF_TEXTREL`, `DT_REL`, `DT_JMPREL`, and an offset outside the writable segment's file

bytes. The code frames are never patched: they are shared as they are in the file. Every process then copies **already relocated** data (the vtables, the export table, the pointer tables): no relocation work per process.

- **Several per address space.** `CAddressSpace` keeps its program's image and up to 16 libraries (`m_pLib[AS_LIB_MAX]`, a reference each, dropped by the destructor as the program's — after the TLB invalidation). `ImageMapLib` maps the segments at `base + p_vaddr` (`MapSegs`, shared with `ImageMap`), notes the regions (`KAPI_VMK_IMAGE`), and returns `base + e_entry`. A mapping that ran out of memory stays recorded (its pages are there) and is never handed out.
- **`lib_open`** (`kapi_lib_open` → `LibraryOpen`, `kernel.cpp`): the name made a canonical path, the image found in memory (no card access) or streamed by the calling task as a program's start does, the version checked against `min_version` (the table's first `u32`, read in the relocated copy), the library mapped, the caches synchronised. One line in the kernel log per mapping: `lib: sd:/lib/uikit.so: loaded in 41 ms at 0x400000000, version 3, 5120 relocations, 640 KB shared, 128 KB private` (`loaded` / `shared` / `shared after a wait`).

The export table. A struct of function pointers in the library's data, relocated with it. Its first fields are every library's (`user/Runtime/lib.h` `TLibHeader`): `unsigned version, size; int (*init) (const TLibImports *)`. `init` is called once by each process (the data is per process) with what the library takes from its importer — the allocator, so that the process has one heap; it runs the library's static constructors (`.init_array`, between `__lib_init_array_start` / `_end`: `user/Runtime/librt.cpp`). The table is **append-only and versioned as the kapi table is**: an entry is never moved, removed or changed; a new one goes to the end and the version goes up. The kernel knows nothing of what follows the header.

What is shared, what is not. The read + execute segment is one set of frames for the whole system (`image_list`: the library once, `refs` = the processes mapping it, `size` = its frames + its data's copy). The read + write segment is private pages in every process: a library's globals are per process.

Preload, unload, the file hook. As a program's: `preload SD:/lib/uikit.so` (or the line in `SD:/etc/preload.ini`) keeps a library in memory with no process; `unload` and a write, rename or removal of the file take its name away — processes that map it keep it, new ones load the file again.

Tests: `sh tools/tests/run_image_test.sh` builds the test library (`user/Libs/demo`) with the cross toolchain and runs the real loader on it (placement, the relocated table, two address spaces on the same code frames with their own data, a second library placed after the first, the file replaced while mapped, the range coming back, out of memory while mapping, 16 libraries, a relocation that is not `RELATIVE` / outside the data / text relocations refused); `sh tools/tests/shlib/check_pic.sh [file.so...]` checks a library's format. On the Pi: **`/bin/libtest`** (docs/04) — 17 checks against `SD:/lib/demo.so`, passed on the Pi 4 on 2026-10-04.

Aliases (v97): the graphics server's library under the common name

(The design: `docs/POCKETUI-TECH-STUDY.md` §3; the call: §8 v97.) Each graphics server has its own UIKit with the same exports (Elegant `SD:/lib/uikit.so`, PocketUI `SD:/lib/pocket/uikit.so`); a program always opens `"uikit"`. **`lib_open_as (path, alias, min_version, err)`** opens and maps `path` exactly as `lib_open` does (the version checked on the real library), then gives its image **a second key, its alias** (`TImage::Alias`, `nAliasOwner`, `bAliasOrphan`, `proc/image.cpp`):

- **Looked up first.** `FindNamed (key)` looks at the aliases before the named images' paths: while `sd:/lib/uikit.so` is an alias, a named image whose real path is `sd:/lib/uikit.so` (the desktop's UIKit, preloaded or still mapped) is not returned for that key. Nothing else changes: the aliased

image keeps its real `Path`, its place in the arena (so the server and every program map it **at the same address**, one copy), its references. `LibraryOpen`'s log names the real path and the alias: `lib: sd:/lib/pocket/uikit.so (as sd:/lib/uikit.so): shared in 0 ms at 0x400000000, ...`.

- **Who, what.** Only the process holding the graphics server's role (`WsIsServer`, `sys/wsrv.cpp`); both names under `sd:/lib/`, never `sd:/lib/appkit.so` (`ImageAliasAllowed`: an alias there would hijack every program); `path` ≠ `alias`. One image a key, one key an image (`-EBUSY` otherwise). The alias is checked before the mapping and set after it (`ImageSetAlias (…, bCommit)`), with no yield between. The order in a server's start: `KAPI_WS_REGISTER`, `lib_open_as`, `KAPI_WS_DISPLAY` — `WsBootStart` returns when the display is taken, so every program init starts finds the alias.
- **Lifetime, anchored on the server.** The alias goes with its image (its last reference, or the end of its pin). Its owner ends (`WsOnProcessGone` → `ImageUnalias (pid, TRUE)`): **orphaned**, kept — programs started meanwhile still get it; the same server started again (`WsPoll`) asking the same real path takes it over (`ImageAliasHeld`: the very image the alias names, even if its file changed since), a different path replaces it. **Another server started** (the fallback to Elegant, `KAPI_WS_SWITCH`): every alias dropped at once (`ImageUnalias (0, FALSE)`) — the images keep their real names and their processes. (Without the anchor a windowless daemon mapping UIKit — `printd`, through PrinterKit — would keep the pocket UIKit as everyone's `uikit.so` after a switch back to the desktop.)
- **Files, preload, lists.** `ImageFileChanged` compares the **real** path only: an update of the desktop's `uikit.so` does not touch the alias; an update of `pocket/uikit.so` unnames the real path but keeps the alias on the old image (the session stays on one UIKit; the new file is the next session's). A preload of the alias key pins the aliased image (dropped with the alias if its path was unnamed meanwhile); `image_unload` of an alias key → `-EBUSY` (it ends with its server). `image_list` flags it `KAPI_IMG_ALIAS` (16): `path` is the real key and the alias follows its NUL in the same field — `/bin/preload` prints `sd:/lib/uikit.so -> sd:/lib/pocket/uikit.so`.

Tests: `sh tools/tests/run_image_test.sh`'s *aliases* (the names allowed, looked up first, the check without the commit, idempotent, `-EBUSY` for a second owner / image / key, unload refused, the two file changes, orphaned / taken over / replaced, dropped, a pin through the alias, gone with the last reference, the list's flag and second path); on the Pi `/bin/aliastest` (docs/04), also a stand-in server.

Tests: on the PC `sh tools/tests/run_image_test.sh` (the real `image.cpp` and `elf.cpp`, the kernel around them stubbed, ASan: the canonical path, the header checks against crafted files, the load, two processes on the same frames, a start waiting for another task's load, a failed load, the references and the pin, unload while in use, the hook).

`LoadELF(image, size, AS, &entry)`

The old entry point, over a whole file in memory. Nothing in the kernel calls it any more; it goes through the same code (an image of its own, without a path: never shared), then `SyncDataAndInstructionCache()`, and returns `e_entry`.

`CUserProcessTask` — one application = one task

`CUserProcessTask` (subclass of `CTask`; its `CTask` stack is the process's 256 KB kernel stack; the app's **user** stack is 8 MB (v75: lazy, so it costs only what is touched), or more if the app's folder's `app.txt` asks for it, `stack = 16M` — `AppUserStack`, rounded up to 64 KB, 8–64 MB):

1. Creates a fresh `CAddressSpace`.

2. Installs stdin/stdout, the process handle, argv, cwd.
3. Its program's image (v77, *Program images* above): found in memory by its path, else streamed from the file; mapped into the address space (`ImageMap`).
4. `SetUserData(AS, TASK_USER_DATA_USER)` + `AS->AddTask(this)` (its main task) + `Activate()` (switches `TTBR0`/ASID).
5. Makes the **user stack** below `USER_STACK_TOP` a lazy `STACK` region (8 MB, or `app.txt`'s `stack`; nothing below it: an overflow is a "stack overflow" kill) and **enters the app at EL0**: `El0Enter (entry, USER_STACK_TOP, 0, El0MainReturnVA ())` — no return; the task's own stack (`EL0_KSTACK_SIZE`, 256 KB) is from then on only the process's **kernel** stack (its traps, the kapis it calls).
6. `main`'s return goes through the blob's `El0MainReturn` (→ `exit (0)`); `kapi_exit`, a fault or a kill end the process in the kernel: `ThreadsEndProcess()` (its other threads end with it) and `Terminate()`; the reaper reaps the tasks and frees the address space with the last one.

Threads (v67)

Source: `kernel/sys/thread.cpp`, `kernel/include/kern/thread.h`.

A thread is a `CUserThreadTask`: one more `CTask` whose `TASK_USER_DATA_USER` is the app's `CAddressSpace` (set in its constructor, before it is first scheduled: no `Yield` in between), so the task switch activates the app's page table and every kapi sees the same process — window, heap, files, sockets, cwd. It enters `fn (arg)` at **EL0** (`El0Enter`) on its own user stack, a lazy `STACK` region (v75) at the top of its record's slot, `USER_THREAD_STACKS` (32 GB) + (index + 1) × 32 MB, the rest of the slot unmapped (the guard); 256 KB by default for `thread_create` (16 KB .. 16 MB), 8 MB for `thread_create_ex`. A slot reused at the same size keeps its region; an ended thread's stack pages are dropped (`thread_exit`). Its `TPIDR_EL0` starts at 0, or at `thread_create_ex`'s `tls` (set before `El0Enter`; Circle's `TaskSwitch` saves and restores it per task from then on); its `CTask` stack is its kernel stack. The timer preempts it in its own code like any app (§5). `fn`'s return, or `kapi_thread_exit`, records its exit code and ends the task; the reaper frees it while the process goes on. A process runs at most `THREADS_MAX` (32) threads besides its main one.

- **The process ends with its main task** (return from `main`, `kapi_exit`), or when any thread calls `kapi_exit`: `ThreadsEndProcess` → `TerminateGroup` (all but the caller). A kill (task manager, `kill_pid`, `proc_tree`, an orphan) ends the group the same way (§5). A fault in any of its threads kills the whole process (§6, *Protected mode*), never the machine.
- **The per-process state** — `CProcThreads`, made on first use, freed by `~CAddressSpace` (`ThreadsFree`, first): the thread records (tid → task, done, exit code; 64: the ended ones are kept until joined, the oldest reused first), the synchronisation objects (256 handles), the posted-call ring (256) and two events: `DoneEv` (a thread ended: its joiners) and `WakeEv` (a post, a window event, the close box: `pump_wait`).
- **The objects** — mutex (recursive, an owner; released when its owner thread ends), event (manual or auto reset), barrier (a count, a round number). A kapi runs on core 0 and is not preempted, so an object's check-then-wait is atomic: the objects need no lock. A waiter blocks on the object's `CSynchronizationEvent`, only ever set-then-cleared (Circle's `Pulse` is private) — a condition variable's broadcast; it counts itself in `nWaiters` across the wait and checks its condition again when woken. An object closed while waited on (`sync_close`) is freed by its last waiter (they return −2); those left at the process's end are pulsed — unlinking the killed tasks still on their wait list, which `~CSynchronizationEvent` asserts is empty — and deleted.

- **Posts**: `post (fn, ctx, value)` appends to the ring and pulses `WakeEv`; `pump_events` runs the pending posts (at most one ring's worth) before the window's events, on the calling thread. `pump_wait` points the window's wake-up (`CWindow::SetWake`: `PushEvent` and `RequestExit` pulse it) at `WakeEv`, sleeps if nothing is pending, then pumps; `wait_for_exit` sleeps on it too (16 ms at most, as before).
- **Word waits (v68, a futex)** — `wait_word (addr, expected, ms)` / `wake_word (addr)`. The word's **physical** address is the key (`WordPhys`: since v75 `UserReadable` — a lazy page filled, pinned for the call, so its frame stays while the caller sleeps — then `AT S1E0R` on the caller's `TTBR0` and `PAR_EL1`; below 4 GB, the kernel's identity map): the same word of a shared surface, mapped at another address in each process, wakes across processes. A waiter (`TWordWaiter`: the address, the value, its process, a `CSynchronizationEvent`) lives on its task's stack, linked in one kernel list under an IRQ spin lock; it is linked, then the word read again, then it waits — with the IRQ masked until it is on the event's list (Circle's `Wait` tests then blocks: a tick's `Set` in between would be lost; `Yield` keeps each task's own `DAIF`). `wake_word` sets the events of the waiters on that address. **Code on an app core changes words without any kapi**: `WordWaitTick`, called by `PeriodicTick` in the 100 Hz timer interrupt, reads every sleeping word through the identity map and wakes those whose value moved (nothing to do, and nothing read, while nobody sleeps). Spurious wakes are allowed (callers loop). A dying process unlinks its waiters first (`ThreadsFree` → `WordWaitsFree`: a killed task's record is on its stack, freed after the batch's handlers). A frame leaving its space (v75: an unmap, `DONTNEED`) wakes its waiters first (`WordWaitsZap`). Test: `/bin/futextest`, `/bin/memtest`.
- **Priority (v68)**: `thread_priority (tid, 1)` makes a thread "real time" (§5): picked first whenever it is ready, as long as it sleeps before its slice ends — for an audio pump.
- **The lists show a process once**: `list_windows`, `list_tasks` and `list_procs` skip the tasks that are not its main task; a thread's name is `<app>:<name>` (or `<app>:<tid>`), which the crash reports' task lists show. `kill` / `kill_pid` refuse the caller's own process.
- **User side**: `umm_malloc` / `umm_free` take a lock (`kapi_lock`: an exclusive swap, a `yield` while it is held — a spin on an app core, which makes no kapi call), and `user/Runtime/libc/onyx_syscalls.c` defines newlib's retargetable locks (the 8 static ones and the 10 `__retarget_lock_*` functions, which keeps newlib's `lock.o` out); a recursive lock's owner is the tid on core 0, the core on an app core. `errno` stays shared.

Launch entry points

- **`LaunchApp(name)`**: builds `SD:apps/<name>.app/main`, reads it into RAM, creates a `CUserProcessTask` (without stdio). Exposed via `kapi_launch`.
- **`SpawnProcess(path, args, stdin, stdout, cwd)`**: creates a `CProcess` handle, increments the streams' refs, creates the task. Exposed via `kapi_spawn` (the terminal for pipes/redirections). `kapi_wait`/`kapi_proc_done` query the handle.
- **`ExecPath(path, args)`**: "fire-and-forget" (without stdio or handle; the task name is derived from the path). Exposed via `kapi_exec` (the file manager to open a document in an app, or launch a program).
- **Only ELF programs**: the kernel no longer knows BASIC (the former `BasicRedirect`). The formats a program runs (`.bas` / `.bax` → `SD:/bin/basic`) are chosen in user space: `SD:/etc/runners.ini` + AppKit's `lx_*` functions (they were `user/launch.h`), which start the runner with **`kapi_exec_as (path, args, name)`** (v49, `ExecPath` with a name), so the process keeps the app's name (windows, `raise_app`, `list_windows`).

---

8. The kapi ABI table

AppKit (2026-10-05): the table below is no longer what the programs are built against. The programs reach the kernel through **AppKit** (`SD:/lib/appkit.so`, `user/Kits/appkit/appkit.c`: the `kapi_*` functions that `user/Kits/appkit/appkit.h` declares — the header the programs include —, exported **by name**; their bodies: `user/Kits/appkit/appkit_calls.inc`), and only AppKit reads this table. The kernel and AppKit are built and shipped together (the package `onyx`), so **the table may be restructured** — entries moved, removed, merged — by adapting AppKit; no program is rebuilt. What is append-only from now on is AppKit's list of names (`user/Kits/appkit/appkit.abi`), kept by the generator.

How the kernel gives it to the programs (no call of theirs):

- `kernel.cpp`, `AppKitAttach`: at the first program's start `SD:/lib/appkit.so` is loaded like any library (§7 *Shared libraries*), its export table read (`ImageLibTableRead`) and **copied at `APPKIT_TABLE_VA`** (`KAPI_TABLE_VA + 0x8000`: the same read-only page as the kernel's own table, `El0InstallAppKit` — entry *n* at `+ 8 n`, 4096 at most); the image is kept for good.
- Every new address space then gets AppKit's code mapped (`ImageMapLib` of that very image) before its program's own image.
- A program's import stubs (`lib/appkit_stubs.o`, linked into every program and library) are five instructions each: the entry read at the fixed address, a jump. No variable, no constructor, nothing to open: valid from the program's first instruction.
- **An update of AppKit takes a restart**: the page every program reads holds the addresses of the image loaded at this start; a newer file on the card is the next start's.
- Without `appkit.so` the log says so loudly and no program built for AppKit can call the kernel (a program built before AppKit, reading the table itself, still runs while the layout is the one it knew).

What still reads the table itself (`-DKAPI_INLINE`, or a PC build): the two tests of the table (`el0test`, `faulttest`) and the simulator's stand-in kernel. Jet (the hosted WebKit build) goes through AppKit like the others since its rebuild of 2026-10-05: its sysroot's `libonyxposix` carries the stubs. A standalone BASIC app carries a copy of the runtime: `basic -u` gives the card's ones the current runtime (docs/04).

Source: `kernel/include/kern/kapi_abi.h`, `kernel/sys/kapitable.cpp`, `kernel/sys/kapi.cpp`. App side: `user/Kits/appkit/appkit.h`.

The mechanism

Rather than linking the apps against the **kernel symbol addresses** (which move on each rebuild), the kernel publishes a **function-pointer table** (`struct TKApiTable`) at a **fixed virtual address**:

- `KAPI_TABLE_VA = 14 GB` — stable "forever" (between the canvas at 12 GB and the stack at 16 GB).
- The kernel's table is a static variable aligned to 64 KB (`s_Table` in `kapitable.cpp`). `KApiTableInit()` fills all the pointers + the `version` field. It is the **system-call dispatch table** only: it is never mapped into an app.
- What an app sees at `KAPI_TABLE_VA` is the **EL0 table** (`El0Init`, `sys/el0.cpp`), one page shared by every process and mapped **read-only** in each `CAddressSpace` (cf. §4): the same layout and `version`, but slot *n* points at a stub `mov x8, #n; svc #0; ret` in the EL0 code page (`KAPI_STUBS_VA`), and the user-side slots (`memcpy`, `memset`, `memmove`, `pump_events`, `wait_for_exit`, `pump_wait`) at routines that run in the app itself (`arch/aarch64/el0blob.S`; §6, *Protected mode*).

- AppKit's side (`user/Kits/appkit/appkit_calls.inc`, the only place) defines `#define KT ((const struct TKApiTable *) KAPI_TABLE_VA)` and one inline function per entry (`kapi_open`, `kapi_launch`, …) that does nothing but call through the table — a plain indirect call into the stub, which makes the system call. `user/BinUtils/kapi_names.h` (generated by `tools/gen_kapi_names.py` from `kapi_abi.h`) names the slots for `/bin/sysstat`.

Consequence: an application binary **embeds no kernel address** and keeps working against any kernel that exposes the same ABI → no rebuild of the apps when the kernel changes.

The *append-only* contract

`KAPI_ABI_VERSION = 77`. The `TKApiTable` struct is **strictly append-only**: you never remove or reorder a field; you add new ones **at the end** and you increment the version. An old app only touches the prefix it knows → it stays compatible. The history of additions is annotated in the file (v1 = `app_dir`, v2 = `set_click_handler`, v3 = `opendir/readdir`, v4 = streams/spawn, … v16 = `set_window_theme`, v17 = `chdir/getcwd`, v18 = `stdin_stream/stdout_stream`, v19 = `klog_read`, v20 = `set_verbose/get_verbose`, v21 = the TCP socket calls, v22 = `set_pointer_handler`, v23 = `meminfo`, v24 = `sbrk`, v25 = `reboot`, v26 = `kbd_ready`, v27 = `set_keymap_data`, v28 = `get_chrome/draw_text_buf` (user-side window chrome), v29 = compat break (kernel-drawn widget API removed, table consolidated), v30 = `random` (hardware RNG), v33 = `ram_detail`, v34 = `set_wheel_speed/get_wheel_speed`, v35 = shared surfaces (`surface_*`) + shell IPC (`register_shell`, `shell_request`, `mailbox_send/mailbox_recv`), v36 = `memset/memcpy/memmove`, v37 = `tcp_listen/tcp_accept`, v38 = `screen_grab/inject_pointer/inject_key`, v39 = `set_menu/get_menu/menu_command`, v40 = `ipc_register/ipc_lookup`, `clipboard_set/clipboard_get`, `set_window_alpha`, `shutdown`, v41 = `fullscreen_begin/present_fb/fullscreen_end`, v42 = `drag_begin/drag_data`, `get_modifiers/inject_modifiers`, v43 = `net_ping/net_resolve/net_info`, v44 = `vfs_register/vfs_next/vfs_req_data/vfs_reply`, v45 = `wlan_scan`, v46 = `sound_acquire/sound_release/sound_start/sound_stop/sound_write/sound_status`, v47 = `sound_instrument`, v48 = `key_held/inject_key_held`, v49 = `exec_as`, v50 = `pad_state`, v51 = `core_acquire/core_run/core_state/core_release`, v52 = `gpu_info/gpu_draw`, v53 = `gpu_texture/gpu_render`, v54 = `kapi_gpu_vertex3` second (added) colour `r2 g2 b2 a2` in place of `reserved`, `KAPI_GPU_B_ALPHATEST(t)`, v55 = `fullscreen_direct`, v56 = `win_list/win_read/win_raise/win_close`, v57 = `seek`, v58 = `code_alloc`, v59 = `fsize64`, v60 = `sound_volume/wlan_reconnect`, v61 = `gpu_program/gpu_render2`, v62 = `gpu_render3`, v63 = `gpu_vbuf`, v64 = `win_minimise/win_geometry/resize_window2` — the modernised CDE desktop's windows: the frame's metrics and title buttons shared with the apps (`KAPI_FRAME_*`), `GUI_EVENT_WINCTL`, `KAPI_WIN_MINIMISED`, the frames' rounded corners, `WIN_FLAG_ALPHA`), v65 = `desk/win_desk` — the workspaces (virtual desktops: `KAPI_WIN_OFFDESK`, `KAPI_WIN_DESK`, `KAPI_DESK_MAX`), v66 = `screen_set` — the resolution changed while running, `GUI_EVENT_DISPLAY_RESIZE`; a window as big as the screen (was 1024 × 768 at most), v67 = `thread_create/thread_exit/thread_join/thread_self`, `mutex_*/event_*/barrier_*/sync_close`, `post/pump_wait` — threads (§7), their synchronisation objects, calls posted to the event pump; the scheduler's task list has no limit, v68 = `sound_config/sound_map` — low-latency sound and the mapped PCM ring (§12), `wait_word/wake_word` — a futex (§7), `thread_priority` — "real time" threads (§5), `midi_read/midi_devices` — USB MIDI input (`struct kapi_midi_event`), v69 = `screen_native` — the monitor's own resolution (its EDID), `set_timezone` — the time zone while running, and the window flag `WIN_FLAG_FIXED` (§10.2) — for Setup, the first-run wizard, v70 = `gpu_texture_rect` — a rectangle of a GPU texture replaced (no whole re-upload), the frame flag `KAPI_GPU_F_ALPHA` (a target that keeps its alpha), the GPU's handles shared fairly by the programs using it at once (`KAPI_GPU_MAX_TEXTURES` 1024 in all, `KAPI_GPU_MAX_TEXTURES_AS` 512 a program; `gpu_vbuf` 32 in all, 8 a program) — for the GPU compositing service (`user/Libs/gpucomp`, §15), v71 = `vol_info` — a volume's size, used and free bytes,

type (`struct kapi_vol_info`, `KAPI_VOL_RAM`); and the **RAM: volume** (§16): the file calls (`open` … `rename`, `save_file`, `file_in` / `file_out`, `chdir`, `seek`, `fsize64`) reach a file system in memory on `RAM:` paths — no new call for that, v72 = `gpu_render`'s **compositing blend presets** `KAPI_GPU_BLEND_MULCOL` … `DSTOUT` (5–12, §15: the layer blend modes of `user/Libs/gpucomp` — multiply, screen, plus, subtract, lighten, mask, cut out); an older kernel draws 5–15 as `ALPHA` (the service checks the version) — no new call, v73 = **protected mode** (§6): `pop_event`, `event_mods`, `pop_post`, `pump_sleep` — the pieces of the event pump an EL0 app runs on its own side (an EL1 app, as apps were then by default, never needed them); the core number in `TPIDRRO_EL0` (`kapi__core`); the file / stream / process handles per process and every pointer checked (no change for a well-behaved app), v74 = **every process at EL0** (the EL1 mode removed), `proc_stats` (a process's system calls: `struct kapi_syscall_stats`), the ID register reads emulated; the kernel table's six user-side slots 0, v75 = **the POSIX layer's kernel half** (`docs/POSIX-PLAN.md`), 43 entries in three blocks after `proc_stats`: memory (slots 199–206), files and processes (207–228), sockets and poll (229–241), with their structures and the `KAPI_E*` error values (newlib's errno numbers: a v75 call returns ≥ 0, or −`KAPI_Exxx`). **The skeleton:** the entries exist (the slot numbers are `static_assert`ed in `kapi_abi.h`) and return `-KAPI_ENOSYS` until their work package lands; the subsections below say what is implemented. `user/Kits/appkit/appkit.h`'s wrappers also return `-KAPI_ENOSYS` on an older kernel or where a host table (the PC simulator, the Windows / macOS builds) leaves the slot 0, v76 = **IPC between processes** (`docs/POSIX-PLAN.md` §14, WP-IPC: what WebKit2 needs), 11 entries after `poll` (slots 242–252): `sock_pair` (local sockets, STREAM / SEQPACKET / DGRAM), `sock_sendmsg` / `sock_recvmsg` (handles carried between processes), `shm_create` / `shm_open` / `shm_unlink` / `shm_ctl` / `shm_map` (shared memory), `handle_close`, `spawn_ex2` / `get_handles` (handles given to a child); `KAPI_SO_RCVBUF` / `SNDBUF` / `PEERPID` / `DOMAIN`, `KAPI_VMK_SHM` (*v76: IPC* below), v77 = **program images** (§7 *Program images*): a program is streamed from its file once and its read-only segments are shared by its processes — no call changes for that —, 3 entries after `get_handles` (slots 253–255): `image_preload`, `image_unload`, `image_list`; `struct kapi_image_info`, `KAPI_IMG_*` (*v77: program images* below), v78 = **executable memory for a JIT** (WebKit roadmap step 3, docs/08): `vm_map` and `vm_protect` accept `KAPI_PROT_EXEC` for anonymous regions; no new entry (*v78: PROT_EXEC* below), v79 = **what the kernel is**: `kernel_info` (slot 256), for `/bin/uname` (*v79: kernel_info* below), v80 = **the cores' load and the network's bytes by process**: `cpu_stats`, `net_stats` (slots 257, 258), for the Task Manager's Processor and Network tabs (*v80: cpu_stats, net_stats* below), v81 = **the pointer's shape**: `set_cursor` (slot 259) (*v81: set_cursor* below), v82 = **a window resized by its frame**: `win_resizable` (slot 260), `GUI_EVENT_WINRESIZE` (*v82: win_resizable* below), v83 = **shared libraries** (§7 *Shared libraries*; `docs/SHARED-LIBS-PLAN.md`): `lib_open` (slot 261), `KAPI_IMG_LIB` in `kapi_image_info.flags` (*v83: lib_open* below), v84 = **the sound's output**: `sound_output` (slot 262), `KAPI_SND_OUT_*` — the jack, a USB audio device or HDMI behind the same producer (*v84: sound_output* below). v85 = **the sound's mixer**: every program that plays has a channel of its own (`sound_acquire`: up to 8 at once), with its volume and its mute; `sound_clients` (slot 263), `sound_client_volume` (slot 264), `struct kapi_sound_client` (*v85: the mixer* below). v86 = **AppKit**: no entry added — the kernel loads `SD:/lib/appkit.so`, copies its table at `APPKIT_TABLE_VA` and maps it into every program (the note at the top of this section). A program built for AppKit needs this kernel: its package says `kapi >= 86`, so the package manager installs it only once the new kernel runs. v87 = **AppKit's small services**: no entry added either — AppKit (shipped with the kernel) now carries the strings, the console, the `.ini` reader and the keyboard layout loader that were `user/applib.h` (docs/03 §8); a program built from now on calls them in AppKit, so its package says `kapi >= 87`. v88 = **AppKit: program starting**: no entry added — the `lx_*` functions (an app or a file started by its runner) are AppKit's (they were `user/launch.h`). The same day **SystemKit** and **NetKit** appear (docs/03 §5.9.0).

The window API moves to UIKit (2026-10-08; `docs/POCKETUI-TECH-STUDY.md` §2.2, §4, phase P2) — no kernel change, kapi stays **97**: the 46 window functions of AppKit (`kapi_create_window`, `kapi_present`,

`kapi_set_menu`, `kapi_win_list`, `kapi_tray_set`, `kapi_desk`, `kapi_drag_begin`...) are **removed** from `appkit.h` and `appkit_calls.inc` — and their names from **`appkit.abi`**: their **slots are kept**, retired (`kapi_retired_<slot>`, not declared, answering 0 or the call's old error value), so nothing is renumbered: a program built before keeps working but for its direct window calls, which now fail (Jet: rebuilt to get its window features back) —, and `appkit_ws.inc` with them. They are UIKit's **`uk_win_*`** (`user/Kits/uikit/win.h`, C linkage, entries of `uikit.abi`; docs/03 §5.10.1 the renaming), spoken to Elegant by UIKit's desktop port (`user/Kits/uikit/port/port_desktop.cpp`, the code of `appkit_ws.inc`, the replay included; the protocol `user/Kits/uikit/port/elegant.h`). AppKit keeps the kernel's side: `kapi_ws_ctl` (the transport), `kapi_fullscreen_begin` / `_end` (the kernel's primitives: no longer telling the server — UIKit's `uk_win_fullscreen_begin` does), `kapi_present_fb`, `kapi_fullscreen_direct`, the pump, `kapi_screen_size`. C programs reach UIKit through `lib/uikit.imp_c.a` (`libgen --bind-c`).

v98 = **generated vertex shaders** (2026-10-11; `docs/3DS-EMULATOR-STUDY.md` 9v, 9x): no table entry changes — `KAPI_GPU_P_VS_1WAY` (a `gpu_program` flag: its vertex and coordinate shaders are run one thread a QPU — the whole register file, 64 registers — and write their outputs to a VPM block of their own, so an output may be written before the last input is read), `KAPI_GPU_B_NOCLIP` (a `gpu_render3` batch flag: the vertices are attributes for the program's own vertex and coordinate shaders, not clip positions — the kernel neither frames nor clips them, the V3D clips from the coordinate shader's position) and `KAPI_GPU_B_INDEXED` (with it: at float `off`, `b.first` vertices of `stride` floats, then `b.count` 32-bit indices among them — triangles; the vertex shader runs once a vertex; not in a `gpu_vbuf`). A program tests `kapi_abi_version () >= 98` before using them.

v97 = **the graphics server per mode** (2026-10-08; `docs/POCKETUI-TECH-STUDY.md` §3, phase P1): + `lib_open_as` (slot **234**: a library mapped and found under an alias too — the graphics server's own UIKit under `SD:/lib/uikit.so`; §7 *Aliases*), `KAPI_IMG_ALIAS` (16) in `kapi_image_info.flags`; the server chosen from `SD:/etc/system.ini` **`shell =`** (desktop: Elegant as before; pocket / console: `SD:/bin/pocketui --serve --mode <m>`; Elegant if it fails), `KAPI_WS_REGISTER` for the started program's name, + `KAPI_WS_SWITCH` (19, a `ws_ctl` operation: the server ended, its successor started) and `KAPI_WS_IN_QUIT` (10) (*v97* below, §10). AppKit: `kapi_lib_open_as` (`-KAPI_ENOSYS` before v97).

v96 = **a window moved by another program** (2026-10-08): no table entry changes — AppKit's `kapi_win_move (id, x, y)` (Elegant's `EL_OP_WIN_MOVE`: a window of `kapi_win_list`, 0 the caller's, its client area put at x, y; not the desktop's, a topmost or a full-screen one). rdpd uses it: a window dragged in Onyx Remote is put at the same place on the Pi (its protocol 2, client message `MOVE`).

v95 = **the status area's icons** (2026-10-08): no table entry changes — Elegant keeps one icon a program (`EL_OP_TRAY_*`), AppKit's `kapi_tray_set` / `_clear` (a program) and `kapi_tray_list` / `_icon` / `_activate` (the menu bar); `KAPI_TRAY_PX` (20), `KAPI_TRAY_MAX` (16), `struct kapi_tray_info`, `GUI_EVENT_TRAY` (21) with `KAPI_TRAY_OPEN` (a double click: Elegant raises the program's first window, back from minimised) or `KAPI_TRAY_MENU`.

v94 = **several windows a program** (2026-10-07; `docs/MULTI-WINDOW-STUDY.md`): no table entry changes — the windows are Elegant's, reached by AppKit (`kapi_win_new`, `kapi_win_select`, `kapi_win_destroy`: docs/03 §6 *Several windows*). The kernel's part (`sys/wsrv.cpp`, `kern/layout.h`): a program's windows 1..16 beside its first have their canvas and their frame's two copies at fixed addresses of their own (`KAPI_WS_VA_WIN (w, p)` = 28 GB + (w − 1) × 256 MB + p × 64 MB; the library arena now ends at 28 GB), the buffers' slots `KAPI_WS_SLOT_WIN (w, p)` (`KAPI_WS_SLOTS` = 53), `KAPI_WS_KICK (window)` says which window was presented, the event queue holds 64 events (32), and the graphics server's view has 256 buffers (`USER_WS_SLOTS`; 128 before: `USER_WS_BASE` = 44 GB). Elegant holds 64 windows (16 before;

`WM_MAX_WINDOWS` = `EL_WINDOWS_MAX`). An event of a window other than the first comes with `sender` = its number.

v93 = **the volumes** (2026-10-06; `kernel/sys/volume.cpp`, `kern/volume.h`, §18): `vol_list` 230, `vol_eject` 231, `vol_mount` 232, `vol_format` 233 (`struct kapi_volume`, `struct kapi_format`, `KAPI_VST_*`, `KAPI_VF_*`, `KAPI_FMT_*`). The USB mass-storage volumes `USB1:`, `USB2:`, `USB3:` (or `USB1P1:`... per partition; `USB:` = `USB1:`) are mounted when a stick is plugged in and unmounted when it is ejected or pulled out; `SD:` is never formatted (refused by the kernel). AppKit: `kapi_vol_list`, `kapi_vol_eject`, `kapi_vol_mount`, `kapi_vol_format`.

v92 = **the 40-pin header** (2026-10-06): `gpio_ctl` (slot **229**; `kernel/sys/gpio.cpp`, `kern/gpio.h`, `KAPI_GPIO_*`): GPIO 0..27 given one process at a time (input, pulled up / down, output, an alternate function), PWM on GPIO 12 / 13 / 18 / 19, edges queued with their time, the I2C bus 1 and SPI 0 — §17 *GPIO*. **Read by GPIOKit directly** (`SD:/lib/gpiokit.so`, not through AppKit: the user's exception, docs/03 §5.10), which ships with the kernel in the package `onyx`; AppKit has no `kapi_gpio_ctl`.

v91 = **a process's tree** (2026-10-06): + `proc_tree` (slot **228**, the table's 229th entry). Each process records its spawner's pid (`CAddressSpace::GetParentPid`: `spawn`, `spawn_ex`, `spawn_ex2`; `exec` / `launch` / `exec_as` — the drawer, `run`, the file manager — start a process **without** a parent). Before v91 the only use of it was the reaper's orphan scan (`TerminateOrphans`, `kernel.cpp`): every 50 ms, a live process whose parent is gone is terminated — a whole subtree ends a level a pass, after its root. `proc_tree (pid, op, out, cap)` (`sys/kapi.cpp`) makes the tree **now**: the root, then every live process whose parent is in the set, pass after pass (256 processes at most); a kapi runs on core 0 without preemption, so nothing spawns or ends meanwhile. `KAPI_TREE_LIST` → how many descendants (up to `cap` pids written, children first); `KAPI_TREE_KILL` → the root and all its descendants terminated at once, **the leaves first** (each: `KAPI_PROC_KILLED` / −9 for `proc_wait`, `TerminateTask` → its whole thread group), the count returned and logged (`proc: proc_tree: …`); `KAPI_TREE_KILL_CHILDREN`: the descendants only. Errors: `ESRCH` (no such process, or a kernel task), `EINVAL`, `EPERM` (a kill whose tree holds the caller), `EFAULT`. A child whose start is still deferred (`SpawnProcess`: not a task yet) is not in the tree: it starts with a dead parent and the orphan scan ends it — which stays for that. Users: the **Terminal** (closing a tab ends its `cmd` and everything under it: a pipeline's stages, a script's `cmd` and its programs), **`cmd`** (Ctrl-C ends each stage with what it started), **`/bin/kill -t`**. AppKit: `kapi_proc_tree` (`-KAPI_ENOSYS` before v91).

v90 = **the table without the windows** (2026-10-05): the 36 entries of the windows (`create_window`, `present`, `set_menu`, `win_list`, `drag_begin`, `wallpaper_*`, `desk`, `set_wheel_speed`...) and the 2 of the activity shell (`register_shell`, `shell_request`: no program used them) are **removed** from `TKApiTable`, and the entries after them moved up: 266 → **228 entries**, `launch` is slot 1, `exit` 5, `pop_event` 158, `ws_ctl` 227 (the `KAPI_CHECK_SLOT`s of `kapi_abi.h`, the table below and `kern/el0.h`'s `EL0_SYS_*` have today's numbers; a slot quoted in an older note is of its time). **AppKit is rebuilt with the kernel and installed with it; no program is** — they call AppKit by name, and AppKit's window functions speak to Elegant (v89). What reads the kernel's table itself must be rebuilt: `el0test`, `faulttest` (on purpose: `KAPI_INLINE`), `sysstat` for its names (`BinUtils/kapi_names.h`, `tools/gen_kapi_names.py`) — and a program linked before AppKit (kapi v86) would call a wrong system call: scan the card first (`orr xN, xzr, #0x380000000` followed by a load from `[xN, #off]`; a `movk xN, #0x8000` after it is AppKit's own table, which is fine). On 2026-10-05 nothing on the card does but those tests. A stand-in kernel on a PC (the simulator, Koton for Windows, macOS) still has a window manager: for those builds the 36 entries are declared **after the table's end** (`#ifndef __aarch64__`), and AppKit's inline calls use them (`appkit_calls.inc`'s `KAPI_HOST`). With it: **Print Screen** and **the wheel's speed** (`wheelspeed=` of `SD:/etc/theme.txt`, read when Elegant starts) are Elegant's.

v89 = **the graphics server's mechanisms** (`kernel/sys/wsrv.cpp`, `kern/wsrv.h`): `ws_ctl` (slot 265 then; 227 since v90). The windows are leaving the kernel for a user process, **Elegant** (`SD:/bin/elegant`, `user/Servers/elegant`; `docs/GUI-USERSPACE-STUDY.md`, the stages in `docs/HANDOFF.md`). The kernel gives it mechanisms only:

- **the role** — one process at a time is the display server (`KAPI_WS_REGISTER`: the program named `elegant`, when no live process has the role); the other operations are its own (`-KAPI_EPERM`);
- **the display** — while the server owns it (`KAPI_WS_DISPLAY`), the kernel's compositor draws nothing; the server sends the rectangles it composed (`KAPI_WS_PRESENT`: copied from its memory into the off-screen buffer, sent by the frame buffer's DMA — `DisplayPresentRect`, `kernel.cpp`). A full screen (`fullscreen_begin`) and a resolution change (`screen_set`) are refused meanwhile; `screen_grab` gives the off-screen buffer (what the server shows: `vncd` keeps working);
- **the raw input** — while the server owns the display, the mouse, the cooked keys, the modifiers and the held keys, from the USB callbacks and from `inject_*` (`vncd`, `rdpd`), go to a ring of 256 events the server reads (`KAPI_WS_INPUT`) instead of `CWindowManager::OnMouse` / `OnKey` (a pointer move replaces an unread one; a full ring drops, counted). (2026-10-08, no version change) The modifiers' event (`KAPI_WS_IN_MODS`) also carries **`KAPI_WS_MOD_SUPER`** (8: a Super key, the USB modifiers' bits 3 / 7) — PocketUI's Home; the common `serve.cpp` keeps it from the window manager (Elegant sees Ctrl, Shift, Alt as before) and gives it to the policy's `mods` hook; `kapi_get_modifiers` is unchanged; (P6) `serve.cpp` gives each pointer event to the policy's **`pointer`** hook first (`policy.h`, `WS_POLICY_HAS_POINTER`): PocketUI's viewport takes the wheel and the drags over its scroll indicators; Elegant's hook is 0 (the window manager's, as before);
- **one wait** (`KAPI_WS_WAIT`, on the I/O generation: an input event wakes it at once);
- **the way back** — the server's process ends (`WsOnProcessGone`, the teardown), or calls nothing for 5 s (`WsWatch`, the compositor's loop): the display and the input are the kernel's again and the whole screen is drawn.
- **the programs' windows** (stage 2b) —
    - *attached programs* (`KAPI_WS_ATTACH`): a program whose windows are the server's keeps, in the kernel, what its pump reads: an event queue, the exit request, the wake of `pump_wait`. They are a window's (`CWindow`), so an attached program has a kernel window that is only that — never given to the kernel's window manager, one page of pixels. `pop_event`, `should_exit`, `event_mods`, `pump_sleep` and `kill`'s clean close work on it unchanged; the server queues events into it (`KAPI_WS_POST`) and asks the program to end (`KAPI_WS_EXIT`);
    - *shared buffers* (`KAPI_WS_BUF_MAP` / `_FREE`): physically contiguous (the GPU may render into a canvas), mapped in the program at the addresses its canvas and frame always had (`USER_WINDOW_CANVAS`, `_CHROME`, `_CHROME_INACTIVE` = `KAPI_WS_VA_*`) and in the server at `USER_WS_BASE` + number × 64 MB (128 slots at the top of the mmap arena, `kern/layout.h`). A buffer lives until the server frees it **and** its program no longer has it (ended, or the slot given another buffer); a server that dies leaves the programs their buffers. `CAddressSpace::UnmapContig` is new;
    - *requests* (`KAPI_WS_CALL`, a program's; `KAPI_WS_NEXT` / `_REPLY`, the server's): the caller sleeps until the server answers (the user-space file systems' pattern, `sys/vfs.cpp`): an operation,

four numbers, up to 4 096 bytes each way; the kernel stamps the caller's pid. 16 in flight; 10 s, or the server gone: `-KAPI_ESRCH`;

  - *the ring's other events*: `KAPI_WS_IN_KICK` (a program's `KAPI_WS_KICK`: its pixels changed) and `KAPI_WS_IN_GONE` (an attached program ended);

  - `kernel/gui/window.cpp` takes its windows' pixels from two functions (`WinPixelsAlloc` / `_Free`: the heap's in the kernel; Elegant, built with `WIN_PIXELS_HOOK`, gives the shared buffers).

  - two more slots for a program's shared memory: its copy of the wallpaper (`KAPI_WS_SLOT_WALLPAPER`, at `USER_WALLPAPER_CANVAS`) and a transfer buffer (`KAPI_WS_SLOT_XFER`, 13 GB + 256 MB: the pixels of `uk_win_read`);

  - **the window as last presented** (2026-10-06, no kernel change): a program paints in its canvas, the memory Elegant reads, so a reader that came between a repaint's first stroke and its present got a picture half made -- Onyx Remote flickered when the pointer moved over a window (`rdpd` reads a window for about 100 ms a round; measured with `tools/tests/rdpd/flicker_bench.py`: 56 of 450 pictures of the Control Panel half painted in 25 s, 0 since). The canvas' shared buffer is now *canvas, one 64 KB page of control, a second copy of the pixels* (`WinPixelsAlloc`, `core.cpp`; a canvas that would not fit twice in a slot's 64 MB has none). UIKit's desktop port (AppKit's until 2026-10-08) asks where they are after the window is made or grown (`EL_OP_SHOT`, `uikit/port/elegant.h`); while Elegant wants it (`el_shot.want`: a `win_read` of that window in the last 5 s), `uk_win_present` copies the rows shown into the second copy before its `KAPI_WS_KICK`, raising `el_shot.seq` before and after (odd: being made). `win_read` (part 0) gives that copy once a present has filled it since it was asked, and reads it again if a present came meanwhile; before that, the canvas itself as it always did;

  - *who has the keyboard* (`KAPI_WS_FOCUS`: the server says; `key_held` and a pad's `focus` answer by it -- the kernel's window manager keeps the keys' state and the modifiers, fed as before) and *a process's name* (`KAPI_WS_PROC_NAME`: the lists of the open programs);

- **the start** (`WsBootStart`, before init) — the kernel starts `SD:bin/elegant --serve` at every boot and waits (5 s at most) until it has the display; init then starts the desktop, whose programs have their windows in Elegant. There is no other window manager (§10): without a server, the kernel's console. (v97) The server is the one `SD:/etc/system.ini` `shell =` names — PocketUI for pocket / console —, Elegant if that one fails (§10 *The server per mode*).

The window calls speak to Elegant — **UIKit's** since 2026-10-08 (`uk_win_*`, `user/Kits/uikit/win.h`; its desktop port `user/Kits/uikit/port/port_desktop.cpp`; docs/POCKETUI-TECH-STUDY.md phase P2), AppKit's `kapi_*` window functions before (`appkit_ws.inc`, removed with them): the protocol is `user/Kits/uikit/port/elegant.h` (private to the desktop UIKit's port and Elegant — it was `user/Kits/appkit/elegant.h` —: 31 operations, each doing what the kernel's call of the same name did). Each graphics server has its own UIKit (loaded under `SD:/lib/uikit.so`, §7 *Aliases*), whose port is the only code speaking that server's protocol; AppKit keeps the transport, `kapi_ws_ctl` (`KAPI_WS_CALL`, `KAPI_WS_KICK`, `KAPI_WS_ACTIVE`). A call that finds no server (it is being started, or started again) waits for it, 5 s at most; what the program asked is kept by the port and asked again of an Elegant started again (the replay). (The builds against a stand-in kernel with a window manager -- the PC's simulator -- have the port relay each call to that table.) The kernel still serves: the pump (`pop_event`, `should_exit`,

`pump_sleep`, `post`), `screen_size`, `screen_grab` (the off-screen buffer), `inject_*` (to the server's ring), `get_modifiers`, `key_held`, `draw_text_buf` (UIKit's `uk_win_draw_text` draws into the canvas with it), the surfaces, the clipboard — and **the full screen**: `fullscreen_begin` / `present_fb` / `fullscreen_direct` / `fullscreen_end` are the kernel's as before (its buffer, the direct mode: no round trip a frame), on the program's kernel-side window (UIKit's `uk_win_fullscreen_begin` makes the program a window in Elegant first, then calls AppKit's `kapi_fullscreen_begin`, which no longer does it itself); the kernel tells Elegant (`KAPI_WS_IN_FULLSCREEN`), which sends that program all the input and shows nothing meanwhile (`KAPI_WS_PRESENT` answers `-KAPI_EBUSY`), then draws the whole screen again. `screen_set` is the kernel's too: its compositor does the resize between two of the server's presents and tells the server (`KAPI_WS_IN_SCREEN`).

The callbacks' value (`gui_handler`: sender, event, value) is the type `gui_value`: `long` on Onyx (64 bits: a pointer event packs its wheel, buttons and position there), `long long` where `long` has 32 bits -- the Windows build of the apps (`pc/Koton`, docs/03 *Koton for Windows*), whose table is filled by a Win32 layer. On Onyx it is the same type as before: no ABI change, no new version.

Categories of exposed functions

Category	Examples
Windowing (on Onyx: UIKit's `uk_win_*` since 2026-10-08 — these entries only in a PC's stand-in kernel, *v90*)	`create_window(_ex)` (the canvas; **0** when the client area is bigger than the screen — `g_nScreenWidth/Height`; before v66, 1024 × 768 — or memory is short — an app must check it: drawing into a null canvas faults, and the app is killed), `resize_window` (the client size shown, ≤ the canvas made at creation; the frame — `OuterW/H`, the chrome copies' size — follows it, and the app redraws its chrome: `uk_decorate_window`), `move_window`, `present`, `exit`. Window flags: `WIN_FLAG_BORDERLESS`, `WIN_FLAG_BACKMOST` (desktop, bottom band), `WIN_FLAG_TOPMOST` (the menu bar: top band, never the active app nor the key target; at y=0 it reserves its smallest logical height — `CWindowManager::TopInset()` — so auto-placement and title-bar drags stay below it), `WIN_FLAG_TRANSPARENT` (client blitted with the magenta key), `WIN_FLAG_SYSTEM` (a shell component — menu bar, notifications, panel, app list: skipped by `list_windows`, so never in the taskbar; a plain flag bit, no ABI change). The z-order is three bands: backmost < normal < topmost (`Add`/`RaiseLocked` keep them). The **key**

Category	Examples
	target is the frontmost non-topmost window; the **active app** (menus, chrome highlight uses the key target) is the frontmost window that is neither topmost, backmost nor borderless.
Menu bar (v39)	`set_menu(spec, handler)` stores the app's menu spec (≤ 2 KB; lines `M<title>`, `I<id>\t<label>\t<shortcut>`, `-`) + a `GUI_EVENT_MENU` (14) handler on its `CWindow`; `get_menu(buf, cap, title, tcap)` returns the **active app**'s spec + title and a serial that changes with the active window or its menu (0 = none); `menu_command(id)` queues `GUI_EVENT_MENU(id)` to the active window (`MENU_QUIT` = -1 → `RequestExit`, like the close box). Used by `menubar` + `uikit::Menu`.
Launch/management	`launch`, `toggle_app`, `raise_app`, `exec`, `kill`, `kill_pid`, `proc_tree` (v91)
Threads (v67)	`thread_create(fn, arg, stack_size, name)` → tid ≥ 2 (main: 1), −1 no memory, −2 too many (32); `thread_exit(code)` (the main thread: the process); `thread_join(tid, timeout_ms, &code)` → 0, −1 timeout, −2 none / joined already, −3 itself; `thread_self`. `mutex_create`/`mutex_lock(h, timeout)`/`mutex_unlock` (recursive), `event_create(manual, initial)`/`event_set`/`event_reset`/`event_wait(h, timeout)`, `barrier_create(count)`/`barrier_wait` (1 for the last one in), `sync_close` — handles, 256 per process; timeouts in ms, 0 = only try, `KAPI_WAIT_FOREVER`. `post(fn, ctx, value)` → queued for the pump (−1 full: 256); `pump_wait(timeout)` sleeps until an event / a post / the close box, pumps → what was pending. See §7.
Word waits, priority (v68)	`wait_word(addr, expected, timeout_ms)` sleeps while the 4-byte word `*addr == expected` → 0 (woken, or the value differs), 1 timeout (0 ms: only

Category	Examples
	check), −1 bad address (unaligned, unmapped); `wake_word(addr)` → the sleepers woken. Keyed by the **physical** address (a word of a shared surface wakes across processes); the 100 Hz tick also reads every sleeping word and wakes those that changed — an app core's write needs no `wake_word` (≤ 10 ms). `thread_priority(tid 0 self / 1 main / ≥ 2, prio 0 / 1 / −1 ask)` → the previous one, −1 bad prio, −2 no such thread: a "real time" task is picked first when ready and a tick preempts an app for it, while it yields by itself (§5). See §7.
Enumeration	`list_apps`, `list_windows`, `list_tasks`, `list_procs`, `get_datetime`
Widgets	none since v29 (the kernel-drawn `add_button`… removed: §10.3; apps use uikit)
Events	`pump_events`, `wait_for_exit`, `should_exit`, `set_key_handler`, `set_click_handler`, `set_pointer_handler` (full pointer stream, v22 — incl. `GUI_EVENT_PTR_WHEEL`, a signed scroll-notch delta in the `lValue` wheel field via `GUI_PTR_WHEEL`)
App-drawn text	`draw_text`, `font_width`, `font_height`
System-call statistics (v74)	`proc_stats(pid, out)` (pid 0: the caller) → 0 and `struct kapi_syscall_stats { syscalls, emulated; rate, slots; top_slot[8], top_count[8]; reserved[4] }` (104 bytes): the `svc`s since the process started, the ID register reads emulated, the calls per second (the last full window ≥ 1 s), the table's slot count, the 8 slots most called (a slot = the field's index in `TKApiTable` in 8-byte words, `version` = 0; `user/BinUtils/kapi_names.h`, generated by `tools/gen_kapi_names.py`, names them) → −1 no such process / a kernel task, −2 a bad pointer.
Protected mode (v73)	`pop_event(struct kapi_event *ev)` →

Category	Examples
	1 and the next window event `{ handler, sender, value, event, mods }` (the handler **not** called), 0 none / no window; `event_mods(mods)` sets what `get_modifiers` reports while a key handler runs → the previous value (`0xFFFFFFFF` = live); `pop_post(struct kapi_posted *p)` → 1 and the next posted call `{ fn, ctx, value }` (not run), 0 none; `pump_sleep(timeout_ms)` = `pump_wait` without the pump → what is pending, −1 not a process. The user-side `pump_events` / `wait_for_exit` / `pump_wait` of a protected app (§6) are built on them. `user/Kits/appkit/appkit.h`: the wrappers (version ≥ 73) and `kapi_is_protected()`.
Files	`open/read/fsize/close`, `save_file`, `opendir/readdir/closedir`, `mkdir/remove/rename`, `chdir/getcwd` (current working directory, inherited by children). `fsize` (and `readdir`'s size) is clamped to 4 GB − 1; **`fsize64(h)` (v59)** gives an exFAT file's real 64-bit size. `rename` across two volumes fails (−1): the caller copies then deletes (FatFs' `f_rename` would otherwise rename inside the source volume). All of these (and the streams, `seek`, `fsize64`, `chdir`) work on **`RAM:`** paths too (v71, §16): the path is resolved first (`ResolvePath`: relative to a current folder on `RAM:` as well), then a `RAM:` path goes to `sys/ramfs.cpp`, a provider's (`FTP:`) to `sys/vfs.cpp`, the rest to FatFs; since v73 a handle is a per-process handle (§6, *Per-process handles*) whose entry records the kind (FatFs, RAM, provider). **v75:** POSIX open files beside them: `file_open/read/write/seek/truncate/sync/stat/close` (64-bit offsets, pread / pwrite, `O_CREAT/EXCL/TRUNC/APPEND`), `path_stat/unlink/mkdir/rename/utime`, `dir_read` (255-character names), unlink / rename of open files, `-KAPI_Exxx` errors (*v75: files and*

Category	Examples
	processes below).
Volumes (v59)	FatFs volume strings (`FF_STR_VOLUME_ID`, docs/05 §13): **`SD:`** = the SD card's first FAT volume (partition 1, the boot FAT32 one, found as before; **`SD0:`** is an alias), **`SD1:`** **`SD2:`** **`SD3:`** = MBR partitions 2–4 (`FF_MULTI_PARTITION`) (FAT12/16/32 or **exFAT**, mounted at boot when present), **`USB1:`** **`USB2:`** **`USB3:`** (v93, §18: the USB devices whole, mounted when plugged in; **`USB:`** is an alias of `USB1:`) or **`USB1P1:`**… `USB3P4:` (the partitions of a device that has several), `FD:`, `NVME:` (declared, not mounted). `ResolvePath`: a volume prefix is upper-cased (`sd1:` → `SD1:`, `SD0:` → `SD:`), a path starting with `/` is relative to the **current directory's volume** root, anything else to the current directory. **`RAM:`** (v71) is the RAM volume (§16), not a FatFs one. The four SD volumes share one FatFs lock slot (`LockSlot`, `sys/fslock.cpp`): they are one card, one command at a time.
Streams/processes	`pipe`, `file_in/out`, `stream_read(_nb)/write/close/eof`, `stdin_read`, `stdout_write`, `spawn`, `wait`, `proc_done`, `get_args`
Modal dialogs	none (v9/v10 `message_box`, `file_open`, `file_save` removed: §10.5)
Desktop	`screen_size`, `wallpaper_generate`, `wallpaper_buffer`, `wallpaper_commit`, `cursor_pos`
Appearance/keyboard	(`set_window_theme`, v16: removed — the chrome is drawn user-side), `set_keymap` (load a country map *by name* — deprecated: the kernel compiles in **no** maps, so it always returns 0; use `set_keymap_data`), `get_keymap`, `kbd_ready` (USB keyboard attached? v26 — informational; `keyb` no longer needs to poll it), `set_keymap_data` (load a layout from a `.kmap` blob, v27 — records it in a persistent snapshot and installs it on the keyboard whenever it attaches, so it needs no keyboard to be present;

Category	Examples
	returns 1 once the blob is accepted, 0 only on a malformed blob. This removed the old boot race where `keyb` could time out waiting for USB enumeration and leave the keyboard map-less. **Ctrl with `-`, `=` / `+`, `0`** (2026-10-01, `CInputTask::SetKeyMapData`): Circle looks a key held with Ctrl up in the layout's Ctrl column unless it is a letter, and the `.kmap` files leave that column empty there -- nothing came; the snapshot's empty Ctrl entry of the key whose own character is `-`, `=` or `+`, or `0` (its own, or its Shift one on AZERTY) now gets the keypad's key of that character (`KeyKP_Subtract` / `KeyKP_Add` / `KeyKP_0`), whose string Circle sends with or without Ctrl: the app receives `-`, `+`, `0` and `get_modifiers()` says `MOD_CTRL` (Jet Browser's zoom keys; letters and the keypad untouched; Ctrl+Shift+= still nothing -- Shift's column wins)), `app_dir`
Logging / memory	`klog_read`, `set_verbose`, `get_verbose`, `meminfo` (total/free/app KB + page size, v23), `sbrk` (per-process heap, v24)
Networking (v21, v37)	`net_status` (live: 0 as soon as the Wi-Fi association drops — `CWPASupplicant::IsConnected ()` — not only before the first DHCP bind), `tcp_connect`, `tcp_send`, `tcp_recv`, `tcp_close`; server side (v37): `tcp_listen(port)` → listening handle (Circle `CSocket::Bind`+`Listen`; `-6` = port in use), `tcp_accept(h, ip, cap)` → **blocks** until a peer connects, returns a connected handle + the peer IP (Circle `Accept`). All handles share the 16-slot table in `sys/net.cpp` and are reclaimed when the owner dies. Used by `/bin/telnetd`.
Power (v25)	`reboot` (restart the machine — applies settings read only at boot, e.g. the WLAN config rewritten by *wpaconf*)
Remote screen (v38)	`screen_grab(dst, w, h)` — runs `CWindowManager::Composite` (windows,

Category	Examples
	wallpaper, cursor; `bCountFrame = FALSE` so the watchdog's fps stays the real compositor's) straight into the caller's `w*h` 0x00RRGGBB buffer (`w`/`h` must be the screen size), or returns **2** without touching it when `g_nScreenGen` has not changed since the previous grab into the same buffer (vncd then skips the diff / encode); `inject_pointer(x, y, buttons, wheel)` → `OnMouse` (+ `OnMouseWheel`), `inject_key(keys)` → `OnKey` — the same paths as the USB mouse/keyboard. Used by `/bin/vncd`.
Windows as objects (v56)	For the window-level remote desktop (`/bin/rdpd`): `win_list(out, max)` fills `struct kapi_win_info` bottom to top (`CWindowManager::Snapshot`): `id` (`CWindow::Id`, a serial never reused), owner pid, the client area on the screen (x y w h; the full-screen window: the whole screen, `KAPI_WIN_FULLSCREEN`), `WIN_FLAG_*`, alpha, `gen` (`CWindow::Gen`, bumped by every `Damage` — drawn, moved, resized, raised — and by `present_fb`), `KAPI_WIN_KEYS` (the key target), the title, the frame (`ow oh` = `OuterW/H`, `il it` = the insets, 0 without a frame) and `chromeGen` (bumped by `get_chrome`, i.e. when the app redraws its frame, by a resize, and once more by the first present after that: the frame read again whole). The state also says `KAPI_WIN_MINIMISED` / `KAPI_WIN_OFFDESK` (not shown: rdpd sends a hidden window's pixels only when it is shown again) and the window's desk (v65). `win_read(id, part, x, y, w, h, dst, stride)`: a rectangle of the canvas (part 0; the full-screen window: its back buffer, or the screen itself when direct) or of a chrome copy (1 active, 2 inactive), clipped. `win_raise(id)` = `Raise`, `win_close(id)` = `RequestExit` (as the close box). The **desktop** is listed first as a pseudo-window, `KAPI_WIN_DESKTOP` (0xFFFFFFFF,

Category	Examples
	backmost, screen-sized, `gen` = the wallpaper's counter + the backmost windows'): `win_read` of it (whole only) runs `CWindowManager::CompositeDesktop` (the wallpaper + the backmost windows) straight into the caller's buffer. A `CWindow` is never freed (see `Composite`), so an id found in the snapshot is safe to read.
Generated code (v58)	`code_alloc(size)` — zeroed memory the app may **write and execute** (a JIT's code: `gcemu`'s PowerPC → AArch64 translation), rounded up to 64 KB pages: `CAddressSpace::CodeAlloc` maps fresh owned frames (`MapNewPage`, freed at teardown) with `KPAGE_ATTR_APP_RWX` (EL1 RW, PXN = 0) in a per-process bump arena `USER_CODE_BASE` (14.25 GB) … `USER_CODE_END` (15 GB, 768 MB); 0 when full. Circle clears `SCTLR_EL1.WXN`, so a writable page may be executable. After writing code the app cleans the D-cache and invalidates the I-cache over it (`DC CVAU`, `DSB ISH`, `IC IVAU`, `DSB ISH`, `ISB`).
File seek (v57)	`seek(h, pos)` — the read position of a file opened with `open` (`f_lseek`); 0, or −1 (a VFS provider's file: not seekable). A file bigger than 4 MB gets its **cluster map** at its first seek (FatFs fast seek, `CREATE_LINKMAP`, docs/05 §11), freed by `close`: any position without walking the FAT. For `gcemu`'s disc images (1.4 GB, read on demand).
Shared surfaces (v35)	`surface_create(w, h)` → an id > 0: a page-aligned, physically contiguous 0x00RRGGBB buffer (`CSurface`, `kern/gui/surface.h`) owned by the caller; `surface_map(id)` maps the **same frames** into the caller (the id travels to another process over IPC) → their address; `surface_size`, `surface_present` (yields toward the process composing it), `surface_destroy`. A process mapping a surface it does not own becomes one

Category	Examples
	of its **users** (v65, ≤ 4): the frames are freed once the owner let it go (`surface_destroy`, or its end: `DestroyByOwner`) **and** every user has ended too — no process keeps a mapping of freed frames, whichever ends first. The Control Panel's applets draw into their host's surface this way (`user/Include/applet_proto.h`, the developer guide).
IPC services (v40)	`ipc_register(name)` makes the caller the service `name` (≤ 16 services; a name held by a live process is refused; freed when the owner dies — `IpcOnProcessGone`); `ipc_lookup(name)` → pid or 0. Messages use the per-process mailboxes (`mailbox_send`/`mailbox_recv`), now up to **512 bytes** (`MAILBOX_MSG_MAX`). The kernel itself can post: `IpcPost(service, type, data, len)` (from pid 0, to a running service: FALSE when it is not registered) and over it `IpcNotify(title, text)` → the `notify` service (e.g. "Network … connected"); Print Screen → the `screenshot` service (§2, *CInputTask*). A **blocking** `mailbox_recv` sleeps until a message lands (`IoWait` on the I/O generation; `CMailbox::Push` calls `IoWake`) — until 2026-10-03 it was a loop of `Yield`, the receiver ready for ever: `clipd`, the one service that waits this way, took most of core 0 (`ps`: state `R`, no system call).
Clipboard (v40)	`clipboard_set(type, data, len)` / `clipboard_get(&type, buf, cap, &serial)`: one typed blob (≤ 64 KB) held by the kernel so it outlives the app that copied (`CLIP_TEXT` 1, `CLIP_FILES` 2, `CLIP_FILES_CUT` 3 — paths `\n`-separated); the serial bumps on every set. Since the shared clipboard (2026-10-01: the service `clipd`, `user/Kits/uikit/clipboard.h`, `docs/clipboard/README.md`) it is only a **fallback**: `clipboard.h` writes the

Category	Examples
	kernel's copy too, and reads it when clipd cannot be reached; the history (10 items, any format) is clipd's, in its own memory.
Window opacity / session (v40)	`set_window_alpha(0..255)` on the caller's window: `CWindow::DrawTo` blends chrome + client over what is below (`BlendRect`, magenta-keyed if `TRANSPARENT`; 0 = not drawn) — used for fades. `shutdown(mode)`: `f_mount(0)` unmounts/flushes the SD card, then `reboot()` (mode 1) or ACT LED off + `halt()` (mode 0).
Full-screen apps (v41)	`fullscreen_begin(&w, &h)` maps a kernel-owned, screen-sized back buffer (`CWindowManager::EnsureFullscreenBuffer`, 64 KB-aligned) at `USER_FULLSCREEN_CANVAS` (15 GB) and makes the caller's window (created if missing, moved to 0,0) the **full-screen window**: the compositor task skips its frames, `OnMouse`/`OnMouseWheel` send the whole pointer stream to it in screen coordinates and it is the key target. `present_fb()` copies the buffer into the displayed `C2DGraphics` buffer + `UpdateDisplay()`, then yields. It first waits, yielding, for the compositor's own display DMA to end (`DisplayPresentIdle`, kernel.cpp): the compositor yields while the desktop's last frame is on its way and holds the frame buffer's DMA meanwhile, and Circle's `CBcmFrameBuffer::SetArea` waits for that DMA in a loop that never yields — an app that took the full screen and sent its first frame in those few milliseconds stopped core 0 for good (the compositor never ran again to end its DMA) and the hang watchdog restarted the Pi (`lastcrash.txt`: *display: waiting for the display DMA*, core 0 in `SetArea` under the app's task; fixed 2026-10-05; the test: `tools/tests/fsrace/`). Any other task that calls `C2DGraphics::UpdateDisplay`

Category	Examples
	itself must call `DisplayPresentIdle` first. `fullscreen_end()` — or the window's removal when the app exits — gives the desktop back. The buffer is made at the first `fullscreen_begin` and made again, bigger, when the screen has grown since (`screen_set`, v66; the old one is left allocated, as the wallpaper's: it may still be mapped) — before, it kept its first size and a full-screen app after a change to a larger resolution wrote past its end, over the kernel heap. `screen_grab` (VNC) returns the full-screen buffer meanwhile. **v55** `fullscreen_direct(&w, &h, &stride)` (after `fullscreen_begin`): maps the framebuffer the display scans out (`CBcmFrameBuffer`, the `C2DGraphics` display, 32 bpp) at `USER_FULLSCREEN_SCREEN` (15.5 GB), normal **uncached** (`KPAGE_ATTR_APP_SCREEN`); `present_fb` then copies nothing (it only yields); `screen_grab` (VNC) reads the screen through the same uncached mapping in the grabber's address space (the kernel's own map of the framebuffer is Device memory, slow to read). What is drawn there shows at once (no double buffering: tearing is possible); the GPU renders there directly (below 1 GB, contiguous), so a full-screen GPU frame costs no copy at all. Returns 0 when not possible (keep the back buffer) -- **on the Pi 5 while the alpha fix is on** (the default; `cmdline.txt` `opaque=0` turns it off), its display taking the alpha byte the programs leave at 0 on older bootloaders: there the pixels are made opaque on their way to the screen (`C2DGraphics`, fork patch 31, docs/05).
Drag & drop (v42)	`drag_begin(type, data, len, label)` — only while the left button is held — copies the payload (≤ 4 KB: 1 text, 2 `\n`-separated paths) into a kapi-side buffer and calls `CWindowManager::DragBegin(src,`

Category	Examples
	`label)`. While the session lasts, `OnMouse` sends `GUI_EVENT_DRAG_OVER` (16) to the window under the cursor (`DND_F_LEAVE` when it leaves), and `Composite` draws a label badge next to the cursor (a **+** when Ctrl is held). At the left-button release, `DndFinishLocked` sends `GUI_EVENT_DROP` (15) to the window under the cursor — `(flags << 32) | (x << 16) | y`, client coords, `DND_F_COPY` if Ctrl — and `GUI_EVENT_DRAG_DONE` (17) to the source: `(flags << 32) | target pid` (`CWindow::OwnerPid`, set by `CreateWindow`), flags `DND_F_COPY` / `DND_F_CANCEL` (Esc, via `OnKey`) / `DND_F_DESKTOP` (no window or a backmost one). All three go to the pointer handler; the normal pointer stream still reaches the source (its capture), so its widgets see the button go up. The target reads the payload with `drag_data(&type, buf, cap)`. A window removed mid-drag ends it. **Modifiers**: `get_modifiers()` = `MOD_CTRL` 1 / `MOD_SHIFT` 2 / `MOD_ALT` 4 — from the USB keyboard's raw report (`RegisterKeyStatusHandlerRaw` in **mixed mode**, so the cooked key path is unchanged) or `inject_modifiers()` (vncd, from the RFB Control/Shift/Alt keysyms). **Per key event**: `OnKey` stores the modifiers in each `GUIEvent` (`nMods` = the global state OR the xterm parameter of `ESC[1;<m>X` / `ESC[n;<m>~`, m − 1 = Shift 1 + Alt 2 + Ctrl 4, which Circle's keymap and vncd send for navigation keys); `kapi_pump_events` sets `CWindow::m_nKeyEventMods` around the app's key handler, and `get_modifiers()` returns it while the handler runs — so Shift+arrow selects even when the live state is late or clobbered (VNC, a second keyboard).
Network tools (v43)	`net_ping(host, seq, timeout_ms, ip, cap)` resolves the host (`CDNSClient` unless a dotted quad), builds an ICMP

Category	Examples
	echo request (id `0x4F4E`, 32 data bytes, `CChecksumCalculator`) and sends it with `CNetworkLayer::Send(…, IPPROTO_ICMP)`; the reply is read from Circle's **secondary ICMP queue** (`EnableReceiveICMP`, enabled only while a ping is in flight, `ReceiveICMP`), matching type 0 + id + seq + sender, yielding while it waits → RTT in µs or `-1` down / `-3` unresolved / `-4` timeout / `-5` send failed. `net_resolve` = DNS only. `net_info` = a text dump for `netstat`: `up`, `link` (`ethernet`, `wlan` or `none`: the port the bring-up took, `system.ini network=`; since 2026-10-09), `hostname`, `ip`, `mask`, `gateway`, `dns`, `dhcp` lines (`CNetConfig`), then one `tcp <handle> listen|conn <local port> <remote ip> <pid>` per socket slot (`TSocketSlot.bListen`); since v75 also `udp bound <default peer
User-space file systems (v44)	`sys/vfs.cpp`, `kern/vfs.h` — FUSE-like. A **provider** app calls `vfs_register(prefix)` (`/bin/ftpfs`: `FTP:`, `FTPS:`). `kapi_open`/`read`/`fsize`/`close`, `save_file`, `opendir`/`readdir`/`closedir`, `mkdir`/`remove`/`rename` on a path with a registered prefix (`VfsHandles`) become **requests** (`VfsCall`): the calling task fills a slot (op, path, path2, a0–a2, a **kernel copy** of the payload), sets the provider's `CSynchronizationEvent` and waits on the slot's own event (200 ms re-checks: provider alive via `IpcPidAlive`, 120 s timeout). The provider takes it with `vfs_next` (blocking = up to 0.5 s, so it can also poll its mailbox), reads the payload with `vfs_req_data`, answers with `vfs_reply(id, status, data, len)` (copied into a kernel buffer, then into the caller's). Ops: `OPEN` (→ fid + size), `READ` (fid, offset, ≤ 64 KB), `CLOSE`, `LIST` (packed `u32 size, u8 is_dir, name\0` entries), `SAVE`, `MKDIR`, `REMOVE`, `RENAME`. Provider-backed handles live in static tables (`s_File`, `s_Dir`), so the file

Category	Examples
	kapis tell them from FatFs `FIL`/`DIR` by address. `FTP:`/`FTPS:` are **auto-started**: the first use execs `SD:/bin/ftpfs` and waits up to 5 s for it to register. A dying provider is dropped and its pending requests fail (`VfsOnProcessGone`, from `IpcOnProcessGone`). Streams (`kapi_file_in`: `cat`, redirections) go to FatFs (and to the RAM volume on `RAM:`, §16), not to a provider.
Volumes' room, RAM: (v71)	`vol_info(path, out)` → 0 and `struct kapi_vol_info { total, free, used; files, dirs; flags; type[12] }` for the volume of `path` (`"SD:"`, `"SD1:/roms"`, `"RAM:"`, a relative path: the current folder's), −1 no such volume. A FatFs volume: `f_getfree` (its free clusters — FSINFO on FAT32, else counted once by FatFs: the first call on a big card can take a moment), `type` `FAT12`/`FAT16`/`FAT32`/`exFAT`. **`RAM:`** (§16): its size, the pages its files take, what can still be written (also bounded by the free page memory less the reserve), its files and folders, `flags` `KAPI_VOL_RAM` (lost at a restart), `type` `RAM`. `/bin/df` prints it; `/bin/ramtest` uses it to check that the memory comes back.
Wi-Fi scan (v45)	`wlan_scan(out, max)` → `struct kapi_wlan_ap` (ssid, bssid, security `WLAN_SEC_OPEN`/`WEP`/`WPA`/`WPA2`, channel, freq, level dBm, connected), strongest first, one per BSSID. `NetWlanScan` in `sys/net.cpp` — **no Circle patch**: it drives the BCM4343 firmware's *escan* through `CBcm4343Device::Control ("escan 5")`, collects `ReceiveScanResult` messages for ~3.5 s (the firmware's `brcmf_escan_result_le` layout, as in hostap's `driver_circle.cpp`), then `escan 0`. Security from the capability privacy bit + the RSN (48) / WPA vendor (221) IEs; `connected` = the BSSID `GetBSSID()` reports while `CWPASupplicant::IsConnected()`.

Category	Examples
	wpa_supplicant reads the same result queue for its own scans: while it is still looking for its network, a scan here may take its results (it scans again). Used by `/bin/wifiscan` and `wpaconf`.
Master volume (v60)	`sound_volume(volume 0..10, mute 0/1)` (−1 keeps a value) → the volume `
Wi-Fi join (v60)	`wlan_reconnect()` — `NetWlanReconnect` (a core-3 request with `netcore=1`): wpa_supplicant's SIGHUP handler, caught at link time (`--wrap=eloop_register_signal_reconfig`, `kernel/Makefile`; docs/05 §14) and run from its own event loop (a 0 s eloop timeout) — deauthenticate, read `SD:/etc/wpa_supplicant.conf` again, rescan, join the highest priority network in range — then `CDHCPClient::Restart()` (a new lease: another network). 0 asked, −1 no Wi-Fi running. Used by the Wi-Fi menu (`wifimenu`).
Low-latency sound (v68)	For the sound owner: `sound_config(chunk_frames 64..1024 (0: 1024), ahead 1..4 (0: 4))` → the latency now in frames, (ahead + 1) × chunk (−1 not the owner); `sound_map()` → the **mapped PCM ring** (`struct kapi_sound_ring`: one 64 KB page, 8192 frames, `wr` moved by the app after its frames, `rd` by the kernel, `dry` underruns, the chunk / ahead now), mixed with the voices and the stream until the owner releases the output — plain memory, so an **app core** fills it (`kapi_sound_ring_write`). Both are back to the defaults / off at `sound_release` or the owner's death. See §12. Test: `/bin/ringtest`.
Sound (v46)	`sound_acquire` (1 ok / 0 busy / −1 no audio: the caller becomes the owner; the output starts on first use), `sound_release`, `sound_start` / `sound_stop` — the voices, **retired on 2026-10-05** (the synthesizer left the kernel: AudioKit's `ak_fm_*`, docs/03 §5.7; the slot stays and answers −1),

Category	Examples
	`sound_write(s16 stereo frames, n)` → frames taken (PCM ring, non-blocking), `sound_status(&rate, &free, &owner)`. Non-owners get −1; the owner's exit silences it. See §12.
Run as (v49)	`exec_as(path, args, name)` → `ExecPath` with the process named `name` instead of after the path (1 = started). User space runs a format's program this way (`launch.h`: `SD:/bin/basic` for an app's `main.bax` is named after the app).
Monitor, time zone (v69)	`screen_native(&w, &h)` → 1 and the monitor's own resolution, 0 unknown: the EDID's first block (the firmware's `PROPTAG_GET_EDID_BLOCK`, Circle's `CBcmPropertyTags`), its header checked, the **first detailed timing descriptor** (bytes 54..71, the preferred timing) read — the active pixels, 8 low bits and 4 high ones of a shared byte, horizontally (bytes 2, 4) then vertically (5, 7); 0 when it is a display descriptor or too small (no monitor, an analog adapter). `set_timezone(minutes)` → 1: the local time's offset from UTC now (−720 .. 840, `CTimer::SetTimeZone`: `get_datetime`, the menu bar's clock), 0 out of range — `system.ini`'s `timezone=` sets it at boot. Setup (the first-run wizard) marks the monitor's size "Best" and changes the zone as it is picked.
USB MIDI (v68)	`midi_read(ev, max)` → up to `max` `struct kapi_midi_event` (`time_us` — the kernel's µs clock, `CTimer::GetClockTicks`, which `kapi_clock_us` reads user side, on an app core too —, `cable`, `status`, `data1`, `data2`, `device` (its `umidiN`), `length` 1..3), oldest first, never waits; −1 bad arguments. One queue for the system (256 events, the newest dropped when full). `midi_devices()` → attached. Circle's USB device factory makes a `CUSBMIDIHostDevice` for every class-compliant MIDI interface (`int1-3-0`) and its `CUSBMIDIDevice` names itself `umidiN`;

Category	Examples
	the input task finds `umidi1..4` every 100 ms (a hot plug too), registers its packet handler (`MidiPacket`: at USB-completion time, stamps the packet and queues it under an IRQ spin lock) and a removed handler that frees the slot. No Circle change. Test: `/bin/miditest`.
Gamepads (v50)	`pad_state(index, out)` → 1 and `struct kapi_pad` filled for USB gamepad 0..3 (`KAPI_PAD_MAX`), else 0: `vid`/`pid`, `props` (Circle's `TGamePadProperty`, bit 0 = a known mapping), `focus` (the caller's window has the keyboard), `seq` (reports received), `nbuttons`/`buttons`, `naxes`/`axes[16]` (value, min, max), `nhats`/`hats[6]` (0..7 = N..NW). Raw state: for pads Circle knows (Xbox 360 / One, PS3 / PS4, Switch Pro) `buttons` are its `TGamePadButton` bits, for other HID pads the report's own. The input task finds `upad1..4` (Circle's names) every 100 ms, registers a status handler that copies each report into a slot under a sequence count (odd while writing: the handler runs at USB-completion time), and a removed handler that frees the slot. The mapping to one button set is user space (`user/Include/gamepad.h`, `SD:/etc/gamepad.ini`).
App cores (v51)	`core_acquire()` → 2 or 3 (a free app core, now the caller's) or −1; `core_run(core, fn, arg, stack_top)` → 0, or −1 (not yours / still running / `fn` or the stack not a user address): the core calls `fn (arg)` in the caller's address space on the given stack (16-byte aligned, the caller's memory); `core_state(core)` → `KAPI_CORE_IDLE` (0: `fn` returned), `KAPI_CORE_RUNNING` (1), `KAPI_CORE_FAULT` (−2: `fn` faulted and was stopped, logged to kmsg) or `KAPI_CORE_NOTYOURS` (−1); `core_release(core)` stops `fn` if it runs and frees the core (done at the app's exit anyway). `fn` makes **no kapi call and no allocation**. See §14.

Category	Examples
GPU (v52)	`gpu_info(buf, cap)` → 1 (the V3D is up; `buf` = "V3D 4.2 (1 core)") or 0 (`buf` says why); the first call brings the GPU up. `gpu_draw(v, n, clear, pixels, w, h, stride)`: `n` vertices `struct kapi_gpu_vertex { float x, y, z; u8 r, g, b, a; }` (a triangle list; normalized device coordinates, y up, z −1 near … 1 far; depth test *less*, both faces; colours interpolated) rendered by the GPU into `pixels` (0x00RRGGBB, `w` × `h` ≤ 2048, `stride` pixels a row) after clearing it to `clear` (0xRRGGBB) → 0, −1 no GPU, −2 bad arguments / too many vertices (`KAPI_GPU_MAX_VERTS` = 196608), −3 the GPU did not finish (it is then left off). See §15.
GPU (v53)	`gpu_texture(handle, pixels, w, h, stride)`: a texture of `w` × `h` (≤ 2048) pixels 0xAARRGGBB; `handle` < 0 makes one (≤ 256 in all; v70: ≤ 1024 in all, ≤ 512 a program), ≥ 0 replaces its pixels, `pixels` = 0 frees it → the handle, −1 no GPU, −2 bad arguments, −4 no memory / no free handle; a program's textures are freed when it ends. `gpu_render(f, v, nv, b, nb)`: one frame into `struct kapi_gpu_frame { pixels, w, h, stride, clear, flags }` (`KAPI_GPU_F_KEEP`: drawn over the pixels instead of clearing them) of `nv` vertices `struct kapi_gpu_vertex3 { float x, y, z, w, s, t; u8 r, g, b, a; u8 r2, g2, b2, a2; }` (v54: the second colour is added after the texel × colour product and clamped to 1 — 0 for the v53 behaviour) in `nb` (≤ 4096) batches `struct kapi_gpu_batch { first, count, texture, flags, float matrix[16]; }`: each batch draws its triangles with its own matrix (row by row, clip = M·(x y z w), transformed by the GPU, which divides by w and clips; `KAPI_GPU_B_NOMATRIX` = identity), texture (−1: colour only, else texel × colour; `B_LINEAR`, `B_WRAP_S/T(REPEAT, CLAMP, MIRROR)`), depth test (`B_ZFUNC`: less by default, … always) and writes

Category	Examples
	(`B_NOZWRITE`), culling (`B_CULL_BACK/FRONT`, front = counter-clockwise, y up), alpha test (v54: `B_ALPHATEST(t)`, fragments whose alpha < t / 255 are discarded) and blending (`B_BLEND(ALPHA, ADD, MUL, PREMUL)`; v72: also the premultiplied compositing presets `MULCOL` (colour s·d + d(1 − sa), alpha kept), `UNDER` (s(1 − da) + d, alpha over), `SCREEN` (s + d(1 − s)), `PLUS` (s + d), `RSUB` (d − s, alpha kept), `LIGHTEN` (max), `DSTIN` (d·sa, colour and alpha: a mask), `DSTOUT` (d(1 − sa): cut out) — each a pair of V3D blend equations, colour and alpha apart) → 0, −1, −2, −3 as `gpu_draw`. The kernel gives the V3D only well-formed triangles: each one is transformed by its batch's matrix on the CPU and clipped against the near plane (z ≥ −w, w > 0) and a guard band 4 × the screen (`kern/v3d_clip.h`, the batches then NOMATRIX; test `tools/tests/run_v3d_clip_test.sh`) — Ocarina of Time's triangles crossing the eye plane, projected ~50 000 screens away, wedged the GPU and froze the whole Pi. See §15.
GPU (v61)	`gpu_program(handle, p)`: the app's own shaders — `struct kapi_gpu_program { vs, cs, fs; nvs, ncs, nfs; inputs, csInputs, csOutputs, varyings, flags }`, three V3D 4.2 QPU programs (≤ 4096 instructions each; built at run time by `user/Libs/v3d/qpu.h`) copied into GPU memory; a vertex is `inputs` floats (4..64, the clip-space x y z w first; the coordinate shader reads the first `csInputs`, writes `csOutputs` ≥ 6: Xc Yc Zc Wc Xs Ys), the vertex shader writes Xs Ys Zs 1/Wc then `varyings` (≤ 64); flags `KAPI_GPU_P_FS_4WAY` (4 threads, else 2), `P_FS_FINAL` (no thread switch — unreliable on wide targets, see §15), `P_FS_ZWRITE`, `P_V71` (the shaders are **V3D 7.1**'s — the Pi 5: its kernel refuses a program without it, −2, and runs an app's fragment shader 2-way whatever

Category	Examples
	`P_FS_4WAY` says; `qpu::programFlags`, §15.1); `handle` < 0 makes one (≤ 256), ≥ 0 replaces it, `p` = 0 frees it → the handle, −1, −2, −4 as `gpu_texture`; freed when the program ends. `gpu_render2(f, v, nv, stride, b, nb, uni, nuni)`: one frame (as `gpu_render`) of `nv` vertices of `stride` floats in `nb` batches `struct kapi_gpu_batch2 { first, count, program, flags, blend, wmask, scissor[4], vsUni, vsNUni, csUni, csNUni, fsUni, fsNUni, tex[8], texFlags[8], texUni[8] }`: each with its program, its uniforms (three ranges of `uni[nuni]`, ≤ 2^20 words), up to 8 textures — the kernel writes each one's TMU words p0 (texture state, 16-bit float RG / BA) and p1 (a sampler from `texFlags`: `B_LINEAR`, `B_WRAP_S/T`) at `fsUni + texUni[i]` —, depth / cull flags (`B_ZFUNC`, `B_NOZWRITE`, `B_CULL_*`), blending `KAPI_GPU_BLEND2(cSrc, cDst, aSrc, aDst, cEq, aEq)` (the V3D factors and equations), a colour write mask (`wmask`: the channels not written) and a scissor (x y w h in the target, top-left origin; w ≤ 0: none) → 0, −1, −2, −3. The triangles are clipped on the CPU as for `gpu_render` (`V3DClipTriangleN`: every float of the vertex interpolated). See §15. **v98**: `KAPI_GPU_P_VS_1WAY` — the vertex and coordinate shaders are generated ones (`user/Libs/v3d/picavs`): run one thread a QPU (the whole register file) with a VPM output block apart from the input's (an output written before the last input is read).
GPU (v62)	`gpu_render3(f, v, nfloats, b, nb, uni, nuni, view)`: `gpu_render2` with each batch's vertices where the app keeps them — `struct kapi_gpu_batch3 { struct kapi_gpu_batch2 b; unsigned off, stride; }`: `b.count` vertices of `stride` floats (≥ the program's inputs, ≤ 64) from float `off` of `v[nfloats]` (`b.first` unused) — and

Category	Examples
	their x / y framed by the kernel on the way, `x' = view[0] x + view[1] w`, `y' = view[2] y + view[3] w` (`view` 0: as they are): no common array for the app to make first → as `gpu_render2`. gcemu's TEV renderer gives it its recorder's frame as it is. See §15. A batch flagged **`KAPI_GPU_B_NOCLIP`** (2026-10-11, no new call) carries attributes for its program's OWN vertex and coordinate shaders, not clip positions: the kernel neither frames nor clips its vertices on the processor -- the V3D clips from the coordinate shader's clip position (checked on the Pi 4: the same picture as the kernel's clipping for triangles beyond the sides, through the eye's plane and through the near and far planes -- `tools/tests/v3d/vsclip_pi.cpp`). **v98**: `KAPI_GPU_B_INDEXED` with it — at float `off`, `b.first` vertices of `stride` floats then `b.count` 32-bit indices among them (triangles): the vertex shader runs once a vertex, not once a corner; refused in a `gpu_vbuf`.
GPU (v63)	`gpu_vbuf(bytes)`: memory the GPU reads too — low, physically contiguous, mapped into the program (the surface arena); up to 8 a program (v70: 32 in all; 8 in all before), 64 MB each, freed when it ends → its address, 0 none. `gpu_render3` with `v[nfloats]` inside one draws the vertices **where they are**: their x / y framed **in place** (the program draws the same vertices again with `view` 0), the triangles inside every plane drawn as runs of them, the others clipped into the buffer's end past `nfloats` (keep room there) — nothing copied.
GPU (v70)	`gpu_texture_rect(handle, x, y, w, h, pixels, stride)`: the rectangle `x, y, w × h` of texture `handle` (the caller's) replaced by `pixels` (0xAARRGGBB, `stride` pixels a row), the rest kept → 0,

Category	Examples
	−1 no GPU, −2 bad arguments (not the caller's handle, the rectangle not inside the texture). The texels are stored where the texture unit reads them for the texture's size (`StoreRect`, `kern/v3d_tiling.h`: a utile row — 4 texels, 16 contiguous bytes in every layout — per offset computed) and only the span of bytes written is cleaned from the CPU caches; it waits for a frame in flight, as `gpu_texture`. `KAPI_GPU_F_ALPHA` (a frame's flag, `gpu_render` / `2` / `3`): the target is 0xAARRGGBB premultiplied — its alpha loaded with `KAPI_GPU_F_KEEP` (not forced to 1), blended, written (the colour write mask no longer spares it in the direct mode), stored, and cleared to `clear`'s top byte; without it nothing changes (0x00RRGGBB: the direct mode leaves the top byte, the copy mode writes 0). Handles: `gpu_texture` gives 1024 in all (256 before), 512 at most a program (−4 past that), so an emulator's texture cache and the browser's layers fit side by side; `gpu_vbuf` 32 blocks in all, 8 a program (8 in all before). See §15 *Sharing the GPU* and *The compositing service*.
Windows (v64)	The modernised CDE desktop's windows. `win_minimise(id)`: window `id` (0: the caller's) minimised — not drawn, not hit, never active nor the keys' target — until `win_raise` / `raise_app` brings it back (`KAPI_WIN_MINIMISED` in `win_list`'s state) → 0 / −1. `win_geometry(out)`: `struct kapi_win_geom { x, y, w, h; cw, ch; ax, ay, aw, ah; state }` — the caller's whole window (frame included), its client size, the **work area** (the screen less the menu bar at the top and the topmost windows standing on the bottom edge: the dock) → 0 / −1. `resize_window2(w, h, &stride)`: as `resize_window`, but the canvas and the frame's copies **grow** past their first size

Category	Examples
	when needed (new memory at the same addresses; their pixels are lost: redraw, `get_chrome` again) → the canvas and its stride, 0 (no memory: the size kept). With it: the frame's metrics `KAPI_FRAME_TITLE_H` 28, `KAPI_FRAME_BORDER` 4, `KAPI_FRAME_RADIUS` 8 (the chrome copies' top byte: a transparency, heeded in the corner squares), the title buttons' places `KAPI_FRAME_BTN_W/H/Y/EDGE/STEP` (the window menu at the left; close, maximise, minimise from the right: `KAPI_FRAME_MENU/CLOSE/MAXIMISE/MINIMISE`); `GUI_EVENT_WINCTL` (18: the window menu, maximise — for the app); `WIN_FLAG_ALPHA` (32: a borderless window's pixels carry their transparency). See §10.2.
Screen size (v66)	`screen_set(w, h)`: the screen's resolution **now** (640 × 480 .. 2560 × 1600, `w` even; `ScreenResizeRequest`, kernel.cpp). The compositor does it between two frames (the display DMA is idle — each present waits for its end): `C2DGraphics::Resize` (Circle's: the frame buffer asked of the firmware again at that size, the drawing buffer made again), the old size back if the firmware refuses; then `CWindowManager::OnScreenResized`: the cursor and every window kept on the screen (moved in when past the right / bottom edge; a window parked at a negative place is left there), the app-written wallpaper dropped (the next `wallpaper_buffer` is a new buffer of the new size — the old one is left allocated: an app may still have it mapped), and **`GUI_EVENT_DISPLAY_RESIZE`** (19, `lValue = w << 16 | h`, `GUI_DISPLAY_W/H`) to every window's pointer handler. The caller waits for it → 0; −1 out of bounds; −2 not now (a full-screen app owns the display — its mapping of the frame

Category	Examples
	buffer would be stale —, the debug console, another change under way); −3 the firmware refused it (the old size kept). `screen_grab` refuses the old size (vncd / rdpd see it and take the new one). Not kept across a reboot: `cmdline.txt`'s `width=` / `height=` are (the Display applet writes both). See §10.2.
Workspaces (v65)	The virtual desktops. `desk(set, count)`: `set` ≥ 0 shows desk `set`, `count` > 0 sets how many there are (1 .. `KAPI_DESK_MAX` = 8; the dock keeps 1–6; the windows of the desks dropped go onto the last one); −1 / 0 keep them → the current desk `
Held keys (v48)	`key_held(key)` → 1 while the key is held **and** the caller's window has the keyboard (`KeyTargetLocked`), else 0 — for games, since key events only report presses. Keys: `KEY_UP/DOWN/LEFT/RIGHT`, `KEY_ENTER`, 27, `' '`, `'a'..'z'` (the **US position** of the key), `'0'..'9'`. The WM keeps two bitsets over the logical codes: the USB one, rebuilt from every raw report (`KeyRawStub` → `SetUsbHeld`, HID usage → key), and the injected one (`inject_key_held(key, down)`, from vncd's RFB key down / up); `KeyHeld` ORs them.
FM (v47)	`sound_instrument` — **retired on 2026-10-05** (the synthesizer left the kernel: AudioKit's `ak_fm_*`, docs/03 §5.7; the slot stays and answers −1). `struct kapi_fm_instrument` (2 operators, OPL2 style: `struct kapi_fm_op op[2]` = modulator / carrier, `feedback`, `connection`) and `SOUND_SQUARE` … `SOUND_FM` stay in `kern/kapi_abi.h`: they are the names AudioKit's FM synthesizer takes.
Memory primitives (v36)	`memset`, `memcpy`, `memmove` — since v74 **user-side** routines of the EL0 code page (`el0blob.S`, no system call; general registers only; they align the destination for the Device-memory framebuffer),

Category	Examples
	the kernel table's slots 0 (before: Circle's kernel implementations). `user/Kits/appkit/appkit.h` wraps them as weak **`kapi_memset`/`kapi_memcpy`/`kapi_memmove`** symbols, and the freestanding app Makefiles alias the C names onto them (`-Wl,--defsym,memset=kapi_memset`, …): GCC may emit these calls on its own (array/struct initialization, copies) even with `-ffreestanding`, and freestanding apps have no libc. Newlib programs keep newlib's own.
Crypto (v30)	`random` (fill a buffer with random bytes — a **software PRNG** (splitmix64) seeded from the high-resolution timer: the BCM2711's hardware RNG stalls the bus in this setup, `sys/kapi.cpp`; not cryptographically strong; for seeding — the TLS entropy source in `user/Libs/tls/onyx_tls.hpp` feeds mbedTLS's CTR_DRBG from it)

All the functions run **in the kernel (EL1), on the calling process's task**: the app's `svc` enters on that task's own kernel stack with the app's page table still active, so the kernel reads and writes the app's memory in place — **after checking every pointer** (`kern/uaccess.h`, §6). A call that waits (a file read in pieces, a socket, `wait`) yields there, on that kernel stack, while the compositor and the other apps keep running.

v75: memory

Work package WP-MEM (`docs/POSIX-PLAN.md` §3.1; `kernel/sys/vm.cpp`, `kern/vm.h`; the threads' two in `sys/thread.cpp`). The mechanism — lazy regions, pins, the deferred zap, the TLB rules, the OOM policy — is §4 *Demand paging*. Every entry returns ≥ 0 or −`KAPI_Exxx`. Sizes: 64 KB pages; addresses and lengths are rounded to them where noted.

Slot	Entry	Does → returns
163	`vm_map (addr, len, prot, flags)`	A zero-filled `ANON` region of `len` (rounded up) in the mmap arena `[34 GB, 60 GB)`, filled on first touch (`KAPI_MAP_POPULATE`: now, a yield every 64 pages, a failure not reported). `addr` is a hint (taken if free, else the lowest gap) unless `KAPI_MAP_FIXED` (aligned,

Slot	Entry	Does → returns
		inside the arena; replaces what is there) or `KAPI_MAP_FIXED_NOREPLACE` (`-EEXIST` if anything is there). `prot` = `KAPI_PROT_NONE/READ/WRITE`, v78 `EXEC` (executable at EL0: a JIT; before v78: `-ENOTSUP`). → the address / `-EINVAL` (len 0, a bad `FIXED`), `-ENOMEM` (no room, 4096 regions, or a writable map larger than the free app pool − 16 MB without `KAPI_MAP_NORESERVE`)
164	`vm_unmap (addr, len)`	Inside the arena (holes allowed; regions split): the pages dropped (a pinned one when its kapi ends) → 0 / `-EINVAL` (unaligned, outside the arena) / `-ENOMEM` (a split at 4096 regions)
165	`vm_protect (addr, len, prot)`	`ANON` regions covering the range without a hole: their protection (split / merged), the present pages re-protected + TLBI (a write taken from a page under another thread's kapi: when it ends) → 0 / `-EINVAL` / `-ENOMEM` (split cap) / `-ENOTSUP` (`EXEC` over an SHM region; before v78: any `EXEC`)
166	`vm_advise (addr, len, advice)`	Lazy regions (`ANON`, `HEAP`, `STACK`) covering the range: `KAPI_MADV_WILLNEED` fills (`-ENOMEM`); `DONTNEED` / `FREE` drop the pages — zeros on the next touch (`ANON` and `HEAP` only); `NORMAL` / `RANDOM` / `SEQUENTIAL`

Slot	Entry	Does → returns
		nothing → 0 / `-EINVAL`
167	`vm_query (addr, out)`	`struct kapi_vm_region { start, end, prot, kind (KAPI_VMK_ANON/HEAP/STACK/IMAGE/FIXED), resident (pages present), flags (KAPI_VMF_LAZY) }` of the region holding `addr` → 0, or of the next one above → 1; `-ENOMEM` none above, `-EFAULT`
168	`vm_stats (pid, out)`	`struct kapi_vm_stats { resident (bytes of owned frames, page tables included), lazy (VA of lazy regions), writable (VA of writable regions), faults (pages filled), pt_bytes, limit (0) }`, `pid` 0 = self → 0 / `-ESRCH` / `-EFAULT`
169	`thread_create_ex (attr)`	`struct kapi_thread_attr { fn, arg, stack_size (0 = 8 MB; 16 KB .. 16 MB, lazy), tls (its initial TPIDR_EL0), name, flags (KAPI_THREAD_DETACHED: no join, its record freed when it ends), prio (0, 1 = "real time"), reserved[2] = 0 }` → tid ≥ 2 / `-EAGAIN` (32 running) / `-ENOMEM` / `-EINVAL` / `-EFAULT`
170	`thread_info (tid, out)`	`struct kapi_thread_info { stack_lo, stack_hi (its top: the initial SP), tid, state (0 running, 1 ended, joinable), guard (unmapped bytes below stack_lo) }`, `tid` 0 = self, 1 = main → 0 / `-ESRCH` / `-EFAULT`

What changed for every app, with no call: the main stack (8 MB by default), the threads' stacks and the heap are **lazy** (a process's resident memory drops by its untouched stack — 1 MB to 8 MB before — and heap); `sbrk` shrinking returns pages; an overflow of a stack is a clean kill with "stack overflow" in kmsg; a

page fault the app pool cannot serve kills the app (`KAPI_PROC_OOM`, −9), not the system. The kernel's probes see the app's permissions (`AT S1E0*`). Test: `/bin/memtest` (docs/04).

v75: files and processes

Work package WP-FILE/PROC (`docs/POSIX-PLAN.md` §3.2): `kernel/sys/ofile.cpp` + `kern/ofile.h` (slots 207–221), `kernel/sys/procx.cpp` + `kern/procx.h` (222–228), the `RAM:` node calls in `sys/ramfs.cpp`, the pipes in `sys/stream.cpp`. Every call returns ≥ 0 or −`KAPI_Exxx`. Host test `sh tools/tests/run_ofile_test.sh` (the real `ofile.cpp` over the fork's FatFs on a RAM disk — `SD:` FAT32, `SD1:` exFAT — and the real `RAM:`; it ends by checking that every cluster came back); `run_ramfs_test.sh` covers the `RAM:` node calls too. Pi tests `/bin/filetest`, `/bin/proctest` (docs/04).

Open files (`HANDLE_OFILE`). `file_open (path, KAPI_O_*, mode)` → a handle (> 0). A FatFs file has **one node** (`TFNode`) whatever the number of its openers: one `FIL`, opened `FA_READ`, re-opened `FA_READ | FA_WRITE` when its first writer comes (FatFs keeps no share locks, `FF_FS_LOCK 0`: two `FIL`s writing one file would corrupt it). Its key is the absolute path (`ResolvePath`), compared case-insensitively. A handle is an **open-file description** (`TOFile`): the node, its own 64-bit offset, its access mode, `O_APPEND` (`dup` is the libc's: it counts its references). Each call holds the node's lock across its `f_lseek` + `f_read` / `f_write` (FatFs yields while the card works); the node table has a lock of its own (open, close, unlink, rename; the order is table → node). Both are sleeping locks, and each call is a **no-kill** section from its start to its end.

Call	Semantics
`file_open`	`O_RDONLY / WRONLY / RDWR` (3: `EINVAL`), `O_CREAT`, `O_EXCL` (`EEXIST`), `O_TRUNC` (writable only), `O_APPEND`; a folder or a volume root → `EISDIR`; `mode` ignored (FAT has a read-only bit only); a provider's path (`FTP:`) → `ENOTSUP`; a bad path pointer `EFAULT`, over 511 characters `ENAMETOOLONG`
`file_read (h, buf, len, off)`	`off = -1`: at the handle's offset (advanced), else a pread; ≤ 1 GB a call; 0 at the end; `EBADF` on a write-only handle. The buffer is checked for what will be filled (the rest of the file)
`file_write`	`off = -1`: at the offset, or at the end with `O_APPEND` (under the node's lock: two appenders never overlap); a write past the end fills the gap with **zeros** (FatFs leaves it undefined); `ENOSPC` when nothing fits, `EFBIG` past 4 GB − 1 on FAT32
`file_seek`	`SEEK_SET / CUR / END` → the new offset (no size change); < 0 → `EINVAL`
`file_truncate`	shrink (`f_truncate`), or grow with zeros (64 KB pieces); `EINVAL` on a read-only handle

Call	Semantics
`file_sync`	`f_sync` (the entry gets the size and the time)
`file_stat` / `path_stat`	`struct kapi_stat`: `size` (an open file's from its `FIL`), `mtime` (FAT's local time → UTC with `set_timezone`'s zone; an open file written since its last sync: the time of its last write), `ino` = FNV-1a 64 of the upper-cased absolute path (`RAM:`: the node's own number, kept by a rename), `mode` `S_IFREG 0644` / `S_IFDIR 0755` (`0444` / `0555` with the read-only bit), `dev` (1 `SD:`, 2–4 `SD1:`–`SD3:`, 5 `USB1:`, 6–9 `USB1P1:`–`USB1P4:`, 10 `USB2:`…, 15 `USB3:`…, 64 `RAM:`), `blksize` (the cluster; `RAM:` 64 KB), `blocks`, `attr` (FAT's), `ctime` = `mtime`; a volume root is a folder
`file_close`	the description freed; the node's last close flushes (`f_close`)
`path_unlink (p, flags)`	a file (`EISDIR` on a folder); `KAPI_UNLINK_DIR`: a folder (`ENOTDIR`, `ENOTEMPTY`); a root → `EBUSY`; a read-only file → `EACCES`
`path_mkdir`	`EEXIST`, `ENOENT` (no parent)
`path_rename`	replaces the target (a file by a file; an empty folder by a folder; `EISDIR` / `ENOTDIR` / `ENOTEMPTY`), `EXDEV` across volumes, a folder into itself `EINVAL`, a case change allowed
`path_utime (p, mtime)`	the entry's time (UTC → local); an open written file is synced first, so its close does not stamp it again
`dir_read (dir, out)`	the next entry of an `opendir` handle as `struct kapi_dirent2`: a 255-character name, 64-bit size, mtime, mode, attr, ino (= `path_stat`'s; `kapi_opendir` records each FatFs `DIR`'s path for it: `OFileNoteDir`) → 1, 0 at the end; the `.~onyx-deleted` folder is not listed
`stream_write_nb (h, buf, len)`	a pipe: what fits now, `EAGAIN` when full; a file stream: a plain write

Unlink and rename of open files. An open file that is unlinked (or that a rename replaces) is moved to `<volume>:/.~onyx-deleted/<n>` (a hidden folder) and deleted at its node's last close (the folder too once empty): it stays readable and writable through its handles, and its name is free at once. A file renamed (or

hidden) while open has its `FIL` closed, the entry moved, then re-opened at the new path — FatFs keeps the location of the file's directory entry in its `FIL` (`dir_sect`, exFAT's `c_scl` / `c_ofs`) and would write the size and the time into the old one. A folder renamed while files in it are open: their paths follow (their entries do not move). Leftovers of a crash are removed at boot (`OFileBootCleanup`, from `StartAutostart`: `SD:`–`SD3:`), on a volume mounted later the first time a file there is hidden. `RAM:` does the same with its own nodes (`RamFsNodeOpen`, `RamFsPRead` / `PWrite` / `Truncate`, `RamFsUnlink`, `RamFsRenameEx`..., §16). A teardown (the interrupts masked) only queues its open descriptions; the reaper closes them (`OFileRunDeferred`, from `HandlesRunDeferred`).

The old file calls (`open`, `read`, `save_file`, `file_in` / `file_out`...) keep their own `FIL`s: mixing them with `file_*` on one file at the same time is not coherent (a `save_file` while a `file_*` handle writes the same file: whichever flushes last wins). FatFs' fast seek (`CREATE_LINKMAP`, the old `seek`'s) is never used on a node: FatFs cannot grow a file in that mode.

Pipes. `CPipeStream::PollMask`: `POLLIN` when not empty, `POLLIN | POLLHUP` once the write end is closed, `POLLOUT` when there is room. A blocking `Read` / `Write` sleeps in `IoWait` (≤ 50 ms per turn) instead of yield-spinning, and every change — written, drained, `CloseWrite` — calls `IoWake`. `PIPE_CAP` stays 8 KB. A write to a pipe nobody reads still waits (no `EPIPE`). `IoWait` sleeps in no-kill slices of ≤ 100 ms: a task killed while it waits leaves the event's list before it ends (the reaper would otherwise free it while still listed).

Processes. Each process has a `TProcInfo` (`CAddressSpace::m_pProcInfo`): an **argv block** and an **environment block** (`"a\0b\0\0"`, ≤ 64 KB each, kernel heap), made in the spawner's context (`LaunchApp`, `ExecPath`, `SpawnProcess`) and installed by `CUserProcessTask::Run` once the space exists (the spawn record then learns the pid). The environment: `spawn_ex`'s, else the spawner's initial one (`spawn`, `exec`, `exec_as`); a desktop launch or a process the kernel starts (init) gets the **system default**, read at boot from **`SD:/etc/environment`** (`KEY=VALUE` lines, `#` comments; without it `HOME=SD:/home`, `PATH=SD:/bin`, `TMPDIR=RAM:/tmp`, `LANG=C.UTF-8`). argv[0] is the program's resolved path, then `get_args`' string split as a shell does (blanks, `"quotes"`). (An empty argument cannot be passed: it would end the block.)

Call	Semantics
`spawn_ex (attr)`	`struct kapi_spawn_attr`: `path` (resolved against the cwd), `argv` and `envp` blocks (0: the path alone / the caller's initial environment), `cwd` (0: the caller's), `in` / `out` stream handles → a process handle; `ENOENT` (no program), `EACCES` (a folder), `EBADF` (a stream), `EINVAL` (a block over 64 KB), `EMFILE`. Returns once the child has its pid (its first time slice, ≤ 1 s). The child's `get_args` string is argv[1..] joined with blanks (an argument with a blank in quotes)
`proc_wait (h, flags, out)`	sleeps (`IoWait`; the child's end wakes it) → 1 and `struct kapi_proc_status {code, reason, pid}`, the handle closed unless `KAPI_WAIT_KEEP`; `KAPI_WAIT_NOHANG` on a running child → 0 with `pid` set and `reason` −1; `EBADF`

Call	Semantics
`get_argv` / `get_env (buf, cap)`	the block, filled up to `cap` → its whole size (call with 0 to size it)
`getpid (which)`	0 the pid, 1 the parent's (0: none); else `EINVAL`
`clock_info (out)`	`struct kapi_clock_info`: `cnt` (CNTPCT) and `utc_us` sampled together (the tick's time: ± 10 ms), `freq`, `tz_minutes`, `flags` `KAPI_CLOCK_REALTIME_VALID` (a real date: NTP / RTC), `boot_cnt` (CNTPCT at boot): a libc reads CNTPCT at EL0 and needs no system call per `clock_gettime`
`sleep_us (us)`	under 1 ms: a `Yield` loop on the µs clock; else `usSleep` (the scheduler's 10 ms tick when idle)

How a process ended (`proc_wait`'s `reason`; `kapi_wait`'s status is the code): `KAPI_PROC_EXITED` (its `exit` code), `KAPI_PROC_FAULT` (−11: `Fault`, `sys/el0.cpp`), `KAPI_PROC_KILLED` (−9: `kill`, `kill_pid` with force or on a windowless app, a dead parent's cascade — before v75 a killed process reported 0), `KAPI_PROC_OOM` (WP-MEM). `ProcInfoTeardown` copies the reason and the pid into the spawn record (`CProcess::nReason`, `nPid`) before it is marked done, and wakes the waiters.

v75: sockets and poll

Work package WP-NET (`docs/POSIX-PLAN.md` §3.3; `kernel/sys/bsdsock.cpp`: the kapis and `poll`; `kernel/sys/net.cpp`: the socket table and its "slot layer"; `kern/net.h`). Slots 229–241 (`struct kapi_sockaddr`: IPv4, the port in host order; `struct kapi_pollfd`). Every call returns ≥ 0 or −`KAPI_Exxx`.

Slot	Entry	What it does
193	`sock_open (type, flags)`	`KAPI_SOCK_STREAM` (TCP) / `KAPI_SOCK_DGRAM` (UDP), `KAPI_SOCKF_NONBLOCK` → the socket number; `EPROTONOSUPPORT`, `EINVAL` (flags), `ENETDOWN` (no network yet), `ENFILE` (table full)
194	`sock_connect (s, to)`	TCP: blocking → 0 or the error (`ECONNREFUSED`, `ETIMEDOUT` after Circle's retries, about a minute, `EHOSTUNREACH`, `ENETUNREACH`); non-blocking → `EINPROGRESS`, then `poll (POLLOUT)` and `SO_ERROR`; `EALREADY`, `EISCONN`. UDP: sets the

Slot	Entry	What it does
		default peer (only its datagrams are received)
195	`sock_bind (s, addr)`	the address 0.0.0.0 or the Pi's own (`EADDRNOTAVAIL`); port 0: an ephemeral port (TCP 61000–61999, UDP Circle's 60000–60999); `EADDRINUSE` (a bound / listening socket of the same protocol has it), `EINVAL` (bound already)
196	`sock_listen (s, backlog)`	backlog clamped 1..32; an unbound socket gets an ephemeral port
197	`sock_accept (s, peer, flags)`	a connection waiting → a new socket (`KAPI_SOCKF_NONBLOCK`: non-blocking); none → `EAGAIN` (a non-blocking listener) or a wait (`SO_RCVTIMEO`); `ECONNABORTED` (the peer left before it was accepted), `ENFILE`
198	`sock_send (s, buf, len, flags, to)`	→ the bytes queued. TCP: all of them when blocking (a wait while Circle's queue holds 64 KB, `SO_SNDTIMEO` → a short count / `EAGAIN`), what fits when non-blocking (`MSG_DONTWAIT`); `EPIPE` (reset, `SHUT_WR`), `ENOTCONN`. UDP: one datagram to `to` or the default peer (`EDESTADDRREQ`), at most 1472 bytes (`EMSGSIZE`); `to` is ignored on TCP
199	`sock_recv (s, buf, len, flags, from)`	→ the bytes, 0 = the peer's orderly end (or `SHUT_RD`); `EAGAIN` (non-blocking, `MSG_DONTWAIT`, `SO_RCVTIMEO`), `ECONNRESET`, `ETIMEDOUT`, `ENOTCONN`. `MSG_PEEK` (leaves the

Slot	Entry	What it does
		bytes), `MSG_WAITALL` (TCP: until `len`, the end or an error). UDP: one datagram, cut to `len` (the rest dropped); `from` = its sender (TCP: the peer)
200	`sock_shutdown (s, how)`	`SHUT_RD`: `recv` answers 0, `poll` says `POLLIN`; `SHUT_WR`: `send` answers `EPIPE` — **no FIN is sent** (Circle's TCP cannot receive after its own FIN; the connection ends at `close`); `ENOTCONN`
201	`sock_close (s)`	0 / `EBADF`. A socket still connecting or inside a send is closed by its last user
202	`sock_getopt (s, opt, &v)`	`SO_ERROR` (the pending error, a positive errno, cleared), `SO_NONBLOCK`, `SO_RCVTIMEO_MS`, `SO_SNDTIMEO_MS`, `SO_BROADCAST`, `SO_NREAD` (bytes in the carry buffer, 1 if more is ready), `SO_TYPE`, `SO_ACCEPTCONN`; else `ENOPROTOOPT`
203	`sock_setopt (s, opt, v)`	`SO_NONBLOCK`, `SO_RCVTIMEO_MS` / `SO_SNDTIMEO_MS` (0: none), `SO_BROADCAST` (UDP); else `ENOPROTOOPT` (libc accepts and ignores `TCP_NODELAY`, `SO_KEEPALIVE`, `SO_REUSEADDR`, the buffer sizes)
204	`sock_name (s, peer, out)`	`peer` 0: the local address (the Pi's IP once connected, else 0.0.0.0) and port; 1: the peer (`ENOTCONN`)
205	`poll (fds, n, timeout_ms)`	`n` ≤ 1024 (`EINVAL`), `timeout_ms` −1 forever, 0 only a look → the number of entries with `revents` ≠

Slot	Entry	What it does
		0. Kinds: `KAPI_PK_SOCKET` (a socket number), `KAPI_PK_STREAM` (a stream handle: `CStream::PollMask`), `KAPI_PK_FILE` (a file / directory / open-file handle: always IN | OUT); a bad one `POLLNVAL`; kind 0 or `h < 0` ignored. `POLLERR`, `POLLHUP`, `POLLNVAL` are reported whatever `events` asks

One table. The BSD sockets live in the same table as the `tcp_*` handles (`sys/net.cpp`, `MAX_SOCKETS` **256** since v75, shared by every process; owner pid, adoption by a descendant, `NetCloseByPid`, as in §11). A slot has a type (TCP / UDP), a state (NEW, BOUND, LISTEN, CONNECTING, CONNECTED, FAILED), the pending error, the options, the local port and the peer, a **carry buffer** (a frame, allocated at the first receive that needs it: a segment bigger than the caller's buffer leaves its rest there — so nothing is lost, and `MSG_PEEK` works; `tcp_recv` goes through it too, which fixes its data loss with small buffers when `netcore=0`) and a readiness snapshot. The `tcp_*` handles are sockets of that table: `poll`, `sock_name`, `sock_recv` work on them; a socket number is not a handle (`HANDLE_*`), and it is the same number space for every process (another process's socket: `EBADF`, `POLLNVAL`).

The slot layer never waits. `net.cpp`'s `Slot*` functions run where the stack runs (core 0, or a worker of core 3 with `netcore=1`, through the request slots of §11: a send or receive moves at most 32 KB per round trip) and answer `EAGAIN` instead of blocking. Every wait is `bsdsock.cpp`'s: a few yields, then `IoWait` on the I/O generation (`kern/iowait.h`), then the call again. TCP data goes into Circle with `MSG_DONTWAIT` only while Circle's send queue is under its 64 KB threshold (`GetStatus ().bTxReady`). Receiving: `CSocket::Receive (MSG_DONTWAIT)` straight into the buffer while a whole frame fits, else through the carry buffer; an empty receive with the connection still up (CLOSE-WAIT: the peer's FIN) is the orderly end (0), else the connection's error.

Connect. A TCP connect is run by a task of its own, so that a blocking connect too only waits on `IoWait` (an app killed meanwhile leaves no task inside Circle): with `netcore=0` a one-shot kernel task (`CNetConnectTask`, `netconn`) on core 0; with `netcore=1` a **detached** request (nobody waits on it; the worker frees it; `NetCloseByPid` leaves it alone). It ends with the slot CONNECTED, or FAILED with its error (`SO_ERROR`); a close meanwhile only marks the slot (`bCancel`) and the connector frees it. The same marking protects a send in progress (Circle's `Send` yields between segments) and a `tcp_accept` waiting in Circle's `Accept`.

Readiness. `POLLIN`: data or carry, the peer's end, an error, `SHUT_RD`, a connection to accept (`CSocket::AcceptReady`, Circle patch); `POLLOUT`: connected and under the send threshold, always for UDP; `POLLERR`: a connect that failed (with `POLLOUT | POLLHUP`, as Linux); `POLLHUP`: was connected and no longer is (reset, timeout), or a TCP socket never connected. With **`netcore=1`** the net core's main loop recomputes every open slot's snapshot at each turn (and a worker after each call on a slot) and bumps a generation word when one changed, then sends core 0 an **inter-core interrupt** (`IPI_NET_READY`, one at a time: a flag core 0 clears before it reads the generation); core 0's handler (`NetReadyIPI` → `NetPollTick`) turns that

into `IoWake` at once. The 100 Hz tick hook (`IoWaitAddTickHook`) remains as the fallback; a waiter's sleep is capped at 100 ms anyway. (Before the IPI the tick alone did it: every blocking `recv` / `send` / `poll` waited up to 10 ms a turn — an echo's round trip was 10 ms on the LAN.) With **`netcore=0`** the readiness is evaluated on the spot and nothing announces a change (a connect's end and a close do call `IoWake`), so a wait over sockets looks again at every tick (10 ms). A wait over streams and files only sleeps until `IoWake` (a pipe's `PollMask` and its wakes are WP-FILE/PROC's).

Circle patch (`tools/circle-patches/wp-net.patch`, docs/05): a connection's handle stays its socket's until the socket lets it go (`CNetConnection::SetReleased`: before, a reset connection was deleted at the next `Process` and its handle reused by the next connection — which the old socket then read, wrote and closed); `CSocket::AcceptReady ()` (a backlog connection is connected: `Accept` will not block; it also replaces backlog connections that died before being accepted, a SYN without its ACK); `CSocket::Accept` lets a failed connection go instead of leaving it listening on its own; `CTransportLayer::IsTerminated (h)`.

Test: `/bin/nettest` (`user/BinUtils/nettest.c`; the PC side `tools/tests/nettest_peer.py`): run it with `netcore=0` and with `netcore=1`.

v76: IPC

Work package WP-IPC (`docs/POSIX-PLAN.md` §14: what WebKit2's Unix IPC uses, the spec): what a browser's UI, web and network processes need to talk. `kernel/sys/lsock.cpp` (local sockets, the handles they carry; `kern/lsock.h`), `kernel/sys/shm.cpp` (shared memory objects), `kernel/sys/vm.cpp` (`shm_map`, SHM regions), `kernel/sys/procx.cpp` (`spawn_ex2`, `get_handles`), `bsdsock.cpp`'s dispatch. Slots 242–252 (`struct kapi_iovec`, `struct kapi_handle_xfer` 24 bytes, `struct kapi_msghdr` 48 bytes). Every call returns ≥ 0 or −`KAPI_Exxx`.

Slot	Entry	What it does
206	`sock_pair (type, flags, sv)`	two connected local sockets: `KAPI_SOCK_STREAM`, `KAPI_SOCK_SEQPACKET` (5), `KAPI_SOCK_DGRAM`; `KAPI_SOCKF_NONBLOCK` → `sv[0]`, `sv[1]` (numbers ≥ `KAPI_SOCK_LOCAL_BASE`); `EPROTONOSUPPORT`, `EMFILE`, `EFAULT`
207	`sock_sendmsg (s, m, flags)`	the `m->iovcnt` (≤ 64) iovecs gathered into one message with `m->nhandles` (≤ 256) handles → the bytes; `EAGAIN`, `EPIPE`, `EMSGSIZE`, `ENOBUFS`, `EBADF` (a handle: nothing sent), `EINVAL`; an IP socket: `EOPNOTSUPP`
208	`sock_recvmsg (s, m, flags)`	into the iovecs; the handles carried added to

Slot	Entry	What it does
		the caller's table and written to `m->handles` (`m->nhandles` in: room, out: count; `m->flags`: `KAPI_MSG_TRUNC`, `KAPI_MSG_CTRUNC`) → the bytes, 0 the end
209	`shm_create (size, flags)`	an anonymous object (`KAPI_SHM_ALLOW_SEALING`, else `KAPI_SEAL_SEAL` set) → a read-write handle
210	`shm_open (name, oflags, mode)`	a named object (`/x`, ≤ 63 characters, a kernel-wide table): `KAPI_O_RDONLY` / `RDWR`, `CREAT`, `EXCL`, `TRUNC` → a handle; `ENOENT`, `EEXIST`, `EINVAL`, `ENAMETOOLONG`
211	`shm_unlink (name)`	the name dropped (the object lives while referenced); `ENOENT`
212	`shm_ctl (h, op, arg)`	`KAPI_SHM_GET_SIZE`, `SET_SIZE` (ftruncate: `EPERM` sealed, `EBUSY` a shrink while mapped, `ENOMEM`), `ADD_SEALS` / `GET_SEALS` (as memfd's `F_SEAL_*`), `GET_ID`, `GET_ACCESS`
213	`shm_map (h, addr, len, prot, flags, off)`	the object mapped `MAP_SHARED` (an SHM region in the mmap arena, placed as `vm_map`) → the address; `EACCES` (write, read-only handle), `EPERM` (`SEAL_WRITE`), `EBUSY` (a fixed place under another task's kapi)
214	`handle_close (h)`	a shm, local socket, open-file or stream handle closed
215	`spawn_ex2 (attr, handles, n)`	`spawn_ex` plus `n` (≤ 256) handles referenced for the child at the descriptors `fd` given

Slot	Entry	What it does
216	`get_handles (out, cap)`	the child's side: those handles, put in its table at the first call → how many

Local sockets. An end (`TLsEnd`) is a core-0 object: its type, a receive queue of messages (`TLsMsg`: the data and the handles carried, one allocation), its peer, the shutdown bits, `bNonBlock`, the timeouts, `SO_RCVBUF` (the queue's limit, 256 KB by default) and `SO_SNDBUF` (the largest packet, 256 KB), both 4 KB–16 MB. **A local socket's number is a handle** (`HANDLE_LSOCK`) of the caller's table — so it is per process, reference-counted and closed by the teardown, and its value (always ≥ 0x10001) tells it from an IP socket (0..255, the global table of §11). `bsdsock.cpp` sends every call on such a number here first (`IpcLocal*`): `sock_send` / `recv` / `shutdown` / `close` / `getopt` / `setopt` / `name` and `poll` (`connect` / `bind` / `listen` / `accept`: `EOPNOTSUPP`: no named local sockets — WebKit only uses `socketpair`). A send copies into a new message on the **peer's** queue: STREAM takes what fits (a blocking send waits for the rest), a packet (SEQPACKET, DGRAM) is whole or refused (`EMSGSIZE` above `SO_SNDBUF`; an empty queue always takes one); a receive copies out of its own queue — STREAM across messages but never past one that carried handles (they come with its first byte, as Linux), a packet whole (cut: `MSG_TRUNC`). Every change calls `IoWake`, so a wait (`IoWait`) or a `poll` over local sockets sleeps until it (no 10 ms polling as for IP sockets). The end: a closed peer (its last handle and every message holding it gone) or its `SHUT_WR` → 0 once the queue is empty; a send to it → `EPIPE`. Readiness: `POLLIN` data or the end, `POLLOUT` room in the peer's queue or a send that would fail at once, `POLLHUP` the peer gone. All the queues together hold at most 64 MB (`ENOBUFS`). A call pins its handle (another thread's close waits for the call to end, as for files).

Carried handles (`TIpcXfer`). `sendmsg` turns each `{h, kind, tag, flags}` into a reference at once (`IpcXferTake`): an open-file description (`HANDLE_OFILE`: ofile.cpp's `nHolders`, so the offset is shared as by a `dup`), a stream (`CStream::AddRef`), a local socket end, a shm object; an **IP socket** keeps its number and the sender's pid and is **adopted** by the receiver at `recvmsg` (`NetSocketAdopt`: it becomes the receiver's — an IP socket has one owner; one the sender closed meanwhile arrives as `KAPI_HK_NONE`, −1). `recvmsg` adds each to the caller's table (`IpcXferGive`); no room in the array or the table: closed, `MSG_CTRUNC`. A message discarded (its end closed, its process gone) closes what it carried (the teardown's way: deferred where a close may wait). Sending the receiving end over its own connection is refused (`EINVAL`: no close could ever free it); longer reference cycles are not collected. The `tag` is the sender's word, given back as it is: libonyxposix puts its descriptor type and `O_*` flags there. `KAPI_HXF_WRITER` on a stream (a pipe's write end): the pipe counts one more writer (`CPipeStream::AddWriter`), and its end-of-file comes when every writer has called `stream_eof` or closed (the receiver's entry is `HKIND_STREAM_WRITER`: its close or its process's end counts as its `stream_eof`, §9).

Shared memory. A `TShm` is a size and an array of 64 KB frames, each taken from the app pool (`palloc_high`, refused under `VM_RESERVE`) and zeroed at its first use, all freed — their word waiters woken first — when the last reference goes: the handles (`HANDLE_SHM`, the entry's kind = the access), the messages and spawn records carrying it, the name table, and **each address space mapping it** (one reference per space, `TVmSpace::pShm`). `shm_map` makes a lazy **SHM region** (`KAPI_VMK_SHM`, `TVma::pObj` + `ulObjOff`; a split keeps each piece's offset; neighbours merge only over contiguous offsets of the same object). A fault there (EL0, a kapi's probe, the safety net, an app core's pager) maps the object's frame with the region's protection **not owned** (`VM_PTE_SW_OWNED` clear): `Release` (unmap, `DONTNEED`), the deferred zap and the teardown never free it. A page beyond the object's size faults (the process is killed: "beyond the shared object", as Linux's `SIGBUS`). The space keeps its reference until no region names the object **and** no deferred zap is pending (a PTE marked ZAP may still point at a frame), checked after each

unmap / fixed map / settled zap (`VmShmGc`), dropped by `VmTeardown`. A shrink while any space maps the object is refused (`EBUSY`), so a mapped frame never goes. `vm_protect` / `vm_advise` / `vm_query` work on SHM regions (`DONTNEED` drops the mapping, the data stays). The futex (`wait_word` / `wake_word`) keys on the physical word: two processes meet on a shm word (WebKit's `IPC::Semaphore`).

Spawning with handles. `spawn_ex2` references the handles (as `sendmsg`, but an IP socket is not adopted: the child, a descendant, adopts it at its first use as before) in the child's `TProcInfo`; the child's `get_handles` puts them in its table at the first call (the same list at every call); those never asked for are closed with the process. CLOEXEC is user space: libonyxposix's `posix_spawn` gives every descriptor without `FD_CLOEXEC` (docs/03 §5.4).

Tests: `/bin/ipctest` (`user/BinUtils/ipctest.c`, kapi level), `posixtest ipc` (the POSIX calls); on the PC `tools/tests/run_ipc_test.sh` (the real `lsock.cpp`, `shm.cpp`, `handle.cpp` with two handle tables, ASan) and the posixsim bench (its fake table implements v76 over Linux socketpairs, `SCM_RIGHTS` and memfd).

v77: program images

The loader's image objects (§7 *Program images*: what they are, their key, their lifetime, the hook). `kernel/proc/image.cpp`, the three entries in `kernel/sys/kapi.cpp`, the preload task in `kernel.cpp`. Slots 253–255; `struct kapi_image_info` (280 bytes: `size` — the bytes of memory the image holds, once whatever the processes —, `file_size`, `refs`, `flags`, `path[256]` — its key). A path is a program file's, relative to the caller's working directory; every call returns ≥ 0 or −`KAPI_Exxx`.

Slot	Entry	What it does
217	`image_preload (path)`	the program loaded ahead and **kept**: a kernel task reads the file, the call returns at once; from then on a run of that path maps the image without reading the card, and the image stays when no process runs it. Kept already: 0, nothing done → 0; `ENOENT` (no such file), `ENAMETOOLONG`, `ENOMEM`, `EFAULT`. A load that fails later is in the kernel log
218	`image_unload (path)`	the path's image loses its pin and its name at once: no new process maps it; its memory is freed when the last process running it ends → 0; `ENOENT` (no image), `EBUSY` (v97: the path is the graphics server's alias), `EFAULT`
219	`image_list (path, out, cap)`	`path` 0: the live images,

Slot	Entry	What it does
		up to `cap` written → how many there are. `path`: the image a run of that path would map → 1 (`out[0]` written if `cap` > 0) / 0. Flags: `KAPI_IMG_KEPT` (preloaded), `KAPI_IMG_LOADING`, `KAPI_IMG_UNNAMED` (unloaded, or its file changed: only its processes still use it), `KAPI_IMG_LIB` (v83), `KAPI_IMG_ALIAS` (v97: also found under an alias, which follows `path`'s NUL)

Users: `/bin/preload` (`preload /boot`, the last line of `/etc/autostart`: the list of `SD:/etc/preload.ini`, `user/Include/preloadini.h`, edited by the Control Panel's Preload applet, `user/Apps/preloadconf`), `/bin/unload` (docs/04 §8), `pkg` (`pkglib.h` `move`: a kept program unloaded before its file is replaced, preloaded again after). `user/Kits/appkit/appkit.h`'s wrappers return `-KAPI_ENOSYS` on an older kernel. Tests: `sh tools/tests/run_image_test.sh` (§7).

v78: PROT_EXEC

A JIT's memory, as on Unix: `vm_map (addr, len, READ | WRITE | EXEC, flags)` gives a **lazy** anonymous region whose pages are mapped `UXN = 0` (executable at EL0; `PXN` stays 1: the kernel never runs an app's code), and `vm_protect` sets or takes away `EXEC` on anonymous regions — so a JIT may keep its code `RWX` (JavaScriptCore's default on Linux) or switch `RW` / `RX` (W^X). A shared object's pages never run: `shm_map` with `EXEC`, or `vm_protect` adding it over an SHM region → `-KAPI_ENOTSUP`. `kernel/sys/vm.cpp`: a region's protection becomes its PTE's AP **and** UXN bits (`PteProt`) wherever pages are filled, adopted, settled or reprotected. An instruction abort in a region not filled yet is paged in where the region is executable (`el0.cpp` `PageFault`, `VmProtAt`; an app core asks core 0 as for a data abort, the retried fetch then faults if the region does not allow it). The program writes its code, then makes the caches coherent over it from EL0 (`DC CVAU`, `DSB ISH`, `IC IVAU`, `DSB ISH`, `ISB`: SCTLR_EL1.UCI / UCT, `el0.cpp`; libgcc's `__builtin___clear_cache` does it). `code_alloc` (v58: an eager region in the code arena) stays for the GameCube emulator. Before v78 `EXEC` was `-KAPI_ENOTSUP` everywhere; libonyxposix's `mmap` / `mprotect` pass the kernel's answer on. Tests: `memtest` (code written, run, made `RX`, rewritten), `posixtest` (`mmap` `PROT_EXEC`).

v97: lib_open_as, the graphics server per mode

Slot	Entry	What it does
234	`lib_open_as (path, alias, min_version, err)`	`path` (`lib_open`'s forms: a bare name or a path) opened and mapped into the caller as `lib_open` does — the version

Slot	Entry	What it does
		checked on it, the real library —, then found under `alias` too: every `lib_open` of `alias` gets this image, before a library whose real path is `alias` → its export table, or 0 with `*err` = `-EPERM` (the caller does not hold the graphics server's role; `alias` or `path` not under `SD:/lib/`; either is `SD:/lib/appkit.so`), `-EBUSY` (`alias` held by another live process, or by the caller on another library; this library has another alias), `-EINVAL` (`path` = `alias`; not a library), `lib_open`'s errors. Asked again by its owner: the same table (the same image, even if its file changed). Lifetime: §7 *Aliases*.

`ws_ctl` (slot 227) gains: **`KAPI_WS_REGISTER`** gives the role to the program the kernel started for the mode (by its task's name: `elegant` or `pocketui`), when no live process holds it — `EPERM` for any other; **`KAPI_WS_SWITCH` (19)**, anyone's but the server's: the server is sent **`KAPI_WS_IN_QUIT` (10)** (Elegant gives the display back and ends; a server that does not is killed after 3 s), every alias dropped, `shell =` read again, its server started and waited for (5 s), Elegant if it fails → 0 (the one asked for has the display), 1 (Elegant instead: the desktop), `-EBUSY` (a full-screen program has the display, or a start / switch is under way), `-EINVAL` (called by the server), `-EIO` (no server took the display: the console). The caller sleeps meanwhile (up to about 10 s). The programs keep running; their windows come back as after a restart of Elegant (AppKit asks the new server). Which server, the fallback: §10.

AppKit: `kapi_lib_open_as` (`-KAPI_ENOSYS` before v97); `kapi_ws_ctl (KAPI_WS_SWITCH, 0, 0, 0)`. `image_list`'s `KAPI_IMG_ALIAS`, `image_unload`'s `-EBUSY` for an alias key. Tests: `sh tools/tests/run_image_test.sh` (*aliases*); on the Pi `/bin/aliastest` (docs/04).

v93: the volumes

Slot	Entry	What it does
230	`vol_list (out, max, flags)`	Every volume → how many there are; `out`: up to `max` `struct kapi_volume` (`name` without the `:`, `state` `KAPI_VST_MOUNTED` /

Slot	Entry	What it does
		`EJECTED` / `UNREADABLE` / `REMOVED`, `flags` `KAPI_VF_SYSTEM` / `REMOVABLE` / `RAM` / `UNSAFE` / `FORMATTABLE` / `IOERR`, `gen` — changes at each event of that volume —, `open` — the files and folders open on it —, `device_size`, `total`, `free` — only with `KAPI_VOLS_ROOM`, else `~0` —, `serial`, `type`, `label`, `device`). The FatFs volumes first (`SD`, `SD1`..`SD3`, `USB`..`USB3` — a USB volume whose device left stays listed as `REMOVED` until another takes its place), then `RAM`. Without `KAPI_VOLS_ROOM` it never reads a disk (polled every second by the menu bar).
231	`vol_eject (vol, flags)`	A USB device, named by any of its volumes (`"USB1:"`, `"usb2"`, `"USB1P2:/x"`, `"USB:"`): its volumes' open-file layer's written files synced, the device's cache flushed (SCSI SYNCHRONIZE CACHE), unmounted → 0 (it can be removed: `EJECTED`); `-EBUSY` files or folders are open on it (the written ones synced, still mounted; `KAPI_EJECT_FORCE`: ejected anyway — their later calls fail); `-EINVAL` not removable (the card's volumes); `-ENOENT` not mounted (0 when already ejected); `-EIO` pulled out meanwhile, or a file could not be written (ejected all the same).

Slot	Entry	What it does
232	`vol_mount (vol)`	A USB device ejected but still plugged in, or one `UNREADABLE` (scanned again: all its volumes), or `SD1:`..`SD3:` → 0 (mounted), `-EINVAL` no FAT / exFAT file system, `-ENODEV` no device there, `-EIO`.
233	`vol_format (vol, fmt)`	`struct kapi_format { fs, flags, cluster, label }`: a new file system (`USBn:` the whole device, one partition; `USBnPm:` that partition) (`KAPI_FMT_AUTO` — FatFs' choice: FAT16 / FAT32 by the size, exFAT from 32 GB —, `FAT`, `FAT32`, `EXFAT`; `cluster` bytes, 0 auto; a label of 11 characters at most) made and mounted → 0. **`SD:` → `-EPERM` always**; `SD1:`..`SD3:` → `-EPERM` unless `KAPI_FMT_CARD` (the caller asked the user twice); `-EBUSY` files open (`KAPI_FMT_FORCE`), `-ENODEV`, `-EINVAL` (label, cluster), `-ENOSPC` (too small / too big for that file system), `-EROFS`, `-EIO`. The caller waits (seconds on a big stick: the driver yields every 10 ms).

v92: gpio_ctl

Slot	Entry	What it does
229	`gpio_ctl (op, a0, a1, a2)`	The 40-pin header (§17; `KAPI_GPIO_*`, `kern/kapi_abi.h`) → ≥ 0, or `-KAPI_Exxx`. Anyone's: `KAPI_GPIO_INFO (struct kapi_gpio_pin *out, max)` → how many (28: GPIO 0..27, their mode,

Slot	Entry	What it does
		level, owner, PWM, the reserved ones' reason); `KAPI_GPIO_READ (pin)` → 0 / 1; `KAPI_GPIO_READ_ALL` → bit n = GPIO n; `KAPI_GPIO_NOW` → the edges' clock (µs). The pin's owner's (the first call that needs it takes it: `EBUSY` another process has it, `EPERM` the system's): `KAPI_GPIO_MODE (pin, KAPI_GPIO_M_*)` (`M_FREE` gives it back); `KAPI_GPIO_WRITE (pin, level)` (`EINVAL` not an output); `KAPI_GPIO_PWM (pin, Hz 1..1 000 000, duty 0..10000)`; `KAPI_GPIO_EDGES (pin, KAPI_GPIO_RISING | _FALLING)`; `KAPI_GPIO_EVENTS (struct kapi_gpio_event *out, max, wait ms ≤ 1000)` → how many; `KAPI_GPIO_I2C_OPEN (Hz)`, `_I2C_XFER (struct kapi_gpio_i2c *)` → bytes (`EIO`: no answer), `_I2C_SCAN (u8 map[16])` → how many answered; `KAPI_GPIO_SPI_OPEN (Hz, mode 0..3)`, `_SPI_XFER (struct kapi_gpio_spi *)`; `KAPI_GPIO_CLOSE (KAPI_GPIO_BUS_I2C / _SPI)`; `KAPI_GPIO_RELEASE` (everything of the caller's). No AppKit wrapper: GPIOKit calls it. **On the Pi 5** (the header on the RP1, Circle's same classes): the pins, the edges, I2C 1 (without the repeated start) and SPI 0 as on the Pi 4; `KAPI_GPIO_PWM` → `-KAPI_ENODEV` (the RP1's

Slot	Entry	What it does
		PWM: to come; docs/PI5-PORT.md).

v85: the mixer

Slot	Entry	What it does
225	`sound_clients (out, max)`	The programs that have a channel now → how many; `out`: up to `max` `struct kapi_sound_client` (`pid`, `name` — the program's, 24 bytes —, `volume` 0..100, `mute`, `peak` — its level now, 0..32767 —, `queued` frames). `0, 0`: only the count.
227	`ws_ctl (op, a0, a1, a2)`	(v89) The graphics server's mechanisms (above; `KAPI_WS_*`, `kern/kapi_abi.h`) → ≥ 0, or `-KAPI_Exxx`. `KAPI_WS_ACTIVE` (anyone): the server's pid while it owns the display, else 0. `KAPI_WS_REGISTER`: 1 the caller is the display server, 0 another one is, `EPERM` not the program the kernel started (v97: `elegant`, or `pocketui` for `shell = pocket / console`). `KAPI_WS_SWITCH` (v97, anyone's): the server switched (*v97* above). The server's own: `KAPI_WS_DISPLAY (take, struct kapi_ws_display *out)` → 0, `EBUSY` a full-screen program has it; `KAPI_WS_PRESENT (struct kapi_ws_present *)`: the rectangle (w ≤ 0: the screen) of its pixels shown; `KAPI_WS_INPUT (struct kapi_ws_input *out, max)` → the events taken; `KAPI_WS_WAIT (ms`

Slot	Entry	What it does
		`≤ 1000)` → `KAPI_WS_PENDING_INPUT` | `_CALL`; `KAPI_WS_ATTACH (pid)`, `KAPI_WS_POST (pid, struct kapi_event *)` → 1 queued / 0 full, `KAPI_WS_EXIT (pid)`, `KAPI_WS_BUF_MAP (struct kapi_ws_buf *)`, `KAPI_WS_BUF_FREE (id)`, `KAPI_WS_NEXT (struct kapi_ws_req *)` → 1 / 0, `KAPI_WS_REPLY (struct kapi_ws_reply *)`. A program's (through AppKit): `KAPI_WS_CALL (struct kapi_ws_call *)` → the server's status, `ESRCH` no server; `KAPI_WS_KICK`. AppKit: `kapi_ws_ctl`.
228	`proc_tree (pid, op, out, cap)`	(v91) A process's tree, by the parent pids recorded at the spawns (above). `KAPI_TREE_LIST` → how many descendants `pid` has, up to `cap` of their pids into `out` (its children, then theirs…); `KAPI_TREE_KILL` → `pid` and all its descendants terminated now, the leaves first → how many; `KAPI_TREE_KILL_CHILDREN` → its descendants only. `ESRCH` no such process, `EPERM` a kill whose tree holds the caller, `EINVAL`, `EFAULT`. AppKit: `kapi_proc_tree`.
226	`sound_client_volume (pid, volume, mute)`	That channel's volume 0..100 (−1: kept) and mute 0 / 1 (−1: kept), applied at once and remembered for the program's **name** until the restart → `volume | 0x100`

Slot	Entry	What it does
		if muted, −1: no such channel. Any program may call it (the mixer's panel).

No existing call changes its shape; what they mean: `sound_acquire` gives **a channel** (1; 0 only when the 8 are taken) instead of the whole output, `sound_write` / `sound_status` / `sound_config` are the caller's own channel's (`sound_status`'s owner: the caller's pid when it has one, else 0), and the latency in force is the shortest any channel asked for. The mapped ring (v68) stays single: the first program that maps it has it until it releases its channel (another gets 0 and plays through `sound_write`). The kernel does not keep the volumes across a restart: the Sound applet and `/bin/volume` write `SD:/etc/mixer.ini` (`media = 60`, `media.mute = 1`: `user/Include/volume.h`, `mixer_set`), read when the sound first starts.

v84: sound_output

Slot	Entry	What it does
224	`sound_output (out)`	`out` = `KAPI_SND_OUT_AUTO` (0) / `_JACK` (1) / `_USB` (2) / `_HDMI` (3): that output from now on — the running sound switches at once; an output that is not there plays nothing until it is. `out` = −1: nothing changed. → what plays now (`KAPI_SND_OUT_NOW`: 0 nothing yet, or no device), what is asked (`KAPI_SND_OUT_ASKED`) and the outputs present (`KAPI_SND_OUT_HAS (r, o)`: the jack unless the board has none, USB when a device is plugged, HDMI), or −1 (a bad value).

The kernel does not keep the choice across a restart: the Sound applet and `/bin/volume` write `SD:/etc/sound.ini` (`output = usb`, beside `volume` and `mute`: `user/Include/volume.h`), which the kernel reads when the sound first starts. No other sound call changes: see §13 *The output:* `COnyxSoundDevice`. Tests: `sh tools/tests/run_sound_resample_test.sh` (the rate converter, on the PC); on the Pi `volume output [auto|jack|usb|hdmi]` then `tone`, and the kernel log's `sound: output: …` lines.

v83: lib_open

Slot	Entry	What it does
223	`lib_open (name,`	the shared library `name`

Slot	Entry	What it does
	`min_version, err)`	mapped into the caller → its export table, or 0 with `*err` (if not 0) = −`KAPI_E*`. `name`: a bare name (`"uikit"` is `SD:/lib/uikit.so`) or a path (anything with a `/`, a `\` or a `:`; relative: to the working directory). Mapped in the caller already: the same table. The table's first `unsigned` (its version) must be ≥ `min_version`, else `-ENOTSUP`. Other errors: `-ENOENT` (no such file), `-EINVAL` (not a library of `user/Runtime/lib.ld`'s shape, a program, a relocation other than `R_AARCH64_RELATIVE`), `-ENOMEM` (memory, or no room in the arena), `-EMFILE` (16 libraries in the process), `-EIO`, `-ENAMETOOLONG`, `-EFAULT`. No `lib_close`: a library stays mapped until the process ends.

What a library is, how the kernel places and relocates it, and its lifetime: §7 *Shared libraries (v83)*. `user/Kits/appkit/appkit.h`'s wrapper returns 0 with `-KAPI_ENOSYS` on an older kernel; a program does not call it by hand — the library's bind object does, before `main` (`user/Runtime/lib.h`, docs/03 *Shared libraries*). `image_list` reports a library with `KAPI_IMG_LIB` (8); `image_preload` and `image_unload` take a library's path as they take a program's.

v82: win_resizable

Slot	Entry	What it does

The kernel does not resize the window: it shows where its frame would be, and the app applies it. An edge is the 6 pixels inside the frame's outer edge (`WIN_EDGE_BAND`), a corner reaches 18 along it (`WIN_EDGE_CORNER`); a title button under the pointer wins (`CWindow::HitResizeEdge`). On an edge the pointer shows the two arrows (v81's shapes: `_SIZE_H`, `_SIZE_V`, `_SIZE_NWSE`, `_SIZE_NESW`). A press there starts a resize (`m_pSizeWindow`): the window gets no pointer event meanwhile, and each move draws the **outline** of the frame to be — three lines, black, white, black, drawn by `Composite` over the windows, only its four sides made dirty (`SizeOutlineDirty`) — kept above the menu bar and never smaller than the smallest client area plus the frame (`SizeDragLocked`). At the release (`SizeEndLocked`) the window's pointer

handler gets **`GUI_EVENT_WINRESIZE`** (20): `lValue = (x << 48) | (y << 32) | (client_w << 16) | client_h`, x and y the frame's new top left on the screen (16 bits signed). The app then calls `resize_window2` and `move_window` — what `Root::frameResize` does (docs/03). An outline rather than a live resize: the window's canvas is made again once, not at every move (a page of Jet would be laid out again each time).

v81: set_cursor

Slot	Entry	What it does

The shape is the window's (`CWindow::CursorShape`), kept until changed. The window manager picks what it shows after each pointer event and each `set_cursor` (`CWindowManager::PickShapeLocked`): the four arrows while a window is dragged by its title; else the shape of the window that holds the pointer, or of the one whose client area it is over; the arrow on a frame, a title bar, the desktop, a window that never asked, and during a drag & drop. The shapes are images built at boot (`kernel.cpp`, `BuiltinCursorShapes`) from `kernel/gui/cursors.inc`, which `tools/gui/gen_cursors.py` writes: each shape is drawn there as a set of black pixels, its white edge added around, its hot spot given (`--show` prints them). Black with a white edge, as the arrow, which stays `kernel.cpp`'s own. A shape is at most 24 x 24 with its hot spot within 12 of its top left (`WM_CURSOR_BOX`, `WM_CURSOR_REACH`): the screen's part made dirty when the pointer moves or changes is that box around it. Apps do not call this themselves: uikit does (docs/03, `uk_cursor`).

v80: cpu_stats, net_stats

Slot	Entry	What it does
221	`cpu_stats (out)`	`struct kapi_cpu_stats` (144 bytes): `now_us` (the clock of the read), `cores`, then a `kapi_cpu_core` a core — `busy_us` (the microseconds it was busy since the boot), `role` (`KAPI_CORE_SYSTEM` 0, `KAPI_CORE_SOUND` 1, `KAPI_CORE_APP` 2, `KAPI_CORE_NETWORK` 3), `pid` (an app core's owner, 0 when free) → 0 / `-EFAULT`. Two reads make a load: `(busy_us' - busy_us) / (now_us' - now_us)`.
222	`net_stats (pid, out)`	`struct kapi_net_stats` (24 bytes): the payload bytes `pid`'s sockets received and sent (`rx_bytes`, `tx_bytes`: TCP and UDP, the old `tcp_*` calls and the BSD sockets) since it started, its

Slot	Entry	What it does
		`sockets` open now; `pid` 0: every process's since the boot → 0 / `-EFAULT`. A process that used no socket: zeros.

What "busy" is, core by core. A core with a **scheduler** — core 0, and core 3 with `netcore=1` — counts the time its tasks ran, the idle task apart: `CScheduler::Yield` adds what the leaving task ran (`m_nBusyUs`; `GetBusyUs` adds the running task's time so far, so a read from another core is live). Core 0's idle task sleeps in `wfi`, so its figure is the machine's real load; the interrupt handlers that wake it are counted as idle. The **network core**'s tasks all wait by yielding (it polls the Wi-Fi chip), so while the network works it reads 100 %; once the network has been quiet for 50 ms it sleeps between two questions to the chip (*The network core sleeps*, below) and that sleep is taken off its busy time (`CScheduler::NoteSleptUs`): a few per cent. **Core 1** counts the time it renders sound (`sys/sound.cpp`, around `Render`); an **app core** the time its jobs ran, from the entry at EL0 to the job's end or drop (`sys/appcore.cpp`; a job that waits in `wfe` still counts).

The network's bytes are counted where the calls return, on core 0 (`NetTcpSend` / `NetTcpRecv`, `NetSockSend` / `NetSockRecv` in `sys/net.cpp`: a process's tasks all run there, so the table needs no lock): one line a process (64 at most; beyond, the totals only), freed when the process ends (`NetCloseByPid`). A byte read with `MSG_PEEK` is counted when it is really read. They are the sockets' payload, not the frames on the air: no header, no retransmission, nothing of the kernel's own traffic (DHCP, DNS, NTP).

v79: kernel_info

Slot	Entry	What it does
220	`kernel_info (buf, cap)`	what the running kernel is, as `key value` lines: `name` (Onyx), `abi` (`KAPI_ABI_VERSION`), `built` (the date and time of the image's link), `rev` (the source's git revision; `+`: built from changed sources), `machine` (aarch64), `model` (the board's name), `ram` (MB), `board` (`pi4` or `pi5`: the distribution the kernel is of, docs/PI5-PORT.md -- `pkg` takes its repository and checks the index by it), and on a board with its own keyboard (the Pi 400, the Pi 500) `keyboard <layout> <code>` (2026-10-10): the country code

Slot	Entry	What it does
		the firmware reads from the board's OTP (`/chosen/rpi-country-code` in the device tree; Raspberry Pi's codes, as its piwiz: 1 GB, 2 FR, 3 ES, 4 US, 5 DE, 6 IT, 7 JP, 8 PT, 9 NO, 10 SE, 11 DK, 12 RU, 13 TR, 14 IL, 15 HU, 16 KR) and the `SD:/etc/keymaps` layout for it (`UK`... `DK`, `-` none) -- Setup takes it. Up to `cap` − 1 bytes and a NUL → the text's whole length; `EFAULT`. Keys may be added: a reader looks its keys up.

The date and the revision are `kernel/buildstamp.cpp`'s, an object that depends on every other object and library of the kernel (`kernel/Makefile`): it is compiled again at each link, so the stamp is the image's — `__DATE__` in a file says when that file was last compiled. The boot log's `Built on …` line prints the same. User: `/bin/uname` (docs/04 §8) — a kernel copied to the card by hand is told from the package's (`uname -v` against `uname -p`). `user/Kits/appkit/appkit.h`'s wrapper returns `-KAPI_ENOSYS` and an empty text on an older kernel.

> **Historical note.** `ARCHITECTURE.md` §11 describes an earlier approach where the build emitted a `user/kernel_syms.ld` (`kapi_x = 0xADDR;`) and the apps were linked against those addresses. This approach is **no longer used**: the fixed-address table replaced it. The apps no longer depend on the kernel build.

---

9. Stream / stdio subsystem

Source: `kernel/sys/stream.cpp`, `kernel/include/kern/stream.h`.

This is what makes the terminal's **pipes and redirections** and inter-process communication possible.

Object model

Abstract base class `CStream` with reference counting (`AddRef`/`Release`; destroyed when the last reference drops). Three implementations:

Class	Role
`CPipeStream`	in-memory FIFO (ring buffer ~8 KB) between two tasks
`CFileStream`	FatFs file presented as a stream (read,

Class	Role
	write/truncate, append)

Virtual methods: `Read` (cooperative blocking: yields until ≥1 byte or EOF; 0 = EOF), `ReadNonBlocking` (`>0` / `0`=EOF / `-1`=would block), `Write` (may block if full), `CloseWrite` (signals "no more writing" → readers see EOF). (v76) `AddWriter`: a pipe's write end carried to another process (`KAPI_HXF_WRITER`) is one more writer — `CloseWrite` ends the data only when every writer has called it (a received write end's close, or its process's end, counts as its call: `HKIND_STREAM_WRITER`).

- **`CPipeStream`**: ring buffer `head`/`tail`. `Read` waits while empty and the write end is not closed; `Write` waits while full. Cooperative → no lock. (v75: the waits sleep in `IoWait` and every change calls `IoWake`; `WriteNonBlocking` / `stream_write_nb`, `PollMask`: §8 *v75: files and processes*.)
- **`CFileStream`**: wraps a FatFs `FIL`; modes 0=read, 1=write+truncate, 2=append.

Processes and stdio

- `CAddressSpace` owns `m_pStdin`/`m_pStdout` (set by `SpawnProcess`).
- `kapi_stdin_read`/`kapi_stdout_write` read/write these streams (or log if absent).
- `kapi_spawn(path, args, in, out)` → `SpawnProcess`: the child task takes a reference on `in`/`out`. `kapi_wait` blocks (cooperatively) on `CProcess::bDone` then returns the exit code (v75: −9 for a killed child, −11 for a crash; `spawn_ex` / `proc_wait` add argv / environment blocks and the reason: §8). When the child's address space is destroyed, stdout gets `CloseWrite` → the reader (the terminal) sees EOF.

The terminal thus chains the `stdout` of one stage to the `stdin` of the next via `CPipeStream`s, and reads the final output non-blocking. App-side details in the developer guide and the user guide.

---

10. Graphics subsystem (GUI)

Since 2026-10-05 the window manager, the compositor and the routing of the input are no longer in the kernel. They are **Elegant**'s, the graphics server, a user process (`SD:/bin/elegant`, `user/Servers/elegant`; §8, v89: what the kernel gives it; `docs/GUI-USERSPACE-STUDY.md`) — and, since 2026-10-08, **PocketUI**'s in the pocket and console modes (below). The window manager described in §10.2 and §10.3 below is the same code, moved: `user/Servers/common/wm/window.cpp` and `wm/kern/gui/window.h` (with `wm/cursors.inc`), built for a user process with stand-ins for the few Circle headers it includes (`user/Servers/common/port`) — read "the kernel" there as "the graphics server".

What the kernel keeps (`kernel/gui/kwin.cpp`, `kern/gui/window.h`, 150 lines):

- `CWindow` — only a program's **queue of events** (with the request to end and the wake of `kapi_pump_wait`): one a program whose windows are Elegant's; Elegant pushes into it. No pixels.
- `CWindowManager` — the **keys' state** (the modifiers, the held keys: `get_modifiers`, `key_held`) and **the full screen** (the program that has it and its buffer; `present_fb` still sends that buffer to the display itself).
- `GImage` (§10.1; Elegant compiles the same file) for `draw_text_buf`'s bitmap font; the surfaces (`surface.cpp`); the frame buffer, its DMA, the change of resolution; the USB input drivers, whose events go to Elegant's ring.

- **The display task** (`CCompositorTask`, `kernel.cpp`: it composes nothing any more): it starts Elegant again when it ends (`WsPoll`), takes the display from a silent one (`WsWatch`), does a resolution change. **No server** (it cannot be started, does not take the display, or ended 5 times): the kernel's **console** takes the screen (`DebugConsoleTakeover`: its log); the Pi is reached by telnet.
- **The kernel's table** has no entry for the windows (kapi v90, §8): the 36 of the windows (`create_window`, `present`, `set_menu`, `win_list`, `drag_begin`, `wallpaper_*`, `desk`...) and the 2 of the activity shell (`register_shell`, `shell_request`) are removed, the table is compacted (228 entries), their functions are gone from `sys/kapi.cpp` (740 lines). AppKit's functions of those names spoke to Elegant (`appkit_ws.inc`); no program was rebuilt. The kernel image lost 43 KB. Since 2026-10-08 the window calls are **UIKit's** (`uk_win_*`, `uikit/win.h`; the protocol `user/Kits/uikit/port/elegant.h`), AppKit's removed (§8 *The window API moves to UIKit*).
- **Print Screen** and **the wheel's speed** are the graphics server's too (§2's input task; `user/Servers/common/serve.cpp`).

The server per mode (v97, 2026-10-08; `docs/POCKETUI-TECH-STUDY.md` §3.8, §8): the kernel reads `SD:/etc/system.ini` **`shell =`** (`sys/wsrv.cpp` `ShellRead`, at the boot and at each switch): `desktop`, no line, or a word it does not know (logged) → **Elegant**, `SD:bin/elegant --serve` — exactly as before; `pocket` / `console` → **PocketUI**, `SD:bin/pocketui --serve --mode <m>` (phase P3, below). The role (`KAPI_WS_REGISTER`) is for the program started (its task's name). The server asked for that is missing, ends before it has the display, or does not take it in 5 s (then it is killed) → **Elegant instead**, its aliases dropped (`KAPI_WS_SWITCH` answers 1) — a bad setting never leaves the screen dark; Elegant failing too → the console. `WsPoll` starts **the server that was started** again (`--serve --restart`, plus its `--mode`), 5 times at most; PocketUI that cannot stay up → Elegant, then the console. A start (the boot's, a switch) holds `WsPoll` off. **`KAPI_WS_SWITCH`** (§8 *v97*): the server ended (`KAPI_WS_IN_QUIT`, killed after 3 s), every alias dropped, the relaunch counter reset, `shell =` read again, its server started with the same fallback. The graphics session's programs are `/bin/session`'s (phase P4: it closes them, writes `shell =`, switches, runs the new mode's session file); until then a switch by hand: `aliastest --switch`.

Two servers, one common code (PocketUI's phase P3, 2026-10-08; `docs/POCKETUI-TECH-STUDY.md` §7): `user/Servers/common/` holds what is not a policy, compiled into both servers (`common/common.mk`): `serve.cpp` (the loop: `KAPI_WS_REGISTER`, the display, the raw input, the requests, the 16 ms pace, the present, the shared buffers `el_shared_*`, the kernel's `el_sys_*`, Print Screen, the wheel's speed, the 5 s statistics), `route.cpp` (the keys to the policy then to the window that has them; the programs' events forwarded; the windows' places kept in the kernel — `KAPI_WS_STATE` — for a server started again; the focus told), `core.cpp` / `core.h` (the window store behind plain functions, the shots), `ops.cpp` (the requests' decoding: Elegant's operations, `uikit/port/elegant.h`) and `wm/` (`CWindowManager`, the compositor). A server's **policy** (`common/policy.h`, `struct ws_policy`: `create` — a window's place, size and flags, or refused —, `op` — a request answered first —, `key`, `tick` — once a turn —, `screen`, `registered` — the role taken, the display not yet —, `start`) is all that differs; every hook may be 0. **Elegant** (`user/Servers/elegant/main.cpp`) has none but `start` (voronoy run again after a restart) and its demonstration: its behaviour is unchanged (checked on the PC: `tools/tests/server_sim/run.sh` composes the same app under Elegant before and after the extraction — the same pixels). **PocketUI** (`SD:/bin/pocketui`, `user/Servers/pocketui/`: `main.cpp`, `wm.cpp` the policy, `band.cpp` the status band) is described in docs/03 §5.10.3 and docs/04 §5 "The pocket and console modes": its UIKit under the name `SD:/lib/uikit.so` (`kapi_lib_open_as` in its `registered` hook, after the role and before the display — §7 *Aliases*), a

24 px status band of its own (a topmost system window at y = 0: the work area below it; none in console), a program's window **filled** (frameless, at the work area's top left; a resizable one sent `GUI_EVENT_WINRESIZE` to the work area's size, then `GUI_EVENT_WINCTL` maximise if it did not apply it) or a **card** (framed, centred, when it fits), its popups where asked, the desktop's bands (a backmost window, a topmost one on the top or bottom edge) refused, one workspace, **one app in front** (the others' windows set aside: `CWindow::SetAside`, hidden as an off-desk window — a flag Elegant never sets), Alt+Tab. Its protocol is `user/Kits/uikit/port/pocket.h`: Elegant's operations and numbers (`elegant.h`, included) with PocketUI's meanings, plus its own from 0x100 (`PK_OP_HELLO`, `PK_OP_SERVER`; the shell's `PK_OP_SHELL`.. `PK_OP_GRAB` since phase P5 — the pocket shell `pocketshell`: its home, the tasks, the thumbnails, the system keys, docs/03 §5.10.5). Since 2026-10-08 (the user's rule) **every app's main window is full screen** (filled; a fixed one smaller than the work area centred over a matte of its colour), cards being a program's other windows and the apps of `SD:/etc/pocketui.ini [cards]`; the keys follow the front program.

Source: `kernel/gui/{gimage,kwin,surface}.cpp` + headers (`kern/gui/`); the window manager: `user/Servers/common/wm`. Rendering core ported from the author's FreeBASIC `SimpleOS`.

10.1 `GImage` — software rendering engine

- **Pixel format: 32-bit `0x00RRGGBB`** (alpha byte unused).
- **Transparency color: magenta `0xFF00FF`** (transparent-blit key).
- A `GImage` owns its buffer (`SetSize`) or **wraps** one (`Wrap`, e.g. the framebuffer's back buffer or a window's canvas).
- Primitives: `Clear`, `SetPixel`/`GetPixel`, `DrawLine` (Bresenham), `DrawRectangle`, `FillRectangle`, blits `PutOtherRaw`/`PutOther` (with key) / `PutOtherPart` (sub-rect, used by the 9-slices skin engine).
- **Text**: Circle's bitmap font (`CCharGenerator`). `FontWidth/Height`, `DrawChar` (transparent background), `DrawText`.
- **BMP**: `LoadBMP` (24 bpp uncompressed, BGR→RGB, handles top-down/bottom-up).

10.2 Windows and compositor

- **`CWindow`**: position, logical size, title, flags (`WIN_FLAG_BORDERLESS`; `BACKMOST`: the desktop's band; `TOPMOST`: the menu bar, the dock — above every window, never active, never the keys; `TRANSPARENT`: the magenta key; `SYSTEM`: not listed as an open app; `ALPHA` (v64): the canvas's top byte is a transparency; `FIXED` (v69): the user cannot move it — no drag, no double-click maximise, `HitTitleButton` finds no button (uikit draws none: `UK_WIN_FIXED`) — and `OnScreenResized` centres it again at a new resolution instead of only moving it in: Setup's), a `GImage` **canvas allocated 64 KB-aligned and physically contiguous** (mapped into the app at `USER_WINDOW_CANVAS` = 12 GB — the app draws directly, with no per-pixel call), the frame's two copies (active, inactive: mapped at `USER_WINDOW_CHROME` / `_INACTIVE`, drawn by the app — kapi v28 `get_chrome`; uikit: `uk_decorate_window`), an event queue (spinlock-protected ring), the app's handlers, its menu.

- **The frame** (v64, the modernised CDE): a 28 px title bar and 4 px borders (`WIN_TITLEBAR_H` / `WIN_BORDER` = `KAPI_FRAME_TITLE_H` / `KAPI_FRAME_BORDER`, shared with the apps in `kapi_abi.h`), rounded corners (radius `KAPI_FRAME_RADIUS` = 8): in the frame's copies a pixel's top byte is its transparency, heeded in the four corner squares only. `DrawTo` blits the frame's **bands** only (the title bar, the bottom border, the two sides beside the client area — never under it) opaque, the corner squares blended by their pixels' transparency (`BlendAlphaRect`), then the client canvas.

`CoversOpaque (rect)` — may the compositor skip what lies below? — leaves out only the corners' see-through pixels (`CornersIn`: on a corner's row *k* from the edge, its first `CornerSpan (k)` pixels — those not wholly inside the arc): a rectangle inside the window still covers. *Borderless* windows have no frame.

- **The present's damage** (`PresentDamage`): an app's present dirties **its client area only** — its whole window when its frame changed since (`get_chrome`, a resize: `ChromeGen`) or it is faded — so a window refreshing alone (an emulator) keeps the compositor's fast path: its client rectangle is covered, nothing below it is drawn. That first present after a frame's redraw also bumps `ChromeGen` once more: the remote desktop (`rdpd`), which may have read the frame while the app was still drawing it (`get_chrome` bumps it **before** the drawing), reads it again, whole — no frame left without its buttons on the PC. A window's two frame copies are allocated both or none (`AllocChrome`: no half-mapped frame).

- **The title buttons** (v64): the window menu at the left; close, maximise, minimise from the right, at `KAPI_FRAME_BTN_*` (the app draws them, the kernel hit-tests them: `HitTitleButton`). Close and minimise act at their release over them (`RequestExit`, `Minimise`); the window menu (at the press) and maximise (also a double click on the title bar) go to the app as `GUI_EVENT_WINCTL` (18; value `KAPI_FRAME_MENU` / `KAPI_FRAME_MAXIMISE`): uikit's `Root` shows its window menu, maximises and restores (the developer guide).

- **Minimised windows** (v64, `SetMinimised`): not drawn, not hit, never the active window nor the keys' target (`ActiveLocked`, `KeyTargetLocked`), their area damaged as they go and come back; `Raise` (`win_raise`, `raise_app`: the dock, the menu bar's Open Windows) brings one back.

- **Workspaces** (v65): a window's desk (`CWindow::Desk`, −1 = every desk: the topmost, backmost and system windows) is the current one when it is added (`CWindowManager::m_nDesk`, of `m_nDesks`, 4 at boot — the dock sets its number from `SD:/etc/dock.ini`). `SetDeskLocked` flags every window of another desk `OffDesk`; `Hidden ()` (minimised or off-desk) is what the compositor, `DrawTo`, `CoversOpaque`, `HitTest`, `ActiveLocked`, `KeyTargetLocked` and the bottom inset heed, and `SetOffDesk` damages the window's area as it goes and comes back. `ForgetHiddenLocked` drops the pointer's state (hover, capture, a drag, a pressed title button) naming a window that went hidden. `RaiseLocked` of an off-desk window shows its desk first. `OnKey` takes **Ctrl+Alt+Left / Right** (the xterm modifier form `ESC[1;7C` / `D` too) before the apps: the previous / next desk, with Shift the active window moved along (`MoveToDesk`), not while an app is full screen. `DeskInfo`, `SetDesk`, `MoveToDesk` for kapi `desk` / `win_desk` (each change bumps `m_nDeskGen`: the dock redraws its pager).

- **See-through windows** (`WIN_FLAG_ALPHA`, borderless): each pixel blended by its top byte (0 opaque .. 255 see-through, times the window's fade); never `CoversOpaque`; a click on a wholly see-through pixel goes to the window below (`HitTest` → `OpaqueAt`). The dock (its rounded corners, the gap under it — almost clear, top byte 254, while a drawer is open, to catch a click elsewhere), the menu bar (its drop-downs), the agenda widget (its text straight on the wallpaper).

- **The work area** (`WorkArea`, v64 `win_geometry`): the screen less the top inset (a topmost window at y = 0: the menu bar, its smallest height) and the bottom inset (the topmost windows standing on the bottom edge: the dock, its smallest height). New windows are placed in it; a maximised window fills it.

- **A window that grows** (v64 `resize_window2`): `CWindow::Grow` allocates a bigger canvas and frame copies; the process's mappings are moved to them at the same addresses (`MapContig`, then `CAddressSpace::FlushTLB`: `tlbi aside1is`); the old memory is retired and freed by the compositor three frames later (`FreeRetired`: a frame begun before may still be reading it).

- **The screen's size, while running** (v66 `screen_set`, the Control Panel's Display applet): the compositor task makes the frame buffer again (`C2DGraphics::Resize`) between two frames, then `OnScreenResized` keeps the windows on the screen and sends them `GUI_EVENT_DISPLAY_RESIZE`. The apps placed by the screen's size place themselves again on it: the menu bar (its canvas grown to the screen: `resize_window2`), the dock (`onDisplayResize`: laid out for the new width, along the bottom), the notifications (the top right corner); uikit re-maximises a maximised window and shrinks / moves one past the work area ~0.3 s later (`Root::displayTick`: the dock has moved, the work area is the new one). vncd sends the VNC `DesktopSize` pseudo-rectangle (−223) to a client that takes it (else it closes the session: the client connects again at the new size); rdpd sends `SCREEN` (8: u16 w h) and Onyx Remote resizes its view of the Pi's screen.

- **`CWindowManager`** (singleton): a Z-ordered list (bands: backmost, normal, topmost; the last = on top), protected by a `CSpinLock`. `Add`/`Remove`/`Raise`/`Minimise`. `Composite(screen)`:

    1. snapshot the list under the lock (the compositor also frees the grown windows' old memory), then blit outside the lock;
    2. find the topmost window that covers the clip rectangle opaquely: nothing below it is drawn;
    3. otherwise clear the desktop and blit the wallpaper;
    4. draw the windows back to front (`DrawTo`, `bActive` for the keys' window);
    5. the drag badge, then the cursor (`mousecur.bin` bitmap or a fallback arrow).
- **Dirty rectangles**: `Composite` honours the screen image's clip rectangle, so the compositor task redraws only the damaged rectangles (see the service tasks above); host tests `sh tools/tests/run_gui_test.sh` (clipped drawing = full drawing inside the clip, nothing touched outside) and `sh tools/tests/desktop_sim/run.sh` (the v64 / v65 windows: `wmtest.cpp` — the corners, `CoversOpaque`, the present's damage, the title buttons, minimise, the see-through windows' clicks, the work area, a canvas growing, the workspaces).

10.3 Events and input (kernel side)

The kernel draws **no widget** any more: the kernel-drawn widget API (buttons, labels, check boxes, text boxes, sliders, scroll bars, icons) was removed from the table by the v29 compat break; every app draws its own controls with **uikit** (`user/Kits/uikit`, docs/03 §6). The window manager keeps the window-level work: the hit-test of the frame (title bar, its buttons, the borders), raising and moving windows, focus, and **pushing events** into the window's queue; the app's `pump_events` (user-side since v74: `pop_event` / `pop_post` in a loop, §6) calls its handlers **in the app, at EL0**. Events (`kern/gui/window.h`, mirrored in `user/Kits/appkit/appkit.h`):

Constant	Meaning
`GUI_EVENT_KEY` (5)	key pressed (`value` = char or `KEY_*`; the modifiers through `get_modifiers`)
`GUI_EVENT_CANVAS_CLICK` / `_MOTION` (6, 7)	client press / drag: `value=(buttons<<32)|(x<<16)|y`

Constant	Meaning
`GUI_EVENT_PTR_MOVE` … `_WHEEL` (8–13)	the pointer stream for app-side toolkits (`set_pointer_handler`, v22)
`GUI_EVENT_MENU` (14)	a menu-bar command (`set_menu`, v39)
`GUI_EVENT_DROP`, `_DRAG_OVER`, `_DRAG_DONE` (15–17)	drag & drop (v42)
`GUI_EVENT_WINCTL` (18)	a title button handed to the app (v64)
`GUI_EVENT_DISPLAY_RESIZE` (19)	the screen's size changed (v66)

Mouse handling: hover tracking, press edge (raises the window, hit-test of the frame / canvas), drag (window move / resize), release edge. The keyboard (Circle's "cooked" VT100 strings) is translated into logical keys (`KEY_UP`, `KEY_ENTER`, …) and delivered to the modal dialog, otherwise to the app's keyboard handler.

10.4 Theme, wallpaper

- **The theme** is the apps' business: the windows' frames, like every control, are drawn by the apps (uikit: `user/Kits/uikit/skin.cpp`, `paint.cpp` — by code, no bitmap) from `SD:/etc/theme.txt` (`theme` = Peach / Steel / Sage / Brick / Slate, or `active`; `inactive`, `face`, `accent`, `outline`, `dock`: read by `uikit/theme.cpp`); the kernel only blits the frames. Of that file Elegant reads `wheelspeed=N` when it starts (the kernel reads nothing of it). (The old 9-slice window skin — `wings.bmp` tinted by `CSkin` — and `kapi_set_window_theme` are no longer used by the desktop.)
- **Wallpaper**: `wallpaper_generate` (toroidal Voronoi generated at runtime), or **an app-drawn background** (what the desktop uses: `voronoy` paints the patterns and pictures): `wallpaper_buffer` maps the screen-sized shared buffer at `USER_WALLPAPER_CANVAS` (13 GB), the app draws, `wallpaper_commit` makes it live. The frames are **owned by the kernel** → the background persists after the app exits (the `voronoy` case). The agenda widget reads it to choose its ink.

10.5 Modal dialogs

None in the kernel any more: `message_box`, `file_open`, `file_save` (v9/v10, `CDialog`) were removed; the dialogs are user-side (uikit's `MessageBox`, `FileDialog`…), modal within their app.

---

11. Network subsystem (WLAN, TCP/IP, NTP)

Sources: `kernel/kernel.cpp` (`CNetBringupTask`), `kernel/sys/net.cpp` (socket backend), `kernel/include/kern/net.h`. Built on Circle's `lib/net` (TCP/IP) + the `addon/wlan` BCM4343 driver + `wpa_supplicant`, all linked into the kernel (see `kernel/Makefile` `LIBS`).

Two placements, chosen at boot by `cmdline.txt netcore=`:

- `netcore=0` (the default): the whole stack on the primary core, as ordinary cooperative tasks among the others; the socket calls run on the app's task.
- `netcore=1`: **the network core.** Core 3 runs the stack with its own scheduler (§5): at boot it waits in `WFE` (`NetCoreMain`, from `COnyxCores::Run`); once the SD card is mounted, core 0 calls `NetCoreStart`, and core 3 creates its `CScheduler`, the bring-up task (below) and 6 **worker** tasks.

Everything the bring-up starts — Circle's `CNetTask`, DHCP, NTP, the WLAN driver's kprocs, `wpa_supplicant` — is created on core 3 and runs there, so the stack is still used from one core only, as it was written for. The WLAN firmware and `wpa_supplicant.conf` are read from core 3 through FatFs: the volume lock (`sys/fslock.cpp`) takes its owner with an atomic compare-and-swap. The SDIO interrupt stays on core 0 (its handler only masks the controller's interrupt enable; the driver polls), and so do the TCP / ARP kernel timers (they set flags under spin locks, which the stack reads in `Process`).

- **Requests.** A socket kapi (`tcp_*`, `net_resolve`, `net_ping`, `net_info`, `wlan_scan`) copies its arguments — the app's memory is not mapped on core 3 — into one of 32 request slots (8 KB of data each: a bigger send goes in pieces), marks it *posted*, and waits: it spins 300 µs (most answers take microseconds), then yields, then sleeps 1 ms, then 10 ms steps (an `accept` may wait for hours). A worker claims the slot (compare-and-swap), runs the same `Do*` function as `netcore=0`, and marks it *done*; a `recv` gathers several segments (up to 8 KB) in one round trip. When every worker waits (accepts, connects), the core adds one (up to 24). `net_status` reads the state directly.
- **BSD sockets (v75).** Their calls are requests too (`NR_S*`), except a connect: a **detached** request (the caller does not wait; the worker frees it; `NetCloseByPid` leaves it alone). The main loop recomputes every open socket's readiness snapshot at each turn and bumps a generation word when one changed, then interrupts core 0 (`IPI_NET_READY`, `COnyxCores::IPIHandler` → `NetReadyIPI`; the 100 Hz tick hook `NetPollTick` as the fallback): `IoWake`, which wakes `poll` and the blocking BSD calls (§8 "v75: sockets and poll").
- **The Wi-Fi driver** is polled on this core instead of waiting for its SDIO interrupt, its scans cover both bands and probe for the networks of `wpa_supplicant.conf` by name, 5 GHz is preferred, a frame is one SDIO command on a 50 MHz bus (`onyx_wlfast = 31`), the frames sent are not aggregated (`onyx_wl_ampdu_tx = 0`: aggregated, half of what the Pi sent during a download was lost — the remote desktop froze) but sent in bursts (`onyx_wl_frameburst = 1`), and TCP scales its window and delays its acknowledgements (`onyx_tcp_ws = 3`, `onyx_tcp_ackn = 8`) (`NetWlanOptions`, `NetWlanNames`; our Circle fork, docs/05 §26 and §27).
- **A trial.** A Pi is often reachable by its Wi-Fi only: a driver change that keeps the Wi-Fi from coming up cannot be taken back from the PC. `SD:/etc/net-trial.txt` holds `name=value` words — `wlfast=` (the driver's fast path bits), `tcpws=`, `tcpwin=`, `ackn=`, `ampdutx=`, `ampdurx=`, `bawsize=`, `rxbawsize=`, `ampdurts=`, `bw5=`, `frameburst=`, `netsleep=`, `netsleepus=`, `netirq=`, `netstat=1` (the statistics, the firmware's counters, TCP's timeouts in the log), `secs=` (default 180) —; the bring-up reads it, **deletes it**, applies it for this boot (`NetTrialLoad`), and core 0's main task restarts the Pi after `secs` (`NetTrialPoll`) unless `SD:/etc/net-trial.keep` exists by then: the next boot is without the trial. Before the restart the kernel log's tail (24 KB) is written to `SD:/etc/net-trial.log`: what the driver said during a trial that cut the network is read after it. A new switch is added there, tried, and only then made the default.
- **The network core sleeps when the network is quiet** (2026-10-04). Its tasks wait by yielding, so the core used to turn all the time: 130 000 rounds of its scheduler a second, each asking the chip whether it has a frame. Now `NetCoreMain`, after each round, sleeps (`wfe`) when all of this holds: no frame read from the chip or written to it for 50 ms (`onyx_wl_lastact`, the driver's), no request posted by core 0 for 50 ms, none waiting, and

the driver has asked the chip once, whole, since the last sleep and it had nothing (`onyx_wl_polls`: a question is several SDIO commands and each waits a round — sleeping between the rounds made a ping take six sleeps). The sleep ends after 10 ms, or at once when core 0 posts a request (its `sev`) or when **the card's interrupt** comes: before it sleeps the core asks the controller whether the card's interrupt is pending (`sdiocardintrpending`, `emmc.c`: a register read, no SDIO command) — pending: no sleep; else that interrupt is enabled, and its handler, on core 0, sends the event. A `wfe` also ends every 1.2 ms (the timer's event stream, `CNTKCTL_EL1`, set on that core) and at every spin lock released on any core (Circle's unlock sends the event): the conditions are looked at again each time. The first frame or request brings the full pace back. Measured (Pi 4, a quiet Pi pinged 150 times): 4.1 ms on average without any sleep; 4.4 ms with a sleep of 1 ms and no interrupt (the core asleep 93 % of the time); 4.0 ms with the interrupt and a sleep of 10 ms (asleep 98–99 %); 500 pings during a download limited to 1 MB/s: none lost. Trial words: `netsleep=<ms>` (0: never sleep), `netsleepus=<us>` (the sleep's length), `netirq=0` (the time alone ends the sleep).
    - **`netstat=1`** (`cmdline.txt`): every 5 s the log says the net core's pace (`net: core 3: N rounds/s, … us a round` — the stack's tasks all wait by yielding, so a round is the unit of every wait: ~13 µs) and the driver's (frames a second, a frame's read time, the link's rate).
    - **A process that dies** (in a request, or with sockets open): `NetCloseByPid` (its teardown, IRQs masked: nothing waits) drops its posted requests, orphans the running ones (the worker then closes what they opened), and queues its pid in a ring that the net core's main loop reads to close its sockets.
    - **Notices.** `IpcNotify` is core 0's: the bring-up leaves its "Connected" notice to `NetCoreNotify`, and the kernel's main task delivers it (`NetCorePoll`, every 250 ms).
    - Core 3 is then no longer an app core (§14).
- **Bring-up is non-blocking and non-fatal.** `CNetBringupTask` (a kernel `CTask`, started from `Run()` once the SD card is mounted) runs `CBcm4343Device::Initialize` → `CNetSubSystem::Initialize(FALSE)` → `CWPASupplicant::Initialize`, waits for the DHCP bind, logs `net: up, IP …`, then **returns** (the task ends cleanly). A missing firmware / `wpa_supplicant.conf` / access point only logs a warning — the GUI boots regardless.

- **The cable or the Wi-Fi** (2026-10-09; `system.ini` `network = auto | ethernet | wlan`, the default `wlan` on the Pi 4 — the path above, unchanged — and `auto` on the Pi 5). Not `wlan`: the device type is set to Ethernet (`CNetDeviceLayer::SetDeviceType`, fork patch 30, docs/05), `CNetSubSystem::Initialize(FALSE)` brings the wired device up (the Pi 5's RP1 MACB; the Pi 4's BCM54213), and the task waits up to 6 s for its link. Link up (or `ethernet`): DHCP runs on it, no WLAN at all. No link with `auto`: `CBcm4343Device::Initialize`, the type set to WLAN — before any link was up, so DHCP (`CNetSubSystem::Process` starts it on the first device running) has asked nothing yet —, then wpa_supplicant as above. `g_nNetLink` (`kern/net.h`) says which; `NetStatus` (the menu bar's network icon) asks the cable's link instead of the association on Ethernet, and `net_info` has a `link` line. The choice holds until the next start.

- **Self-driving.** `CNetSubSystem::Initialize` spawns Circle's own `CNetTask` / `CPHYTask`; they run as ordinary cooperative tasks on our scheduler (§5), so no explicit pumping is needed. (This is also why the net task is CPU-busy — the reason for `netcore=1`.)

- **Globals.** `g_pNet` (the `CNetSubSystem`) and `g_bNetUp` are published in `net.h`; `NetIsUp()` gates the socket calls.

- **Sockets.** `sys/net.cpp` keeps a small table of Circle `CSocket`s behind integer handles. `tcp_connect` resolves a dotted-quad or a DNS name and connects (blocking, cooperative); `tcp_send` blocks with a 5 s timeout; `tcp_recv` is **non-blocking** (so a GUI app polls it from its frame loop; with `netcore=1` one call gathers the segments that fit in its 32 KB request buffer, `NET_REQBUF`); `tcp_close` drops it. A connect **takes its slot before it blocks** (`SLOT_CONNECTING`, invisible to the other calls): the net core's workers run other connects during a DNS lookup or a handshake, and two of them once got the same slot (both fetches then read one connection: a browser's style sheets failed). **DNS cache**: 32 names kept 5 minutes (`ResolveName`, used by `tcp_connect`, `net_resolve`, `net_ping`) in front of Circle's `CDNSClient` -- which answers in milliseconds now (it slept 1 s per query: docs/05 §18). Each socket records its **owner pid**; `kapi_exit` calls `NetCloseByPid` as soon as the app's other threads have ended (2026-09-30: it waited for the janitor's reaping, `AddressSpaceTaskTerminate`, which calls it again and finds none left), so a process that exits or dies without closing does not hold its connections or table slots -- a browser relaunched at once found them still held and its pages waited (10 s). The table has **256 slots** since v75 (`MAX_SOCKETS`; 64 before, 16 before that: a browser keeps a dozen open), and it also holds the **BSD sockets** of v75 (TCP and UDP, non-blocking calls, `poll`: §8 "v75: sockets and poll"); `tcp_recv` reads through a slot's carry buffer (a buffer smaller than a segment lost the rest of it with `netcore=0`). `net_info` lists every socket that has a Circle socket: `tcp <n> listen|conn …` and, since v75, `udp <n> bound <port> <default peer|-> <pid>`.

- **TCP fixes in our Circle fork** (2026-10-01, `docs/05` §20–22; host test `tools/tests/run_circlenet_test.sh`): only a **real duplicate ACK** (no data, no SYN / FIN, the window unchanged, data in flight, ACK = SND.UNA: RFC 5681 §2) counts towards a fast retransmit -- a remote desktop client's input messages (data segments that do not advance the ACK) started a spurious fast retransmit + recovery while the mouse moved, the Pi's sending throttled to ~2 segments a round trip. The **RTO** starts at 1 s (RFC 6298; was 3 s) and its floor stays **1 s** (200 ms was tried and undone: spurious timeouts over Wi-Fi to Windows collapsed the remote desktop's throughput), Karn's algorithm also after a fast retransmit; a connection is given up after 6 timeouts (63 s). **`tcp_send`'s count** is exact: Circle's `CSocket::Send` queues MSS-sized chunks, and when one times out (5 s, its queue full) the chunks before it are counted (it answered the error, those bytes already on their way: an app resending "the rest" put bytes twice in the stream); `NetTcpSend` adds up its 32 KB requests the same way. A short count means the bytes queued, then a timeout: resending the rest is right (rdpd does).

- **Clock.** Once the link is up the bring-up task starts a `CNTPDaemon` (`system.ini ntp=`; `off` or `none`: no daemon, the clock is not set), which updates `CTimer`'s wall clock; the boot reads `system.ini timezone=` (minutes from UTC) and calls `CTimer::SetTimeZone` so `get_datetime`, the agenda and the log timestamps show local time (`kapi_set_timezone`, v69, changes it while running).

- **Host name.** `system.ini hostname=` (letters, digits, `-`) is handed to `CNetSubSystem` before the bring-up task starts (`SetHostname`, an Onyx addition to Circle: docs/05) — the name DHCP announces; default Circle's `raspberrypi`. Setup writes it; it takes effect at the next start.

- **Caveats.** The kernel's sockets are plain TCP and UDP (TLS is done in user space: `user/Libs/tls`, used by Jet, Mail, `httpsget`, `wget`, the package manager…); the scheduler's task list has no limit any more (§5: the old `MAX_TASKS` of 40, raised for the net workers, is gone); the firmware load uses FatFs and is not locked against concurrent app file I/O (low risk, one-shot at boot) — with `netcore=1` it is (the atomic volume lock).

The apps that use it: Jet Browser, Mail, the IRC client, Lisa (an LLM), the package manager (`pkgman`, `pkgd`, `/bin/pkg`), and the `/bin` network tools (`net`, `ping`, `nslookup`, `netstat`, `wget`/`httpget`/`httpsget`, `ftp`, `whois`) and servers (`ftpd`, `telnetd`, `vncd`, `rdpd`) — docs/04 §12 and the `/bin` table.

---

12. Sound and the second core

Source: `kernel/sys/sound.cpp`, `kern/sound.h`.

- **Multi-core.** Circle is built with `ARM_ALLOW_MULTI_CORE` (fork patch #4, Circle Changes). `CKernel::Initialize` starts cores 1–3 through a `CMultiCoreSupport` subclass (`COnyxCores`, kernel.cpp) right after the kapi table is published. **Everything else stays on core 0** (but the network with `netcore=1`, on core 3: §11): the scheduler, every process, the interrupts (the GIC routes peripherals to core 0; core 1 runs Circle's own `VectorTable`, not our `KVectorTable`). Core 1 runs `SoundCoreMain`; cores 2 and 3 are **app cores** (§14). A failed start is only a warning (no sound producer, no app cores).
- **The mixer (v85).** `sys/sound.cpp` keeps `SND_CLIENTS` (8) channels, one per program that plays (`TClient`: the pid, the program's name, a PCM ring of 0.5 s made once and kept, the volume 0..100 and the mute turned into a 16.16 gain on a squared curve, the latency asked, a peak meter). `Render` adds every channel's frames at its gain into a 32-bit buffer, then the mapped ring at its program's gain, and clips once; the master volume comes after, as before. A channel is freed by `sound_release` or the process's end. The volumes are remembered by name (`TRemember`, 24 names; `SD:/etc/mixer.ini` read at the first start) so a program finds its own again each time it plays. Before v85 one process owned the output and a second one was refused.
- **The output: `COnyxSoundDevice` (v84).** One producer, several outputs, one running at a time (`sys/sound.cpp`): the **jack** (`COutJack`, Circle's `CPWMSoundBaseDevice`: PWM + DMA, 44.1 kHz, the producer's chunks as they are), a **USB audio device** (`COutUSB`, `CUSBSoundBaseDevice`: a headset, a DAC; 48 kHz, 16-bit samples or 24-bit ones packed in three bytes) and **HDMI** (`COutHDMI`, `CHDMISoundBaseDevice`: the screen; 48 kHz, each sample framed for IEC958, 576-frame chunks). Each class overrides Circle's `GetChunk` — called from the output's completion interrupt — and takes the producer's frames (`SourceTake` / `SourceDone`). **The output adapts, the producer and the apps do not change**: the 48 kHz outputs pull through a rate converter (`sys/sound_resample.h`: linear interpolation, integer, a fraction `nFrac / 48000` that never drifts), in the device's sample format. Which output: `SD:/etc/sound.ini`'s `output = auto | jack | usb | hdmi` (read when the sound first starts, or when an app first asks), changed at once by `sound_output` (`OutputUpdate`: the running device cancelled, waited for, deleted, the new one made). `auto`: a USB audio device if there is one (Circle's `uaudio1-1`), else the jack, else HDMI on a board without a jack (the Pi 400). The sound is started on the first `sound_acquire` (not at boot), and runs whether or not an output could be started.
- **No output: the drain.** A USB device unplugged — or an output asked for that is not there, or that could not start — plays nothing, and **no other output takes over**: a kernel timer (every tick, `DrainTimer`) drops the producer's chunks at the rate they would have played, so the apps' streams go on as if they were heard. `SoundPoll` (every 100 ms, from the kernel's input task, beside the USB plug-and-play) makes the USB output again when a device is back — the same one or another —, without a restart. In `auto`, once a USB device was chosen it stays the output (unplugged: silence), until another output is asked for.
- **The volume.** The master volume (0..10, mute: v60) is one interface for the whole system; the output applies it — through the device's own control when it has one (`CSoundController`:

`ControlVolume` in dB below the control's maximum, the level's squared curve: a USB headset), else by the software gain (`s_Gain`, in `GetChunk`); the mute is always the software's. It is applied again at each change of output and when a USB device comes back.

- **Core 1 = the producer.** It sleeps in `WFE` until the device starts, then keeps `SND_AHEAD` (4) chunks rendered ahead in a ring; `GetChunk` only copies the next chunk (zeros if core 1 fell behind) and `SEV`s core 1. Without `ARM_ALLOW_MULTI_CORE` the same code renders in `GetChunk` itself (interrupt, core 0).
- **Rendering** (integer only, no FP in the kernel): the **PCM ring** (0.5 s of s16 stereo frames, `sound_write`) and the owner's **mapped ring** (v68), added and clipped. Nothing else: **the kernel only puts sound out**.
- **No synthesizer in the kernel (since 2026-10-05).** The 16 voices and the two-operator FM instruments (v46 / v47: `sound_start`, `sound_stop`, `sound_instrument`) were rendered here until then. They are AudioKit's now, in user space — `user/Kits/audiokit/fmsynth.h` (the same code, the same sound; its tables: `fm_tables.h`), exported as `ak_fm_*` by `SD:/lib/audiokit.so` (docs/03 §5.7) and played by the library's thread through `sound_write`. BASIC's `SOUND` / `PLAY`, the games' effects (`user/Apps/games/game.h`), `tone`, FM Tracker, Doom's music and the Sound applet's test all go through it. The three kapi slots stay in the table (it is append-only) and answer −1. The PC tools compile the same header: `tools/tests/run_fms_test.sh` renders instruments with it, and the Windows FM Song player (`tools/fmsplayer`) plays with it.
- **Low latency (v68, `sound_config`).** `CPWMSoundBaseDevice`'s chunk size is fixed when it is made — but it is only the size of its two DMA buffers: Circle's `CDMASoundBuffers` programs each transfer with the length `GetChunk` **returns**. So the device is made once, with the biggest chunk (1024 frames), and a smaller chunk is just a shorter return: nothing re-created, the PWM clock untouched, no Circle change. The owner picks the chunk (`s_nChunkFrames`, 64 .. 1024 frames) and how many chunks core 1 keeps ahead (`s_nAheadCfg`, 1 .. `SND_AHEAD` 4); each slot of the ahead ring records its own length (a change applies from the next chunk rendered — the ones already rendered play out first). What is heard lags the rendering by about **(ahead + 1) chunks** (the ahead ring + the DMA buffer queued behind the one playing): 1024 × 4 (the default, what the emulators use) ≈ 116 ms; 256 × 2 ≈ 17 ms; 128 × 2 ≈ 9 ms; 64 × 1 ≈ 3 ms (a DMA interrupt every 1.5 ms; core 1 must never be late: with 1 chunk ahead any hiccup is a click). Add what the app keeps queued in the stream or the ring. `GetChunk` now frees a slot only after copying it (it used to free it first: with 4 slots core 1 could render into the one being copied). `sound_release` / the owner's death restore 1024 × 4.
- **The mapped ring (v68, `sound_map`).** One 64 KB page (`SoundRing`: a 64 KB-aligned heap block, made on first use and **kept for ever** — core 1 may be reading it when its owner dies) holding `struct kapi_sound_ring`: a header (`wr` the app's, `rd` the kernel's — free-running frame counters —, `dry`, the rate, the chunk / ahead now) and 8192 s16 stereo frames (~186 ms). `kapi_sound_map` maps it into the owner like a shared surface (`CAddressSpace::MapSurface`, the surface arena; the teardown drops the mapping only), once per process. `Render` reads `wr` once per chunk, mixes up to a chunk of frames with the voices and the stream, then moves `rd` (`DMB` in between both sides); `dry` counts the times the ring ran out while playing. The kernel only ever indexes it masked by its own capacity and treats `wr - rd > 8192` as empty, so nothing an app writes there can make it read elsewhere. Since it is plain memory, **an app core fills it** (`kapi_sound_ring_write`), and a core-0 thread can sleep on `rd` with `wait_word` (the tick sees core 1 move it) — the DAW's engine writes straight into the kernel's ring, no pump thread in the audio path. A former owner keeps the page mapped in its space (the mapping is dropped only with the process), so it could still

write into it; the kernel stops mixing it at the release. `ringtest` plays a tone from an app core this way.

- **Ownership.** One pid owns the output (`sound_acquire`); every other call from another pid returns −1. `sound_release`, or the owner's exit (`SoundOnProcessGone`, called from `IpcOnProcessGone`), silences the voices, empties the ring and frees the output. The state is shared between core 0 (kapi calls) and core 1 (rendering) under a `CSpinLock`.
- **kapi v46**: `sound_acquire`, `sound_release`, `sound_write (frames, n)`, `sound_status` (`sound_start` / `sound_stop`, and v47's `sound_instrument`: retired on 2026-10-05, see above). Users: `/bin/tone`, BASIC `PLAY` / `SOUND` / `BEEP` / `NOTEON` / `NOTEOFF`.
- **kapi v68**: `sound_config (chunk_frames, ahead)`, `sound_map ()` (above).

13. Post-mortem debug console

Source: `kernel/sys/debugcon.cpp`, `kernel/include/kern/debugcon.h`.

Purpose: when an app terminates (or in case of trouble), make the logger's last messages visible **directly on the framebuffer**.

- **`CFbConsole`**: text console that renders the characters (green on black) on the back buffer of `C2DGraphics`, with line wrapping, scrolling, and `UpdateDisplay()` after each write.
- **`CLogSwitch`**: routes the logger's output either to the boot console (`m_Screen`), or to the framebuffer console. At boot: `SetNormal(&m_Screen)` + `DebugConsoleRegister`.
- Once the compositor is started, `MuteNormal()` stops the writes to the boot console: it is no longer shown, and each line drawn there (a text-mode scroll of the whole screen inside `CTerminalDevice::Write`, IRQs masked) froze everything for ~170 ms — an app that `printf`s without a terminal (Doom at start-up) froze the GUI and the network for seconds. `kmsg` still sees every line (CLogger's event ring does not depend on the target).
- When the debug console takes over (`DebugConsoleTakeover`, called on `exit`), the compositor **detects** `DebugConsoleActive()` and **stops presenting** so as not to contend for the framebuffer.

The crash record (`lastcrash.txt`)

Source: `kernel/sys/crashlog.cpp`, `kern/crashlog.h`.

A frozen Pi (the scheduler stopped: the green LED stays lit or dark, no SOS) left nothing to read. Now 64 KB of RAM kept out of the heap (Circle fork, docs/05 §15) hold, for the running session, cleaned to RAM as they are written:

- the **tail of the kernel log** (`CLogSwitch::Write` copies every line, ~62 KB ring);
- the **last 16 places core 0 was interrupted at** (`KernelIRQExit`, each IRQ): PC, LR, SP_EL0, whether IRQs were masked, the task's name — a kernel loop shows itself there;
- **breadcrumbs**: the GPU (`idle / clipping / binning / rendering / texture upload`) and the compositor's display copy (`composing / display DMA / waiting for the display DMA`);
- the reaper's **last pass** (uptime), and a **panic**'s registers (`DumpAndHalt`, before the panic screen) with the **return addresses found on the faulting context's stack** (`CrashLogStack`);
- the **free memory**, once a second (`CrashLogMemory`, breadcrumbs 5–7): the kernel heap, the kernel pages (low pager), the app pages (the high zone) — an "Out of memory" shows its approach;
- the **app the GUI watchdog finds frozen** (below): its pid, the last 16 places its task was interrupted at, and its stack's return addresses.

Return addresses. The apps are built `-O2` (no frame pointers to follow): `ScanStack` reads 16 KB up from the context's SP and keeps the words that point just after a `BL` / `BLR`, in an app's code (`[8 GB, 10 GB)`) or the kernel's (`[0x80000, _etext)`) — 24 at most, newest first. Each page is probed first with `AT S1E1R` (a wild SP, an unmapped page: no fault). Read them with `aarch64-none-elf-addr2line -f -C -e user/<app>.elf <addr>` (or `kernel/kernel8-rpi4.elf`); a few are stale words, the first ones are the calls in progress.

The **BCM watchdog** is armed by the reaper (every second, `cmdline.txt hangreboot=` seconds, default 15, at most ~15; 0 = off): when the scheduler stops, the Pi restarts by itself (a panic too, after its screen and SOS). `kapi_shutdown` / `kapi_reboot` stop it and mark the session clean. At the next boot, a session that did not end cleanly (its record still in RAM: a power cut loses it) is written to **`SD:/etc/lastcrash.txt`** and noted in `kmsg` (`crashlog`). Best effort: nothing if the RAM did not keep its contents through the reset.

Core 1 writes the report itself (the RAM does not survive the Pi 4's watchdog reset: the first real freeze left no record). At boot (`CrashLogReport`, SD: mounted) the kernel keeps the previous dump if `SD:/etc/crashdump.txt` holds one (`ONYX CRASH REPORT` … `--- end of the report ---` → `SD:/etc/lastcrash.txt`), then fills that 64 KB file with "armed" and notes each of its 128 sectors' LBA (through FatFs: `f_lseek` + a one-byte `f_read` → `FIL.sect`). Core 1, the sound core (`SoundCoreMain`), is woken at least every ~1.2 ms by the generic timer's event stream (`CNTKCTL_EL1.EVNTEN`, `EVNTI` 15) and calls `CrashLogCoreCheck`: when the reaper's `CNTPCT` stamp has not moved for **10 s**, it formats the record (no heap, no logger), writes it **raw** through the `emmc1` device into those sectors (runs of consecutive sectors in one write; `g_bCrashDumping` keeps `OnyxDriverWait` from yielding) and restarts the Pi (`CBcmWatchdog::Restart`). No FatFs, heap, logger or scheduler call: core 0 may hold their locks; core 1 takes the sound lock only with `TryAcquire`, so a core 0 frozen while holding it does not stop the watch. Its progress is kept in the RAM record (`nDumpStep`: started, text ready, written, the write failed): when the SD write cannot finish (a wedged bus), the hardware watchdog restarts the Pi and the RAM report (the record of an 8 GB Pi sits at the top of the RAM above 4 GB) says how far core 1 got. `CrashLogRequest` asks for the same report from a task (core 0 masks its IRQs and waits, core 1 writes it, then the restart); the GUI watchdog task used it for a compositor without a frame for 12 s and no longer does (2026-10-04): the compositor is paused under a full-screen app, so a slide show left on one slide, or a full-screen game, restarted the Pi. It now only warns in the log (`compositor STALLED`), and the paused compositor counts as alive (`CWindowManager::CompositorAlive`). **A Circle panic** (an assertion, the kernel heap's "Out of memory", ...) used to leave nothing: Circle's logger halts **every** core after the panic line (`CMultiCoreSupport::HaltAll`), core 1 too, so the Pi stayed frozen ~15 s and the hardware watchdog restarted it without a report. The logger's panic handler is now `CrashLogPanicHandler`: the log's last line (the panic's) goes into the record and core 1 writes the report at once (`CrashLogDumpNow`), before the halt. An exception (`DumpAndHalt`) asks for it at once too, after its screen and one SOS (it used to wait the 10 s of a stopped reaper).

The app watchdog (`SD:/etc/apphang.txt`). When the GUI watchdog finds an app not taking its window's events (2 s, some queued), it watches that app (`CrashLogWatchPid`: core 0's IRQs note where its task is and, every 8th time, scan its stack) and, every 2 s while it lasts, has the `apphang` task (its own: never the GUI watchdog stuck on the card) rewrite `SD:/etc/apphang.txt`: the app, its tasks' states (running / ready / blocked / sleeping) and pages, then the whole record (the free memory, core 0's samples, the app's samples and stack) and the log's last 12 KB. Recovered (the app takes its events again, or is closed / killed): renamed `SD:/etc/lasthang.txt`; a clean shutdown / reboot renames it too. Still there at the next boot (the session never ended cleanly — the freeze grew into a restart): appended to `SD:/etc/lastcrash.txt` (`MergeAppHang`), or written as it when the kernel kept no record of its own. **Power and heat.** Once a second the GUI watchdog task calls `CrashLogPower`: the firmware's `GET_THROTTLED` bits

(under-voltage, ARM frequency capped, throttled, soft temperature limit — now, or since boot) and the SoC temperature. A change (or each 5 °C above 60 °C) is logged (`power: SoC 71 C, throttling: under-voltage (earlier)`), and the last values plus the session's highest temperature go into the crash record (breadcrumbs 2–4), printed in every report — a freeze that only a cold boot brings back smells of the supply or the heat. **The Pi 5's fan** (2026-10-09, `#if RASPPI >= 5`): with `gpiofanpin=45` in `cmdline.txt` (the Pi 5's card has it) the same task calls `CCPUThrottle::Update ()` (Circle: every 4 s at most), which switches the Active Cooler's fan on above `socmaxtemp` (default 60 °C) and off 5 °C below; without a fan it is not called (it would lower the clock instead, and Onyx keeps it at its maximum).

The clock across boots. The Pi has no battery-backed clock: until NTP answers (~15 s), the time was 0 and the files written meanwhile (`crashdump.txt`, `lastcrash.txt`) had no date. `SD:/etc/clock` keeps the last time seen (UTC seconds, text; written every 10 minutes by the GUI watchdog task and by `kapi_shutdown` / `kapi_reboot`) and is read back at boot, right after SD: is mounted (`CrashLogClockRestore`) — behind by how long the Pi was off, until NTP corrects it. **The Pi 5 has one** (in its PMIC, kept by the optional RTC battery): read first at boot (the firmware's `GET_RTC_REG`, as Circle's `addon/rtc/firmwarertc.cpp`; a time before 2023 — no battery, the board was off — is ignored and the file taken), written with the file (`SET_RTC_REG`).

The Pi 5's power button (`#if RASPPI >= 5`): the kernel's main task polls it every 100 ms (`is_power_button_pressed`, Circle, debounced) and starts the end-of-session dialog (`SD:/apps/shutdown.app`, a press each 5 s at most). Its Shut Down is `kapi_shutdown (0)`, which on the Pi 5 calls `poweroff ()` (PSCI `SYSTEM_OFF`) instead of halting: the board is off and a press on the button starts it again (the EEPROM's `POWER_OFF_ON_HALT=1` also cuts its 3.3 V rail).

Headless signs on the green ACT LED (a GPIO set / clear, no lock): **3 s of fast blinks** when core 1 sees the hang, the SD write's own flicker, then **3 s lit** (written) or **3 slow blinks** (the write failed) before the restart; no blinking at all = core 1 stuck too (a wedged bus). `hangreboot=0` turns it off with the watchdog. `kmsg` tells at boot where the record is and whether the dump is armed (`crashlog: ...`). Test: none from an app any more (an app at EL0 can neither mask the IRQs nor stop the scheduler).

The panic screen is now copied into the displayed frame buffer **by the CPU** (not the DMA: the compositor's display DMA may be in flight, and waiting for it hung the panic before its screen and its SOS).

---

14. App cores (cores 2 and 3)

Source: `kernel/sys/appcore.cpp`, `kern/appcore.h`; user side `user/Emulators/emucore.h`, test `user/BinUtils/coretest.c`.

Cores 2 and 3 (core 2 only with `netcore=1`: core 3 then runs the network, §11) are a **resource an app acquires**, like the sound output: it gets a whole core and runs one function of its own code there, undisturbed (no scheduler, no timer, no other task on that core). The rest of the system stays on core 0 as before, so none of the kernel, Circle's drivers, FatFs or the network has to be multi-core safe.

- **The core's loop** (`AppCoreMain` → `AppCoreLoop`, from `COnyxCores::Run`): it installs **our** `KVectorTable` on that core (`VBAR_EL1` is per core), notes its kernel stack, enables IRQs (for the stop IPI only: the GIC routes no peripheral there) and waits in `WFE`. A job: load the owner's `TTBR0` (its L2 table + ASID, `CAddressSpace::GetTTBR0`), `tlbi vmalle1` on that core only (an ASID may have been reused since its last job), then `El0Enter (fn, stack, arg, El0CoreReturn)` — at **EL0**,

`SP_EL0` = the app's stack, like the app itself on core 0; the job's end is an `svc` from `El0CoreReturn` (→ `AppCoreEl0Done`), any other `svc` or a fault stops the job (`KAPI_CORE_FAULT`). When `fn` returns: back to the kernel address space, state `IDLE`, `SEV`.

- **The kapi side (core 0).** `core_acquire` hands a free, started core to the caller's address space; `core_run` checks the owner, that nothing runs, that `fn` and the stack are user addresses, fills the job, then `bGo` + `DSB` + `SEV`. Every word shared between the cores is plain cacheable memory (inner-shareable, coherent) with `DSB ISH` barriers; only core 0 writes the ownership.
- **Stopping a job** (`core_release`, the app's exit or kill): core 0 raises `bAbort` and sends the core an IPI (`SendIPI`, `IPI_USER`). The IRQ interrupts the job at EL0 and enters `El0IrqEntry` on that core (`IrqEntry` if the core was in its own kernel code); the exit path (`El0IrqExit` / `KernelIRQExit`) sees it is not core 0 (no scheduling there) and calls `AppCoreOnIRQExit`, which **rewrites the trap frame** to return into `AppCoreRestart` on the core's own kernel stack (EL1t, IRQs masked): the job is simply dropped. `AppCoreRestart` goes back to the kernel address space and clears `bAbort` — core 0's signal that the core is out of the app's memory. Core 0 waits for it at most 200 ms.
- **Faults.** A synchronous exception from the job at EL0 (a bad access, an undefined instruction, any `svc` but the job's end) reaches `El0SyncHandler`, which hands it to `AppCoreOnEl0Sync` → `AppCoreOnFault` (instead of killing a process): the ESR class, PC and fault address are kept, the state becomes `FAULT` and the frame is rewritten to `AppCoreRestart` as above. Core 0 logs it once (`appcore: core N: fault EC=... at pc ... (address ...)`) when the owner asks the state or releases the core.
- **Memory (v75, §4 *Demand paging*).** No allocator runs on cores 2–3, so what a job touches is filled beforehand: `core_acquire` fills the owner's heap and makes it **eager** (filled as `sbrk` grows it — the emulators `malloc` on the main thread and touch on the core), `core_run` fills the top 256 KB of the job's stack. Anything else still unfilled (a `vm_map`'ed buffer, deeper in a lazy stack) takes the slow path: the job's translation fault becomes a **page-in request** in its `TAppCore` (`nPageIn = 1`, the address, write or not; `DSB`, `SEV`) and the core waits in `WFE` with its IRQs on (a stop still drops the job). On core 0 an `IoWait` tick hook wakes the **`vmpager`** task (made at the first `core_acquire`), which fills that page and up to 7 more of the region by a software walk of the owner's tables, answers 2 (retry: the `eret` repeats the access, no TLBI needed) or 3 (no region, or out of memory: `FAULT`) and `SEV`s — up to one 10 ms tick per request. Frames leaving or protections lowered on core 0 reach the job's TLB by the inner-shareable `TLBI`.
- **Instruction cache (2026-10-10).** A job's start empties the core's own instruction cache (`ic iallu`, beside the TLB's flush): the loader's `SyncDataAndInstructionCache` reaches core 0 only, and a program loaded again -- a new build of it -- gets the frames the last one had, so the core would run the old instructions wherever it still held them.
- **TLS (v75).** `core_run` reads the caller's `TPIDR_EL0` (the kernel never changes it) and the core writes it before `El0Enter`: a job shares its caller's thread-local data (errno included).
- **Teardown.** `~CAddressSpace` calls `AppCoreReleaseAS (this)` **before** freeing the window and the frames: a job still running uses them. A core that does not answer the stop (its code masked the interrupts — never do that) is **retired** (`bLost`, never used again) and the dying space's memory is kept (leaked) rather than freed under it.
- **The rules for `fn`**: no kapi call (the kernel is not called from two cores at once), no allocation (newlib's and umm's `malloc` are not multi-core safe), no masking of the interrupts. It computes, reads the clock directly (`cntpct_el0`) and exchanges data with the app's main thread through memory, with barriers. It may use `WFE`, woken by the main thread's `SEV`.
- **`user/Emulators/emucore.h`** packages that for the emulators: the machine runs on the app core (`ec_thread`), the main thread asks for frames (`ec_request`), gets the pictures from a **triple buffer**

(`ec_publish` / `ec_take`: the machine never waits for the display and the display always gets the latest complete picture) and the sound from a single-producer ring (`ec_audio_push` / `ec_audio_pop`), and stops the machine between two frames to touch it (`ec_hold` / `ec_resume`: reset, palettes, battery saves). Without a free core, `ec_pump` runs the same frames on the main thread. `gbemu`, `gbaemu`, `nesemu` and `snesemu` use it.

- **Test**: `/bin/coretest` (the same computation on an app core and on core 0, a job stopped by its flag, an endless job stopped by `core_release`, a faulting job, both app cores at once); `coretest exit` leaves a job spinning and exits (the teardown must stop it). `/bin/memtest` (v75): a job writing 4 MB of unfilled memory (the pager, timed), its TLS.
- **Bigger programs on an app core**: newlib's syscalls can run on the main thread when called from an app core (`libc/onyx_syscalls.c`, `onyx_rpc_*`: the caller posts the call and waits in `WFE`, the main thread runs it in `onyx_rpc_serve` and `SEV`s). Doom's engine runs that way (`user/Ports/doom`), with its malloc, files and saves.

15. The GPU (V3D)

> **The Raspberry Pi 5 (V3D 7.1)**: §15.1 below — the same kapi, another GPU underneath.

Source: `kernel/sys/v3d.cpp`, the packets `kern/v3d_cl.h`, the shaders `sys/v3d_shaders.qasm`, the texture layouts `kern/v3d_tiling.h`; demos `user/Apps/teapot` (v52), `user/Apps/gpudemo` (v53), `/bin/gpcdemo` (the compositing service over v53 + v70).

The Raspberry Pi 4's VideoCore VI 3D core (**V3D 4.2**) driven from the kernel, bare metal — after Random06457's *rpi4-gpu-bare-metal-examples* (MIT: the packet structs, the three shaders, the control-list recipe) and macoy's `rpi-system` notes (cache cleaning, the binner's memory).

- **Bring-up** (the first `gpu_info` / `gpu_draw`, never at boot): the firmware turns the V3D power domain (10) and clock (5) on and the clock to its maximum (mailbox); the reset is released (`PM_GRAFX` `V3DRSTN`, with the PM password) and the V3D's async AXI bridges opened (`ASB_V3D_M_CTRL` / `_S_CTRL` at `0xFEC11008/C`, waiting for the ACK bit with a time limit); then `HUB_IDENT1` must say version 4 (kmsg: `HUB_IDENT0..2`, `CTL_IDENT0`). Any failure leaves the GPU off; `gpu_info` says why. The interrupt is connected there too (see below).
- **Memory**: no V3D MMU — the GPU gets physical addresses, all its buffers come from Circle's low heap (`HEAP_LOW`, < 1 GB, page aligned): the control lists, the shaders and default attributes, the vertices, the render target (raster RGBA8), the tile allocation (the tiles × 64 B + 2 MB), the tile state (256 B a tile) and a 4 MB overflow pool. CPU caches: cleaned after the CPU writes (the lists, the vertices), the target cleaned and invalidated before the GPU writes it and invalidated before the CPU reads it; the GPU's L2T and slice caches are flushed before each job.
- **A frame** (`gpu_draw`): the vertices are copied into the GPU buffer; the **binning list** (`TILE_BINNING_MODE_CFG`, the clip window, `CFG_BITS` with the depth test *less* and depth writes, the viewport scaling and offset, `GL_SHADER_STATE` → a shader record + two attribute records over the same 16-byte vertices — position 3 floats, colour 4 normalized bytes —, `VERTEX_ARRAY_PRIMS` triangles) sorts the triangles into 64 × 64 tile lists; the **rendering list** configures the tile buffer (one RGBA8 target, Z cleared to 1.0), clears, then for each tile runs its list, stores the colours (`STORE_TILE_BUFFER_GENERAL`, raster) and clears again. Jobs: `CT0QMA/QMS/QTS/QBA/QEA` (bin), `CT1QBA/QEA` (render); the end is polled on the frame counters `BFC` / `RFC` (the caller's task yields meanwhile; one frame at a time), 0.5 s at most — past that the state of the control-list executors is logged and the GPU left off. The binner's out-of-memory requests (`INT_STS` `OUTOMEM`) are

answered from the overflow pool (`PTB_BPOA` / `BPOS`). The RGBA8 picture is copied into the caller's pixels as 0x00RRGGBB.

- **Shaders**: fixed — a vertex shader (position → screen, colour → varyings), its coordinate shader for the binner, a fragment shader writing the interpolated colour: Mesa-compiled QPU code taken from Random06457's example. The geometry, the lighting and the projection are the app's (the CPU); the GPU does the rasterization, the interpolation and the depth test.
- **The full pipeline** (kapi v53, `gpu_render`): the same frame recipe with **batches**. Each batch has its own shader record (the attributes: position x y z w — 4 floats —, s t, colour; s t, colour, added colour; 32-byte vertices), its uniforms (the batch's 4 × 4 matrix, then the viewport scales), its `CFG_BITS` (depth function and writes, culling: `clockwise_primitives` set, front = counter-clockwise in y-up coordinates, as GL), its blending (`BLEND_ENABLES` + `BLEND_CFG`: alpha `SRC_ALPHA / INV_SRC_ALPHA`, add `SRC_ALPHA / ONE`, multiply `DST_COLOR / ZERO`, premultiplied `ONE / INV_SRC_ALPHA`) and one `VERTEX_ARRAY_PRIMS` over its range of the vertex buffer. `KAPI_GPU_F_KEEP` loads each tile from the target first (`LOAD_TILE_BUFFER_GENERAL`, raster RGBA8, alpha forced to 1) instead of clearing it. The target's load and store packets are built in one place (`V3dLoadTarget` / `V3dStoreTarget`, `kern/v3d_cl.h`). **Fixed (2026-09-30):** the load packet's packed bit-fields lacked 3 bits of padding after `r_b_swap`, so its stride landed at bit 25 instead of 28 and the V3D read it divided by 8 — every frame drawn over the target's pixels (the compositing service without a clear; `gpcdemo test`'s "over the target's pixels" failed on the Pi) loaded the wrong rows. `tools/tests/run_v3d_cl_test.sh` now checks each field of the load / store packets against Mesa's `v3d_packet.xml` positions, and the PC's software V3D (`tools/tests/gpucomp/hostkapi.cpp`) decodes the kernel's own packets for the target's stride (it re-implemented the frame and could not see the packets; `gpcdemo test` also drew over a single colour, which hid wrong rows — it draws over a pattern now).
- **Shaders of v53** (hand-written: `sys/v3d_shaders.qasm` → `v3d_shaders.inc`, assembled by `tools/qpu/qpuasm`, see below): `VS_CLIP` — the 10 inputs, the matrix product (16 `ldunif`), 1/w on the SFU (`recip`, r4), Xs Ys in 24.8 fixed point, Zs, 1/Wc, then the 10 varyings s t r g b a r2 g2 b2 a2 (v54); `CS_CLIP` — its binner copy (clip X Y Z W, then Xs Ys); `FS_COLOR` — the interpolated colour (varying = `ldvary` × W + C, C in r5 two instructions later); `FS_TEX` — the texture lookup (T to `tmut`, the TMU configuration p0 / p1 by `wrtmuc`, S to `tmus` starts it, as Mesa orders them; a thread switch while it runs; `ldtmu` returns R G and B A as half floats) multiplied by the colour; both then add the second colour and clamp to 1 (v54). `FS_COLOR_AT` / `FS_TEX_AT` (v54, `B_ALPHATEST`) read the threshold as one more uniform and discard the fragment (`fsub.pushn`, then `setmsf.ifa -, 0`) when alpha is below it. The fragment shaders end with the TLB writes (`vfpack tlb`) after the last-segment thread switch.
- **App shaders** (kapi v61, `gpu_program` / `gpu_render2`): the app brings its QPU code (built at run time with the C++ builder `user/Libs/v3d/qpu.h` over Mesa's instruction packer, `tools/qpu/mesa`; tested on the PC by the simulator `tools/qpu/qpusim` — `tools/tests/run_qpu_test.sh`). A program (`TProgram`, ≤ 256, owned by its process) holds the three shaders in low GPU memory, each at a 256-byte boundary and followed by NOPs (the QPU fetches ahead). Each batch of `gpu_render2` gets its shader record from the program: the VPM segment sizes (⌈inputs / 8⌉, ⌈(4 + varyings) / 8⌉, the coordinate shader's ⌈csInputs / 8⌉ and ⌈csOutputs / 8⌉), the fragment shader's threading (`P_FS_4WAY`, `P_FS_FINAL`) and Z writes; one float vec4 attribute per 4 inputs (≤ 16 arrays: the `GL_SHADER_STATE` packet is written by hand, its count has 5 bits), its uniforms copied from the call's array — the samplers first (32-byte aligned), then the vertex, coordinate and fragment ranges, the texture words patched in the latter —, then `CFG_BITS`, `BLEND_CFG` (the batch's own factors and equations), `COLOR_WRITE_MASKS` and `CLIP_WINDOW` (the scissor) when they change. The

kernel does not check what the shaders do: a program that never ends is stopped by the frame's time limit (the GPU is then left off), and the TMU's general writes could reach any low memory — apps are trusted, as with `code_alloc`. **`P_FS_FINAL` is best avoided**: a fragment shader started in its final thread section (no last-segment pair) that reads uniforms stopped the GPU now and then on targets 512 pixels wide or more (5 runs out of 7, mostly the first frame after boot; 64 × 64 frames always passed; `v3dprog ww`, docs/03). Mesa never uses it for fragment shaders; neither do `user/Libs/v3d/gxtev` nor gcemu since: they end with the last-segment pair (two `thrsw` in a row, the live values in registers of the register file — the accumulators do not survive a thread switch) before the TLB writes.

- **`gpu_vbuf` / `gpu_render3` in place** (v63, `ClipInPlace`): a vbuf is a `HEAP_LOW` block (64 KB aligned, below 1 GB: its address is the GPU's) mapped into the program by `MapSurface` (`s_VBuf`, freed by `V3DReleaseAS`). When `gpu_render3`'s array lies in one of the caller's, each batch's vertices are framed where they are and tested; the triangles inside every plane make runs `{first, count}` of the batch's own vertices (`s_pRun`, `s_pRunAt`: `Render2` draws each run with the batch's shader record, its attributes at the batch's address and stride), the others are clipped into the buffer's free end, each fan at a whole number of the batch's strides from it (a run too); then the touched range's cache is cleaned. The copying paths (`ClipFrame2` / `3`) make one run a batch.
- **`gpu_render3`** (`ClipFrame3`): each batch's triangles are read at its offset and stride in the caller's array, framed (x, y) into the GPU's vertex buffer at that stride — kept as they are when inside every plane, else clipped in their place — so the output is each batch's own packed run (`s_pOutOff`, `s_pOutStride`); `Render2` gives each batch its attribute records at that address and stride, and draws from its vertex 0 (`gpu_render2`'s batches too, at the common stride). The Wind Waker's display no longer copies its ~7 MB of vertices a frame before the call (gcemu's `Out::prepare` 5.9 -> 0.1 ms): the machine's core, sharing the L2 and the RAM, 33.5 -> 29.9 M cycles a field.
- **`gpu_render2`'s clipping** (`ClipFrame2`): a triangle whose three vertices are inside every plane (`V3DInsideN`: the near plane, w > 0, the guard band — nearly all of them) is copied once, its program's inputs only, straight into the GPU's vertex buffer (`s_Verts2`, sized for the clipped fans); the others go through `V3DClipTriangleN` (every float interpolated) into the same buffer. Before, each vertex was copied four times (staged, clipped, into a buffer, into the GPU's) — ~30 MB a frame for The Wind Waker's ~100k vertices through the L2 the app cores share: its `gpu_render2` went from ~34 to ~25 ms a frame. Every 256 frames kmsg gets `render2, a frame: clip … us, lists … us, GPU … us, target … us; … vertices` (the clipping and its copy; the lists, the uniforms and the target's cache cleaning; the binning and rendering; the target after) — Wind Waker's game: ~9.8 ms, ~0.9, ~14, ~0.2 for ~80k vertices drawn.
- **Textures** (`gpu_texture`, ≤ 1024 — 512 a program — since v70, owned by the program that made them, freed with its address space): a 32-byte `TEXTURE_SHADER_STATE` (base address, size, RGBA8, swizzle, and for UIF: *level 0 strictly UIF*, its `UB_PAD`, XOR) then the texels, in the layout the texture unit reads for that size (Mesa's `v3d_setup_slices`, `kern/v3d_tiling.h`): **LT** (a line of 4 × 4-texel utiles, a side ≤ 4), **UBLINEAR** 1 / 2 columns (8 × 8 blocks of 2 × 2 utiles, width ≤ 8 / 16), else **UIF** (columns of 4 blocks, padded rows against the page-cache banks, XOR on odd columns when aligned on them). A batch's `SAMPLER_STATE` (filter, wrap S / T) is written per frame; the fragment uniforms are p0 = the texture state | 3 (two 16-bit words returned) and p1 = the sampler state. Host test (every texel of every size up to 300 × 300 against Mesa's functions): `sh tools/tests/run_v3d_tiling_test.sh`.
- **The QPU assembler** `tools/qpu/qpuasm` (host, `cd tools/qpu && make`): Mesa's QPU encoder / disassembler (`qpu_instr.c`, `qpu_pack.c`, `qpu_disasm.c`, MIT, vendored in `tools/qpu/mesa/`)

behind a parser of the disassembler's own syntax; a line is accepted only if its encoding disassembles back to it, then each shader is checked against the instruction restrictions (Mesa's `qpu_validate.c` and its scheduler's timing rules: r4 ≥ 3 instructions after an SFU write, r5 ≥ 2 after `ldvary`, one TMU / SFU / VPM / TLB access a line, no regfile read right after its write, the thread-switch rules, no TLB access before the scoreboard wait). `qpuasm -d HEX…` disassembles. It re-assembles the three v52 shaders bit for bit.

- **Straight into the target** (v52 and v53): the caller's pixel pointer is translated page by page (`AT S1E1R` → `PAR_EL1`, IRQs off); when the whole span (`(h − 1) × stride + w` pixels) is physically contiguous below 1 GB — a window canvas is — the tile stores (and the `KEEP` loads) go straight there: raster RGBA8 with **R/B swap** (= 0x00RRGGBB in memory), the row pitch = the caller's stride, alpha not written (`COLOR_WRITE_MASKS` 0x8) and cleared to 0; the span is cleaned + invalidated from the CPU caches before and after. Other buffers go through `s_Target` and a copy, as before. kmsg says which mode is used (when it changes).
- **Completion by interrupt**: the V3D core interrupt (GIC SPI 74, shared by core and hub in the Pi 4's device tree; the hub's are masked) is connected at bring-up with `FLDONE` (bin done), `FRDONE` (render done) and `OUTOMEM` unmasked. The handler answers the binner's memory requests from the overflow pool (and masks `OUTOMEM` once the pool is used up) and sets the bin / render events; the drawing task sleeps on them (`WaitWithTimeout`, 2 ms slices, the frame counters `BFC` / `RFC` re-checked each time, 0.5 s at most). Without an interrupt it degrades to that 2 ms polling (logged once) and serves the binner itself.
- **Sharing the GPU between programs** (the GameCube emulator and the browser at once, say). The kernel is not preempted (§5) and every GPU call runs on core 0, so the `while (s_bBusy) Yield (); s_bBusy = TRUE;` that starts each `gpu_texture` / `gpu_texture_rect` / `gpu_program` / `gpu_render*` is atomic: **one GPU job at a time, whole** — a frame's lists, its binning and rendering, its target's copy — and another program's call waits (it yields) until that frame is done. Nothing of a frame survives it: the control lists, the vertex buffers, `s_Target`, the tile memory are rebuilt by each frame (their memory only grows). What a program owns is marked with its address space (textures, programs, vbufs), checked by every call, freed when it ends (`V3DReleaseAS`, after the frame in flight). So two programs can use the GPU together; what they compete for is the pools of handles: until v70 one program could hold all 256 textures (gcemu and n64emu keep up to 256) or all 8 vbufs, and the other got −4 / 0 — **v70**: 1024 textures, 512 a program; 32 vbufs, 8 a program. Still shared: the GPU's time (frames are served in the order the scheduler runs the callers: a 14 ms Wind Waker frame delays a composite by up to that much), and its state — a frame that hangs turns the GPU off for everybody until a reboot (`s_nState = −1`; the compositing service then goes on with the CPU, `GPC_LOST`). The GPU's low memory (below 1 GB, `HEAP_LOW`) is shared with the kernel's other users.
- **The compositing service** (`user/Libs/gpucomp/gpucomp.{h,c}`, a user library over v53 + v70; docs/03 *GPU compositing*): an app's layers — ARGB premultiplied textures of any size (past 2048 a side kept as tiles of 2048 with a one-texel border of their neighbours, so that bilinear filtering is seamless) — composited into a target (the window's canvas: the direct mode, no copy) with, per layer, a 2D affine matrix, a clip rectangle, an opacity, a blend mode (source-over; v72: multiply, screen, add, subtract, lighten, mask, cut out — multiply and subtract are two passes, `MULCOL` or `RSUB` then `UNDER`, so that they are exact over a transparent target too) and bilinear (or nearest) filtering; scrolling = the layer's source rectangle moved over its texture. A composite is one `gpu_render` frame (a target past 2048 a side: equal parts, one frame each): per layer and tile, the tile's inner rectangle (within the source rectangle) mapped into the target, clipped there (Sutherland-Hodgman against the clip rectangle and the target: the texture coordinates are

affine, interpolated exactly) and drawn as a fan — the kernel's `FS_TEX` (texel × colour, the colour = the opacity), blend `PREMUL` (`ONE, ONE_MINUS_SRC_ALPHA`, or none for an opaque layer at full opacity), `Z_ALWAYS` + `NOZWRITE`, clamped, `NOMATRIX` clip-space vertices on the kernel's viewport (`X = x ⌊w / 2⌋ + ⌊w / 2⌋`: the halves are integers in the binning list's scaling). Zero-copy textures are not possible: V3D 4.2's `TEXTURE_SHADER_STATE` has no raster layout for 2D images — the texture unit derives LT / UBLINEAR / UIF from the size (§ *Textures* above) —, so an upload always stores texels into the tiled layout (hence `gpu_texture_rect` for the damaged parts only); what is zero-copy is the target (the canvas, or a `gpu_vbuf` block). The same API is implemented on the CPU (the fallback: no GPU, an older kernel, the GPU stopped; a texture the GPU refused is drawn by the CPU between two GPU frames that keep the target), checked against the GPU path on the PC (`tools/tests/run_gpucomp_test.sh`: a software V3D runs this `FS_TEX` in the QPU simulator) and on the Pi (`/bin/gpcdemo test`).

- **Not yet**: several frames in flight (the caller waits for its frame), mipmaps, shaders chosen by the app (v61 has them), antialiased polygon edges (the compositor's rotated layers have hard edges: MSAA 4x in the tile buffer would give them).

15.1 The Raspberry Pi 5: V3D 7.1 (VideoCore VII)

The Pi 5's kernel (`make BOARD=pi5`, `RASPPI = 5`) drives its V3D 7.1 with the same kapi and the same frames; what differs is under `#if RASPPI >= 5` in `kernel/sys/v3d.cpp` (the Pi 4's code is the same machine code as before, checked object by object). **Built and tested on the PC, not run on a Pi 5 yet** (2026-10-09; docs/PI5-PORT.md §9, its test ladder).

- **Bring-up**: the registers at `0x10_0200_0000` (hub) and `+0x8000` (core 0); the firmware's clock 5 on and at its maximum, the reset released (`PM_GRAFX_2712` = `ARM_PM_BASE + 0x304`, `V3DRSTN`, the clock off around it — Linux's `bcm2835-power.c`), **no ASB**, then the **SMS** woken (`TEE_CS ← CLEAR_POWER_OFF`, state idle) and reset (`REE_CS ← 4`, out of isolating / resetting); `HUB_IDENT1` must say version 7. The core's interrupt is GIC SPI 249 (the hub's, 250, unused); the interrupt bits used (FRDONE, FLDONE, OUTOMEM) and the CLE / PTB / L2T registers are where they were. No V3D MMU, as on the Pi 4 (physical addresses, buffers below 1 GB). `cmdline.txt gpu71=0` leaves the GPU off.
- **The packets**: built from Mesa's own description — `tools/v3d/genpackets.py` turns Mesa's `v3d_packet.xml` (`tools/v3d/v3d_packet.xml`) into `kern/v3d_pack71.h` (and `v3d_pack42.h`, whose bytes `tools/tests/v3d/pack_test.cpp` checks against the hand-written `kern/v3d_cl.h` that runs on the Pi 4: 74 006 checks). Nine of the packets the kernel uses change: the binning and rendering configurations (the tile size explicit, `log2` 3 = 64 pixels; no maximum bpp), the colour configuration and the clear colours replaced by **`Render Target Part1`** (RGBA8 = type-and-clamp 8, 32 bpp, a stride of 32 × 128 bits, at 0 in the tile buffer, the clear colour), `Clear Tile Buffers` by **`Clear Render Targets`** (the depth cleared early: `early_depth_stencil_clear`), `Cfg Bits` (no early-Z bits), `Blend Cfg` (an 8-bit mask), the **clipper's XY scaling in 1/64 pixel** (1/256 on 4.2 — and the vertex shaders' x / y scale uniforms the same), and the **shader state record** (no default attributes' address: 32 bytes, every later field 32 bits earlier; `Rec71` makes it from the 4.2 record the draws fill — the vertex and coordinate shaders 2-way). The loads, stores, tiles, attributes, texture and sampler states are the same bits.
- **The kernel's shaders**: `kernel/sys/v3d_shaders71.qasm` (→ `.inc`, `tools/qpu/qpuasm`'s `.version 71`), the same six programs as the 4.2 ones for the 7.1's QPU — no accumulators, `ldunif` / `ldvary`'s C term in rf0, W in rf3, `recip` an add operation, no `vpmwt`; the fragment shaders in

rf0..rf15 (4-way). They give the same results as the 4.2 ones, bit for bit, in the simulator (`tools/tests/v3d/kshaders_test.cpp`). `gpu_draw` (v52, whose shaders are Mesa's 4.2 code) is drawn on 7.1 as a `gpu_render` batch.

- **The apps' shaders** (v61 `gpu_program`): `user/Libs/v3d/qpu.h` translates the programs written for 4.2 as they are built, once `qpu::setVersion (qpu::versionOf (gpu_info's text))` says 7.1 (r0..r4 → rf 25..29, r5 → rf0, rf0 (W) → rf3, rf2 / rf3 moved; a program has rf0..rf24); the program carries `KAPI_GPU_P_V71` (`qpu::programFlags`). The GameCube's TEV (`gxtev`): 99.5 % of random 1..3-stage configurations fit, 52 % of 1..16-stage ones (the others: the emulator's software path) — run_qpu_test.sh.

16. The RAM: volume

Source: `kernel/sys/ramfs.cpp`, `kern/ramfs.h`; the stream: `CRamStream` (`sys/stream.cpp`); the kapi side: `sys/kapi.cpp`; host test `tools/tests/run_ramfs_test.sh`; Pi test `user/BinUtils/ramtest.c`.

`RAM:` is a volume in memory, reached by the **same file calls** as the card: `open` / `read` / `fsize` / `fsize64` / `seek` / `close`, `save_file`, `file_in` / `file_out` (streams, appending too: `cp`, `cat`, the shell's redirections), `opendir` / `readdir` / `closedir`, `mkdir`, `remove`, `rename` (within `RAM:`; across volumes −1, the caller copies then removes, as between `SD:` and `SD1:`), `chdir` (`cd RAM:/x`, relative paths there), and `vol_info` (v71). newlib's `fopen` / `fread` / `fwrite` / `remove` and newlib's `opendir` sit on these, so a program needs nothing new to use it. Its files live **until the Pi restarts**: never written anywhere, not tied to any app (an app that ends leaves its files; its open handles are closed, `RamFsOnProcessGone` from `IpcOnProcessGone`). Programs cannot be started from it (`exec`, `spawn`: the card only). Browser keeps its disk cache and its JS code cache there (`RAM:/jet/cache`, `RAM:/jet/jscache`, docs/08): no browsing data on the card, and no SD write freezing core 0.

The tree. Folders and files are `TNode` records on the kernel heap — one size, so a freed one is reused as is (Circle's heap keeps freed blocks by size, it never merges them): a name of up to 255 characters (**case-insensitive, case kept**, as on the card's FAT), the parent, the children (a list in creation order: `readdir`'s order), the open count (`nRefs`). A file removed while open leaves its folder at once and is freed at its last close (it can still be read to its end); a folder must be empty to be removed; a folder cannot be moved into itself; `rename` onto an existing name fails (as FatFs). Lookups walk the path's names (a folder's children one by one: fine for the few thousand files of a cache).

The bytes are in **64 KB pages of the page allocator** (`palloc_high`: the apps' pool, the high zone; without one — a 1 GB Pi — the low pager), never in the kernel's BSS (the 2 MB image limit) nor in the heap. A file is a list of **extents**: a whole page, or a **chunk** of 256 B … 32 KB (8 sizes, powers of two) of a page cut into chunks of one size (`TSlab`: the page, a bitmap of its chunks; each size keeps the pages that have a free chunk). Every extent but the last is full. A file written whole (`save_file`) is cut exactly: whole pages, then the rest in falling powers of two (40 KB + 100 B → 32 KB, 8 KB, 256 B: under 256 B lost a file). A stream writes whole pages; at its close its **tail** (the chunks after its last whole page, and that page) is cut again exactly (`Trim`) — a log appended a line at a time stays compact. The extent list itself is in a chunk (a page holds ~2700 extents: 170 MB; a file is limited to `RAMFS_FILE_MAX`, 128 MB, and to the volume). A chunk freed gives its page back as soon as nothing else in it is used: removing a file returns its memory (`vol_info` shows it, `ramtest` checks it).

Limits (`kern/ramfs.h`):

The volume's size	`SD:/etc/system.ini` `ramfs=`: megabytes

	(`ramfs=64`) or a share of the page memory free at boot (`ramfs=10%`, at most 90 %); `ramfs=0` (or under 1 MB): no `RAM:` volume (`vol_info` −1, its paths fail). **Default** (no line): **128 MB, at most a quarter** of the free page memory (a 1 GB Pi: ~60 MB). Never more than the free memory less the reserve. Read at boot (a restart applies a change).
The reserve	`RAMFS_RESERVE` = 32 MB of page memory always left to the apps: a write that would go below it fails (the volume is then "full" before its size).
Files and folders	`RAMFS_MAX_NODES` = 16384 (their records: ~250 bytes of heap each); a new record also needs 8 MB of heap never handed out (Circle panics when its heap runs out).
A file	`RAMFS_FILE_MAX` = 128 MB (and the volume's room).
Open handles	256 files, 64 folder listings (on the heap); streams are not counted.

A full volume: `save_file` fails (−1) and **leaves no half file** (a file that cannot fit is removed — the old content was already replaced); a stream's `write` answers the bytes it could store (−1 for none). The boot log says the size: `ramfs: RAM: volume, up to 128 MB (2900 MB of page memory free)`.

Locking. One re-entrant **sleeping lock** for the volume (an atomic owner, its waiters `Yield` — as the FatFs volume lock, §5); nothing yields while holding it. A big read or write is cut into **1 MB slices** (`RAMFS_SLICE`) with a `Yield` between them, the lock free meanwhile — a 16 MB copy does not hold the other tasks (the kernel is not preempted, §5). A `save_file` in progress is in a **no-kill** section (`EnterNoKill`): killed between two slices, the task ends once the file is complete or removed. The file calls run on core 0's tasks (and core 3's with `netcore=1`); an app core never calls the kernel (§14). Opening a file / listing a folder walks the tree under the lock.

v75 (POSIX open files, §8 *v75: files and processes*). `file_open` on a `RAM:` path holds the file's node (`RamFsNodeOpen`, its `nRefs`): `RamFsPRead` / `RamFsPWrite` anywhere (overwriting in place, a gap past the end filled with zeros, `O_APPEND` at the end under the lock), `RamFsTruncate` (shrinking gives the extents back, growing writes zeros), `RamFsNodeStat` / `RamFsStat`, `RamFsUnlink` / `RamFsRenameEx` (replaces the target; an open one is unlinked and freed at its last close), `RamFsMkdirEx`, `RamFsUtime`, `RamFsReadDir2` (`dir_read`). Every node keeps a write time (UTC) and a number (stat's `ino`, unique, kept by a rename); `dev` 64, `blksize` 64 KB. A node's last close cuts its tail exactly (`Trim`), as a stream's.

Speed (no card): a `save_file` or a `read` is a `memcpy` into / from the pages (GBs a second), plus the yields between the slices; `ramtest` prints the time of a 16 MB file on the Pi.

Tests. On the PC: `sh tools/tests/run_ramfs_test.sh` — `sys/ramfs.cpp` built with `RAMFS_HOST_TEST` (the pages from the PC's allocator, no scheduler: its few platform calls are at the top of the file) under AddressSanitizer: thousands of random operations (saves, streams written / appended / truncated, reads whole / in pieces / after a seek, listings, mkdir, remove, rename across folders) checked against a model, for 5 seeds; then the edges: the names (case, 127 / 128 characters), a folder not empty, a folder into itself, a file removed while open, a dead process's handles, small files taking their size rounded to 256 B, a streamed file's tail fitted, a log appended 300 times, the volume filled (a save that does not fit leaves no half file, the room back after the removes), and every page given back at the end. On the Pi: `/bin/ramtest` (`ramtest full` also fills the volume).

---

17. GPIO: the 40-pin header (v92)

`kernel/sys/gpio.cpp` (`kern/gpio.h`) gives the programs the Raspberry Pi's header through one table entry, `gpio_ctl` (§8 *v92*), over Circle's `CGPIOPin`, `CGPIOManager`, `CI2CMaster` and `CSPIMaster`. The programs use it through **GPIOKit** (`SD:/lib/gpiokit.so`, docs/06 *GPIOKit*, docs/19), the only code that calls it.

- **Pins** are BCM GPIO numbers 0..27 (the header's). Each has **one owner** (a pid): the first call that needs the pin takes it (a mode, PWM, a bus); `KAPI_GPIO_M_FREE`, `KAPI_GPIO_RELEASE` or the process's end give it back — `GpioOnProcessGone`, called by `IpcOnProcessGone` when the process is reaped, after a crash too: its pins become **inputs with no pull** again (nothing left driving a wire), its PWM channels stop, its buses close, its edge queue goes. Reading a level is anyone's.
- **Reserved, never given** (`EPERM`, `KAPI_GPIO_F_RESERVED` with the reason in `kapi_gpio_pin.reason`): GPIO 0 / 1 (the HAT's ID EEPROM) and 14 / 15 (the kernel's serial console, `CSerialDevice`). The other pins the system uses are off the header and out of `gpio_ctl`'s reach: 30..33 (the Bluetooth UART, docs/BLUETOOTH-AUDIO-STUDY.md), 34..39 / 48..53 (the SD card, the Wi-Fi's SDIO), 40 / 41 (the jack's PWM audio), 42 (the activity LED).
- **PWM** — the header's PWM pins are PWM0's (12 and 18 its channel 1, 13 and 19 its channel 2: one pin of each pair at a time, `EBUSY`); the jack's sound is PWM1's (`sys/sound.cpp`, GPIO 40 / 41). The two blocks share **one clock**, which the sound sets to 125 MHz and **stops** when the jack's output stops. So the header's PWM never changes the clock's rate: it uses the sound's 125 MHz (mark-space mode, range = 125 MHz / frequency, data = range × duty / 10000), starts the clock at that rate when it is not running, and `OutputStop` (`sound.cpp`) calls `GpioPwmClockKeep` after it stopped the jack. When the jack starts while a header PWM runs, the clock restarts at the same rate: a glitch of a few microseconds on the header's wave.
- **Edges** — an input of the caller's: `KAPI_GPIO_EDGES` makes the GPIO interrupt's manager (`CGPIOManager`, created at the first need) call `EdgeIRQ` on both edges of the pin; the level just after says which edge it was, and only the asked ones are queued, with `CTimer::GetClockTicks64 ()` (µs), for the pin's owner: up to 8 processes, 256 events each (full: the newest are dropped and the next event has `lost` = 1). `KAPI_GPIO_EVENTS` takes them, waiting up to 1 s (2 ms naps). No debouncing: a button gives a few edges a press.
- **I2C bus 1** (GPIO 2 SDA / 3 SCL; the board's 1.8 kΩ pull-ups) and **SPI 0** (GPIO 7 CE1, 8 CE0, 9 MISO, 10 MOSI, 11 SCLK) — one process at a time, their pins taken with them (`KAPI_GPIO_M_I2C` / `_SPI`). A transfer is copied through a kernel buffer (4096 bytes at most each way); an I2C write then read of at most 16 written bytes is one transaction with a repeated start (`CI2CMaster::WriteReadRepeatedStart`). The scan reads one byte from each address 0x08..0x77.

Not done: the alternate functions of pins beyond what `KAPI_GPIO_M_ALT0 + n` sets by hand, I2C bus 0, SPI 1 (the AUX SPI), the UARTs 2..5, a debounce, a pin's drive strength, the Pi 5's PWM (the RP1's PWM0: Circle's `pwmoutput-rp1.h`, one range for its channels). **The Pi 5** (2026-10-09): the same code over Circle's RP1 classes (`gpiopin2712`, `i2cmaster-rp1`, `spimaster-rp1`, `CGPIOPin::ReadAll` for the levels); its I2C has no repeated start (a write then a read, two transactions); `KAPI_GPIO_PWM` → `-KAPI_ENODEV`. Not run on a Pi 5. **Tested on the PC only** (GPIOKit's simulator; docs/HANDOFF.md lists what the Pi must check).

---

18. The volumes: USB sticks, eject, format

Source: `kernel/sys/volume.cpp`, `kern/volume.h`; the hooks in `sys/ofile.cpp` (`OFileVolume`), `sys/kapi.cpp`, `sys/handle.cpp`, `sys/stream.cpp`, `kernel.cpp`; the Circle fork's side: docs/05 §28. Host test: `tools/tests/run_fs_test.sh` (`fs/usbtest.cpp`). kapi v93 (§8).

Who mounts what. `SD:` is mounted by `CKernel::Initialize` as before; `VolMountCard` then mounts the card's partitions 2–4 as `SD1:`..`SD3:` (they were mounted in `kernel.cpp` until v93). The USB mass-storage devices are Circle's `umsd1`..`umsd3` (numbered in the order they come: a number freed by an unplug is given to the next device), physical drives 1–3. **The names (the user's choice, 2026-10-06)**:

- **`USB1:`** (`USB2:`, `USB3:`) — device n **whole**, when it has **one partition or none**: `VolToPart {n, 0}`, FatFs' auto search (a "superfloppy" — a file system from sector 0 — or its only MBR partition);
- **`USB1P1:`, `USB1P2:`** … `USB1P4:` — its MBR partitions when it has **several** (`{n, m}`): each FAT / exFAT one mounted, the others listed `UNREADABLE` (each can be formatted alone);
- **`USB:`** is an alias of `USB1:` (as `SD0:` is of `SD:`): `ResolvePath` writes it `USB1:`, the program images' keys too (`ImageCanonPath`), `vol_*` accept it.

`UsbLayout` reads the device's sector 0 (under its lock): a FAT / exFAT boot sector (a jump, `FAT` / `FAT32` / `EXFAT` at their places, `55 AA`) → one volume; else an MBR (`55 AA`) with two or more entries used (a type and a start) → the partitions; else one volume (an empty device: `UNREADABLE`, to format). FatFs' volumes: `FF_VOLUMES 21` — `SD`, `SD1`..`SD3` (0–3), `USB1`, `USB1P1`..`P4` (4–8), `USB2`.. (9–13), `USB3`.. (14–18), `FD`, `NVME` (docs/05 §28; `kapi_stat`'s `dev` is the volume's number + 1). **One FatFs lock a physical drive** (`fslock.cpp`'s `LockSlot` = `VolToPart[vol].pd`): the card's volumes share one, a USB device's another — its driver yields between transfers, and `diskio.cpp`'s bounce buffer and sector cache are per drive. No logical partitions (FatFs reads the four primary ones), no GPT (`FF_LBA64` is off: it would change `LBA_t` and every `FATFS`; Circle's driver stops at 2 TB anyway).

Hot plug. `VolPoll` runs every 100 ms in the kernel's input task, right after Circle's `UpdatePlugAndPlay` (beside `SoundPoll`, which follows a USB audio device the same way). A device found under `umsdN` gets a removed handler of ours (it only sets a flag: Circle calls it from the device's destructor, inside `UpdatePlugAndPlay`), then scanned and its volumes mounted (`f_mount (.., 1)`; tried 3 times a second apart when it does not answer at once). No FAT / exFAT: `UNREADABLE` (it can be formatted); no answer: `UNREADABLE` + `KAPI_VF_IOERR`. Each change bumps the volume's `gen`; the kernel logs it (`volume: USB1: mounted (exFAT, 15264 MB, KINGSTON)`; `USB1: several partitions: USB1P1..P4`).

A stick pulled out without an eject. Nothing in the kernel waits for it:

- the kernel is not preempted and Circle's USB driver waits for a transfer in a busy loop (`NO_BUSY_WAIT` is off), so the device is deleted (in the input task) only **between** two transfers. `diskio.cpp`'s own removed handler clears its device: every later transfer fails (`RES_NOTRDY`), and

the FatFs call in flight ends with an error (a transfer that was running when the stick left times out, ≤ 3 s, in Circle's xHCI driver);
- since v93 the FatFs disk layer yields between two transfers of a USB drive once the task has run 10 ms (`OnyxDriverPoll`, docs/05 §28) — a long operation (a format, a folder of thousands of files) no longer holds core 0 — which is safe for the same reason: the volume lock is held, a stick gone only fails the next transfer;
- `VolPoll` sees the flag and, for each volume of the device: `OFileVolume (vol, OFV_DROP)` closes the open-file layer's files of that volume (their node is **lost**: every later call on it is `-EIO`, the program keeps its descriptor until it closes it; the node's path is cleared so that a new open — of the next stick — never shares it), `ImageFileChanged ("USB1:/")` forgets the programs' images read from it, then — **holding the volume lock**, so after the FatFs call in flight — the volume is unregistered (`f_mount (0, ..)`). The other FatFs objects of the volume (a read handle of `kapi_open`, a folder, a stream) are invalid from then on: FatFs' `validate` sees `fs_type` 0, or another mount's `id` once a stick is mounted again in the same `FATFS`. Our fork tests that **again once the volume lock is held** (a task that waited for the lock while another unmounted the volume) and keeps the lock usable across an unmount (docs/05 §28). The state becomes `REMOVED` with `KAPI_VF_UNSAFE`;
- the volume lock is never left held: its holder is in a no-kill section (§5, `fslock.cpp`) and only waits for transfers that fail. The processes go on.

Eject (`vol_eject`) acts on the **device** — any of its volumes names it (`USB1P2:` ejects `USB1P1:` too): the open-file layer's written files synced (`OFV_SYNC`: `f_sync` writes the data, the FAT, the entry, FSINFO), then, if nothing is open (the open-file nodes + the other FatFs objects, which `VolTrack` / `VolUntrack` count: `kapi_open`'s read handles, `kapi_opendir`'s folders, `CFileStream`s, `kapi_save` while it writes — on any of its volumes), its volumes are unmounted as above and the device's cache flushed (`CTRL_SYNC` → the fork's `CUSBBulkOnlyMassStorageDevice::IOCtl`: SCSI SYNCHRONIZE CACHE (10); a stick without a cache may refuse it, ignored). `EJECTED` until it is pulled out (then `REMOVED`, without `UNSAFE`) or mounted again (`vol_mount`). With files open: `-EBUSY` (they were synced: pulling it out then loses nothing written so far), or `KAPI_EJECT_FORCE`. The sector cache is write-through (docs/05 §8): nothing else is held in memory. A program's current folder on the stick does not make it busy (its next calls fail).

Format (`vol_format`): `SD:` is refused here whatever the caller says (the system's volume); `SD1:`..`SD3:` need `KAPI_FMT_CARD` (the UI asks twice); a USB volume needs its device. **`USB1:`** formats the **whole device** whatever it had (its partitions' volumes unmounted and forgotten: it becomes one volume, `USB1:`); **`USB1P2:`** only that partition (one the device has now). The volume's files are dropped and it is unmounted, then **under the drive's lock** (the whole card, or the whole device, waits meanwhile) `f_mkfs` (`FF_USE_MKFS`, the fork): on `USBn:` the whole device — an MBR with one partition from sector 63, as Windows does it (`FatFs` aligns the data to 1 MB: `align` 2048 sectors on a device of 256 MB or more), two FATs on FAT / FAT32 —, on `USBnPm:` and `SD1:`..`SD3:` **inside their MBR partition** (FatFs sets the partition's type). A 128 KB work buffer; the label by `f_setlabel` once mounted (`FF_USE_LABEL`, checked first: 11 characters, none of `"*+,./:;<=>?[\]|`). The labels are read at each mount (`f_getlabel`) and kept.

What runs where. `kapi_shutdown` calls `VolSyncAll` first (every written file of every volume synced, the USB caches flushed). `kapi_vol_info` (v71) and `ResolvePath` know `USB1:`, `USB1P2:` like any volume; `ResolvePath` turns `USB:` into `USB1:`. Everything that takes a path works on a stick: the `/bin` tools (`cp SD:/x USB1:/`, `cd USB:`), the File Viewer, the file dialogs (they list `USB1:`..`USB3P4:`, shown when mounted), Photos (`USBn:/DCIM`, `USBnP1:/DCIM`, `USBnP2:/DCIM`).

The user's side. The menu bar shows a drive icon while a stick is there; its box lists each volume (a device of several partitions: a row each, one Eject button for the device) and opens Disks; it polls `vol_list` every second and tells notifyd what changed (connected — a click opens it in the File Viewer —, can be removed safely, not formatted — a click opens Disks —, removed without an eject). Disks (`user/Apps/disks`): the volumes, Open / Eject / Mount, Format (file system, label; on a partition, "Whole device" formats `USBn:` instead). The File Viewer: the sticks in its sidebar (followed every second), Go > USB Stick / Eject USB Stick (Ctrl+E) / Disks. `/bin/mount`, `/bin/eject`, `/bin/mkfs`, `/bin/df` (docs/04).

Several partitions on one USB disk: built (2026-10-06)

Built as above (the names `USBn` / `USBnPm`: the user's choice). The proposal's other points, kept for later: **GPT** (a Mac's disks, disks over 2 TB) needs `FF_LBA64` (a full rebuild: `LBA_t` 64-bit, `FIL` / `FATFS` grow — wlan and wpa_supplicant too, docs/05 §13); **logical partitions** (an MBR's extended one) are not read by FatFs; a device with several partitions **and** a FAT superfloppy is not possible. `vol_list` gives each volume its `device` (`umsd1`): what a program groups by.

Annex — useful constants

Constant	Value	File
`KPAGE_SIZE`	64 KB	layout.h
`L2_BLOCK_SIZE`	512 MB	layout.h
`USER_VA_BASE`	8 GB	layout.h
`USER_WINDOW_CANVAS`	12 GB	layout.h
`USER_WALLPAPER_CANVAS`	13 GB	layout.h
`KAPI_TABLE_VA`	14 GB (the EL0 table)	kapi_abi.h
`KAPI_STUBS_VA`	14 GB + 64 KB (the EL0 code page: stubs, then the blob at +8 KB)	el0.h
`USER_STACK_TOP`	16 GB	layout.h
`EL0_USTACK_MIN`	8 MB (an app's user stack, lazy since v75; `app.txt stack`, 8–64 MB)	el0.h
`USER_MMAP_BASE` .. `USER_MMAP_END`	34 GB .. 60 GB (the mmap arena, v75)	layout.h
`USER_THREAD_STACKS`	32 GB (+ 32 MB a thread)	el0.h
`KAPI_ABI_VERSION`	91	kapi_abi.h
`RAM:` volume	128 MB by default (≤ ¼ of the free page memory; `system.ini ramfs=`), 32 MB reserve, 16384 files + folders, 128 MB a file	ramfs.h
`USER_HEAP_BASE`	10 GB	layout.h
Tasks	no limit (a linked list, §5;	scheduler.h

Constant	Value	File
	Circle's `MAX_TASKS` is not used)	
`ASID`	8 bits (1..255; 0 = kernel)	layout.h
Kernel stack of an app task (`EL0_KSTACK_SIZE`)	256 KB	el0.h
Screen resolution	1024×768 by default (`cmdline.txt` `width=` / `height=`; changed while running: `screen_set`, v66)	window.h / cmdline.txt
`GIMAGE_TRANSPARENT`	`0xFF00FF`	gimage.h